Concurrency(동시성) 프로그래밍

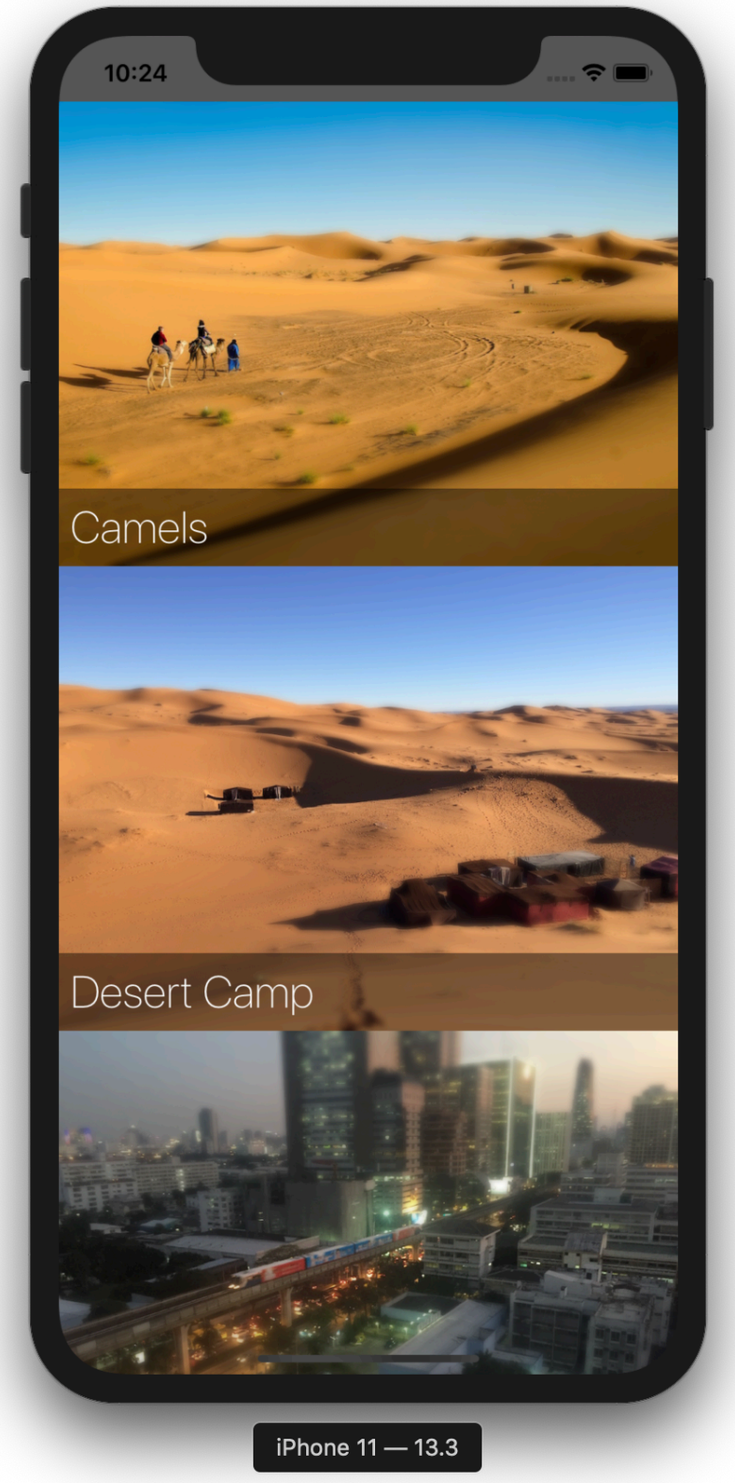
- iOS에서의 비동기 처리, **GCD/Operation**에 관하여 -

강의주소: https://www.inflearn.com/course/iOS-Concurrency-GCD-Operation#

왜 필요할까?

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

예제 **폴더 1.TiltShiftDemoExample**



스크롤 할때마다,
버벅임

아이폰의 화면은 “16밀리 초”마다 리프레싱

예제 출처: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

성능 / 반응성

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

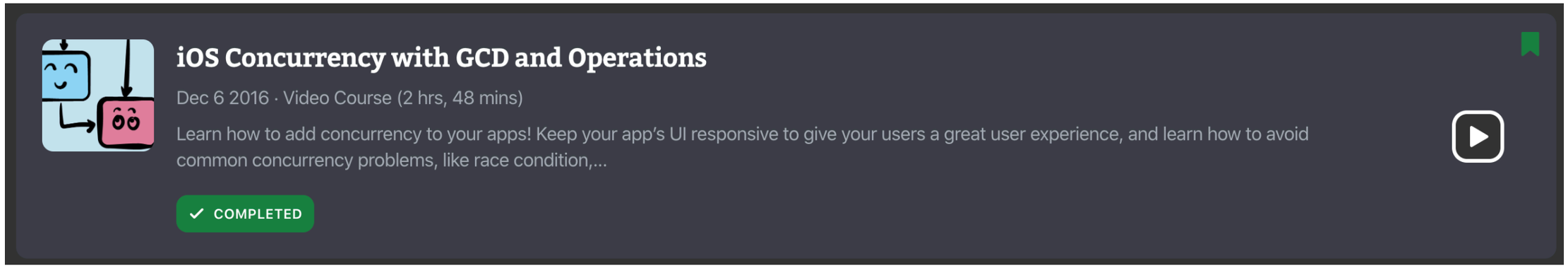
최적화

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

< 본 강의의 참고 자료 및 예제 코드 등 출처 >

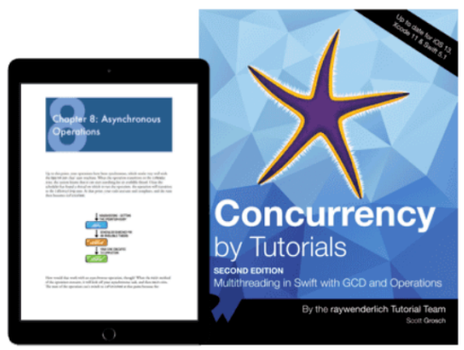
(1) 동영상 강의 참고 www.raywenderlich.com

주소: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations



(2) 서적 참고 www.raywenderlich.com

주소: https://store.raywenderlich.com/products/concurrency-by-tutorials



Concurrency by Tutorials

$39.99

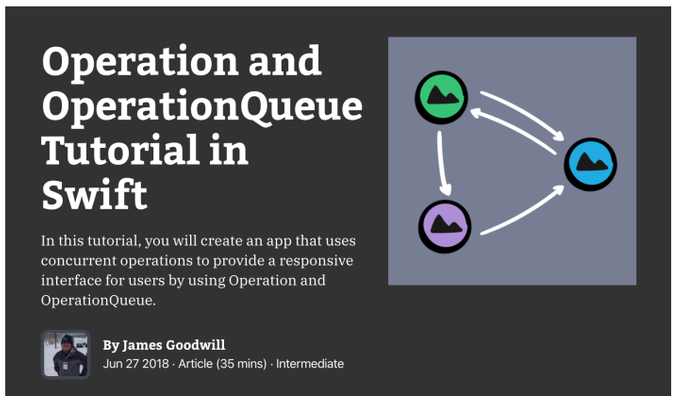
The book that teaches you everything there is to know about how to write performant and concurrent code for your iOS apps.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

< 본 강의의 참고 자료 및 예제 코드 등 출처 >

(3) 아티클 참고 www.raywenderlich.com

주소: https://www.raywenderlich.com/5293-operation-and-operationqueue-tutorial-in-swift



(4) 플러스 알파

(이해하기 쉽게) 모든 구조 그림표현/설명 + 코드들의 재배치 + 실제 주로 어떤 상황에서 쓸 수 있을지 상황예시

(반드시 본 강의의 출처를 남겨주시길 부탁)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

강의 목표:

GCD / Operation과 관련된 어떤 주제도 A ~ Z까지 완벽 정리

강의주소: https://www.inflearn.com/course/iOS-Concurrency-GCD-Operation#

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

목차 (1부) - GCD

1. GCD/Operation에 앞서

1) 왜 동시성(Concurrent) 프로그래밍이 필요할까?

2) 간단한 GCD/Operation 소개

3) Synchronous(동기) VS Asynchronous(비동기)

4) Serial(직렬) VS Concurrent(동시)

2. 디스패치큐(GCD)의 종류와 특성

1) 메인 큐

2) 글로벌큐와 Qos의 이해

3) 프라이빗(커스텀) 큐

3. 디스패치큐(GCD) 사용시 주의해야할 사항

1) 반드시 메인큐에서 처리해야하는 작업

2) sync 메서드에 대한 주의사항

3) weak, strong캡처 주의

4) 컴플리션핸들러의 존재이유

5) 동기적 함수를 비동기함수 처럼 만드는 방법

4. 디스패치 그룹

1) 디스패치 그룹의 개념

2) 디스패치 그룹의 사용
- 디스패치 그룹과 동기함수
- 디스패치 그룹과 비동기함수

3) (참고) Dispatch WorkItem

4) (심화) Semaphore의 이해

5. (참고) GCD 프로젝트 살펴보기

6. (심화) 동시성과 관련된 문제

1) Race Condition (경쟁상황)

2) Deadlocks (교착상태)

3) Priority Inversion (우선 순위의 뒤바뀜)

7. (심화) Thread-safe한 코드의 구현과 방법

1) TSan 사용법

2) 시리얼큐와 sync

3) 디스패치 배리어(Dispatch Barrier)

4) 객체 설계 시 주의할 점

8. (심화) Lazy var와 관련된 이슈들

목차 (2부) - Operation

1. Operation에 앞서

1) GCD와 Operation의 차이

2) 언제 Operation(오퍼레이션)이 필요할까?

3) 들어가기에 앞서

2. Operation / OperationQueue

1) 오퍼레이션(Operation)

2) OperationQueue (오퍼레이션큐)

3) BlockOperation (블락오퍼레이션)

3. AsyncOperation (비동기 오퍼레이션)

4. 오퍼레이션큐의 주요 기능

1) 순서 관리 (종속성 - Dependencies)

2) 오퍼레이션큐의 작업 취소

5. 오퍼레이션 프로젝트 살펴보기

6. 총정리 - GCD / Operation의 비교와 사용

GCD

강의주소: https://www.inflearn.com/course/iOS-Concurrency-GCD-Operation#

1. GCD/Operation에 앞서

---

1) 왜 동시성(Concurrent) 프로그래밍이 필요할까?

2) 간단한 GCD/Operation 소개

3) Synchronous(동기) VS Asynchronous(비동기)

4) Serial(직렬) VS Concurrent(동시)

1) 왜 동시성(Concurrent) 프로그래밍이 필요할까?

PC 사양

4코어 8쓰레드

8코어 16쓰레드

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

아이폰 11 프로

6코어

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

CPU 코어와 스레드(ft.하이퍼스레딩) 가장 쉬운 설명 듣고 가시죠 - [高지식] 거니

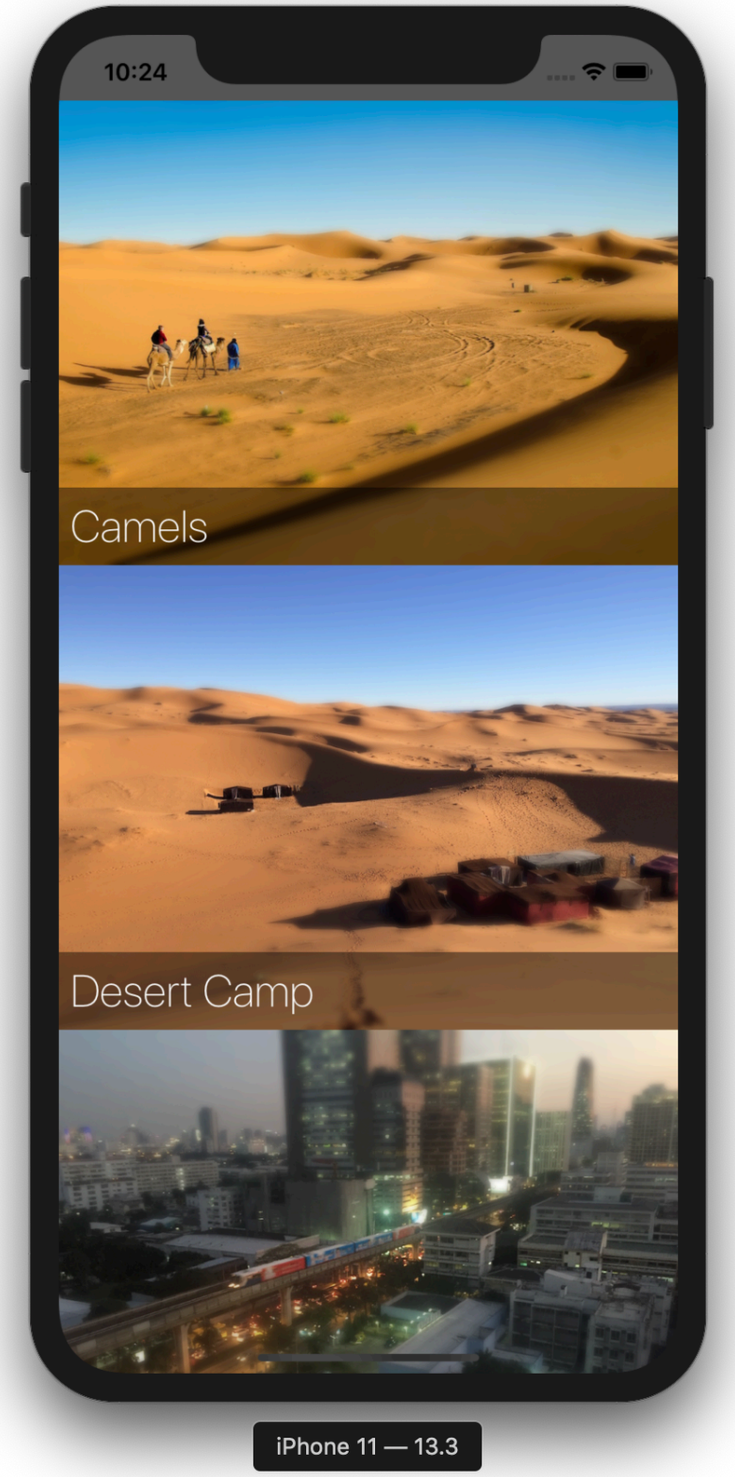
알고리즘 투게더 with 거니 • 조회수 18만회 • 12개월 전

이번 고지식 열네 번째 영상에서는 CPU를 구매할 때 제일 중요한 코어와 스레드에 대해서 얘기 합니다. 코어와 스레드가 각각 정확히 ...

유튜브: 알고리즘 투게더 with 거니 or 검색어: '스레드'

https://youtu.be/_dhLLWJNhwY

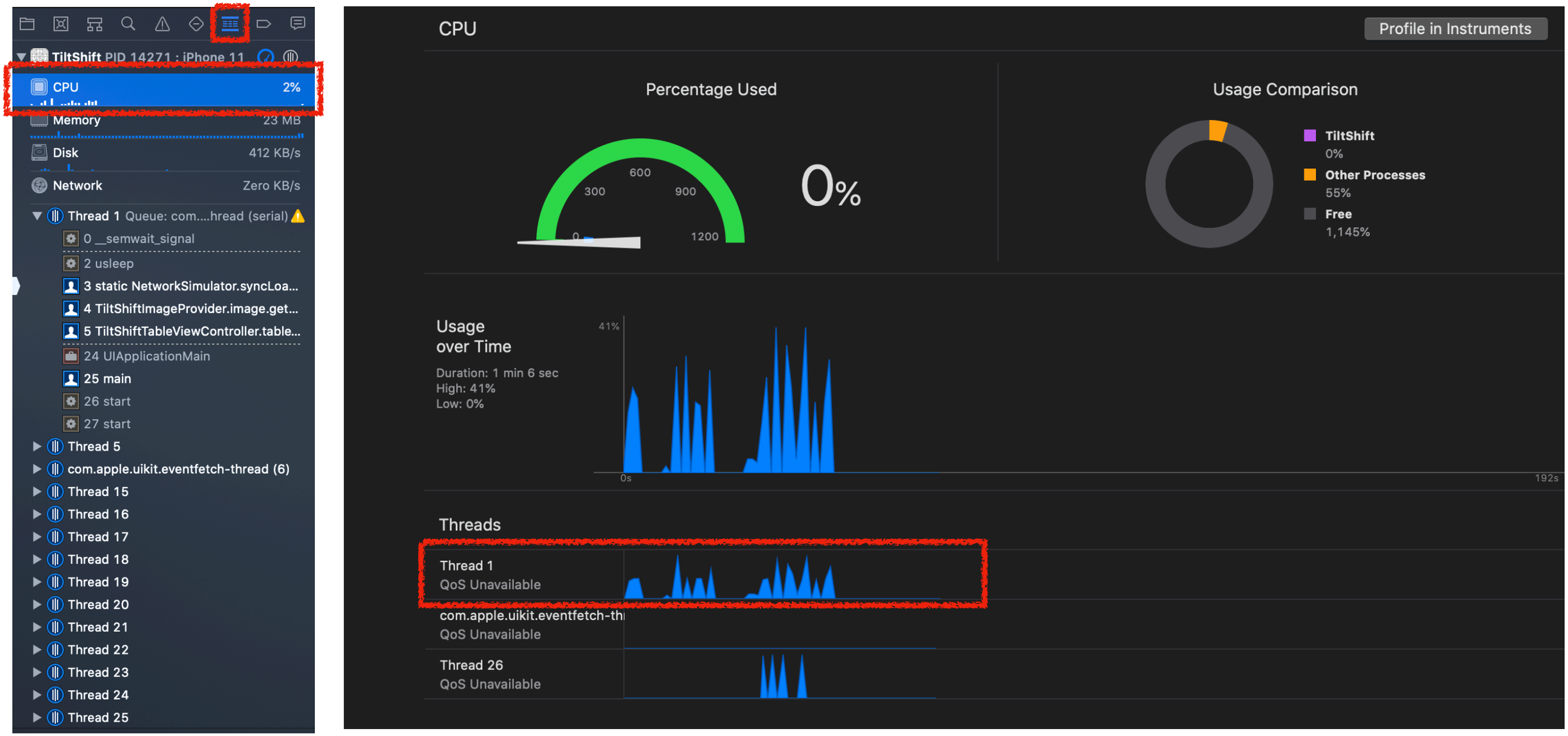
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com



버벅였던 이유를 좀더 자세하게

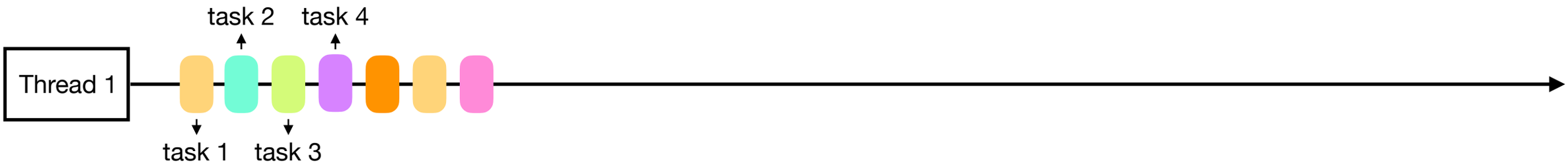
예제 출처: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com



command + 7

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

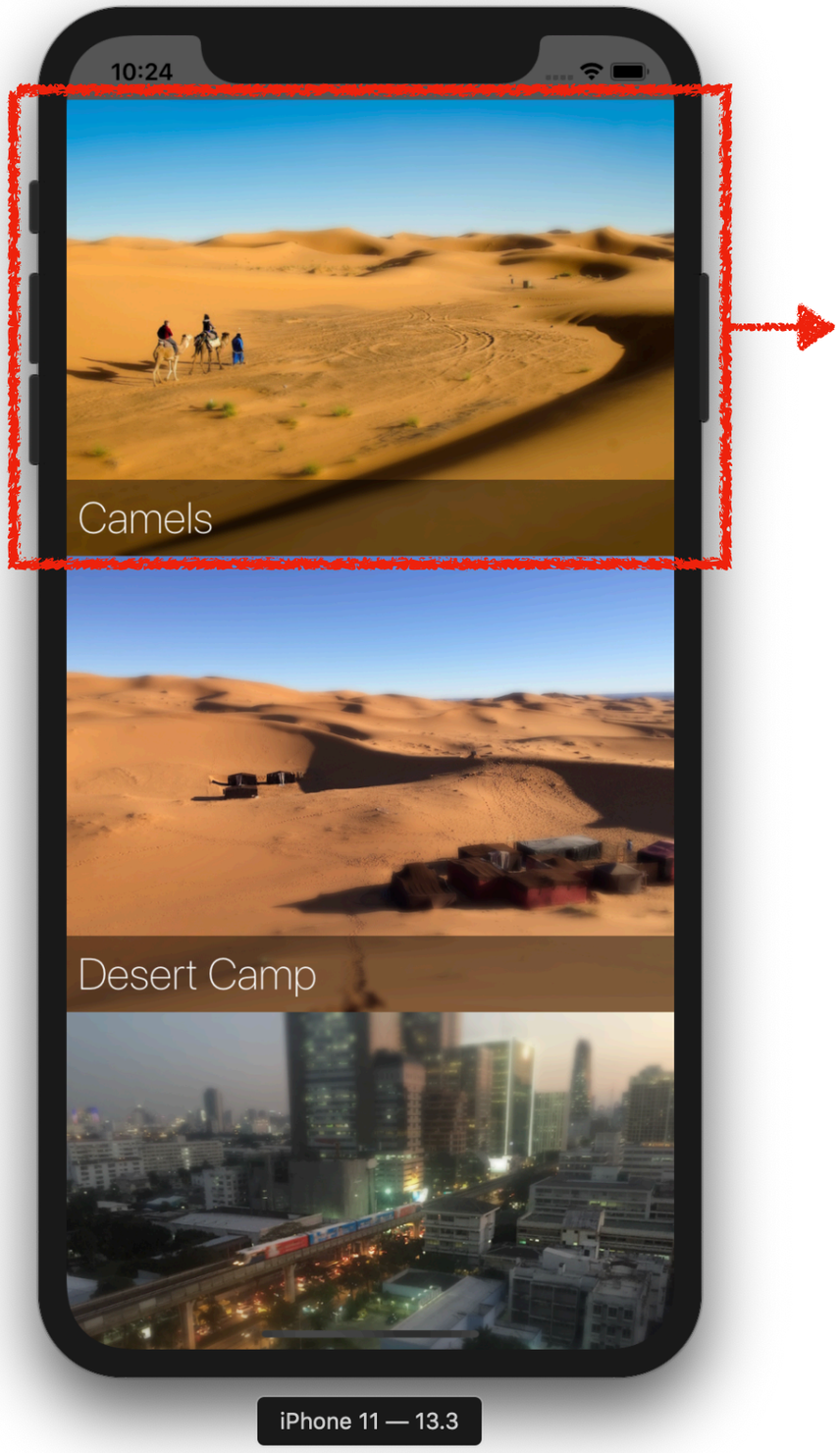



```
print(“task1: 출력하기”)
print(“task2: 출력하기”)
print(“task3: 출력하기”)
print(“task4: 출력하기”)
print(“task5: 출력하기”)
```

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

1.Preview



네트워크를 통해

(1)압축파일을 다운로드
(2)압축을 풀고
(3)이미지파일을 변형(블러처리)
(4)셀에 이미지 표시

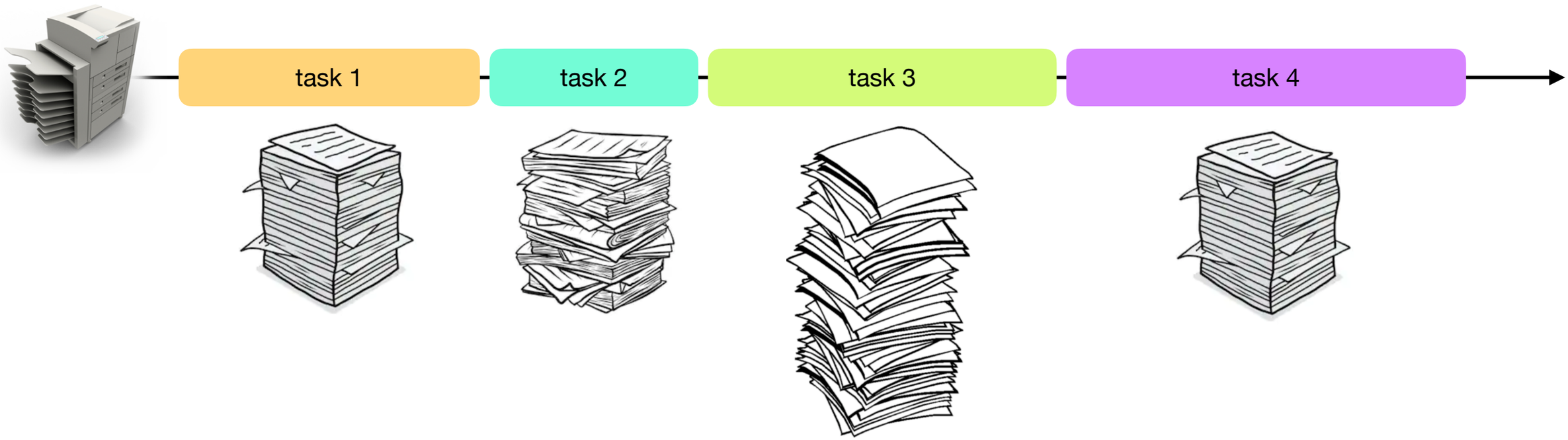
예제 출처: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

복사집
(실생활 예시)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실생활 예시



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Thread 1

어떻게 ?

Thread 2 task 1

Thread 3 task 2

Thread 4 task 3

Thread 5 task 4

Thread 6

Thread 7

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

작업(Task)을 어떻게 분산 시킬 수 있을까?

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

작업(Task)을 어떻게 다른 쓰레드에서 동시에 일을 하게 할 수 있을까?

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

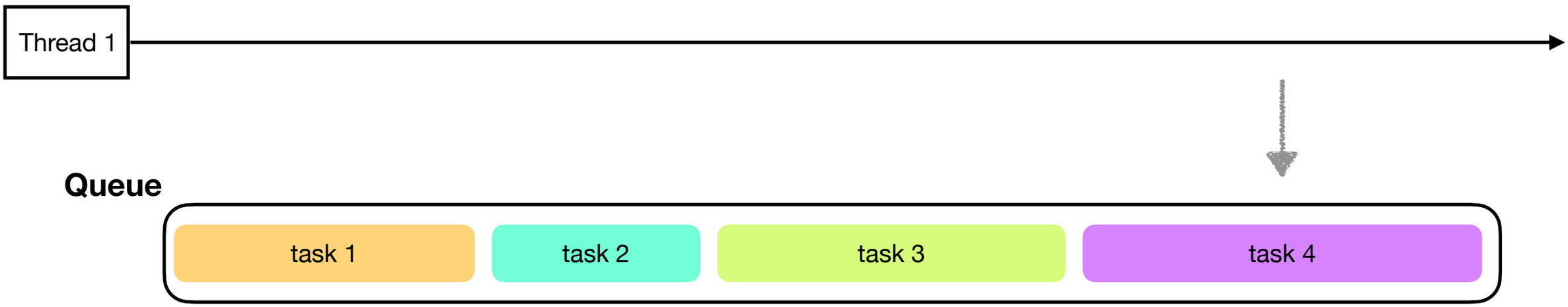
작업(Task)을 “대기행렬”에 보내기만 하면 된다.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

작업(Task)을 큐(Queue)에 보내기만 하면 된다.

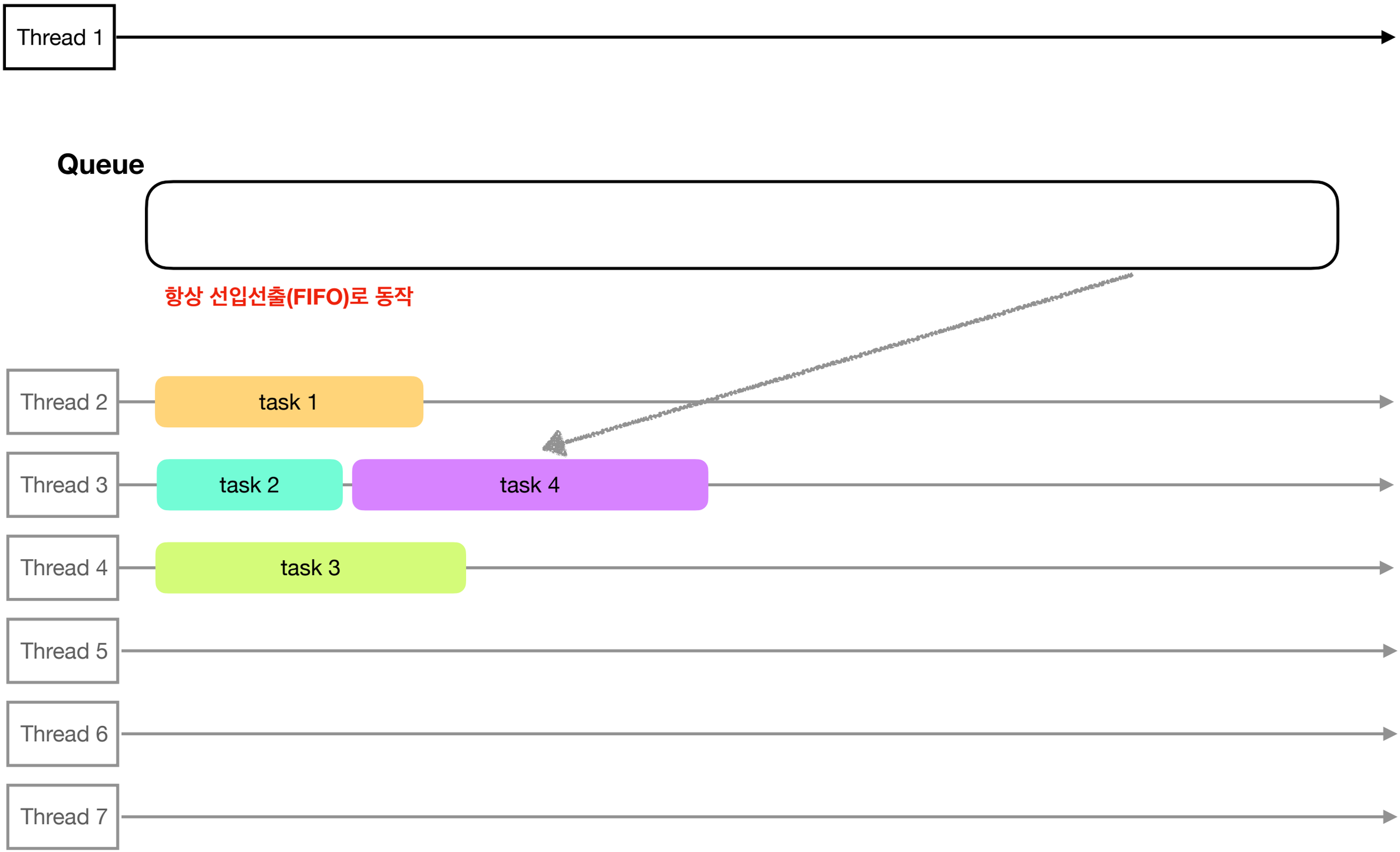
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

대기열 개념을 이용



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

대기열 개념을 이용



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

우리가 할일. 작업을 큐(Queue)로 보내자.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

2) 간단한 GCD/Operation 소개

iOS 프로그래밍

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

GCD / Operation

- 직접적으로 쓰레드를 관리하지 않고, **“큐(Queue)”(대기열/대기행렬)**라는 개념을 이용해 작업을 분산 처리

- GCD / Operation을 사용해 **시스템에서 알아서 쓰레드 숫자를 관리**함
  (직접쓰레드를 생성하는 것은 하드웨어나 일의 부하(load)와 같은 시스템에 대한 지식없이 사용하면, 오히려 앱이 느려질 수 있음)

- 쓰레드 보다 더 높은 레벨/차원에서 일을 한다고 보면됨

- 쉽게 다른 쓰레드에서 **(오래걸리는) 작업들이 “비동기적으로 동작”**하도록 만들어 줌
  (어떤 API들은 내부적으로 다른 쓰레드에서 비동기적으로 실행되도록 설계되어 있음)

(보통) 네트워크와 관련된

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

큐에 보낼꺼야 글로벌큐에 비동기적으로

```
DispatchQueue.global( ).async {
    클로저(Closure)를
}
```

코드예시

보낼 큐의 종류를 선언

```
let queue = DispatchQueue.global( )
```

(그 해당 큐에) 비동기적으로 보낼꺼야

```
queue.async {
```

클로저(Closure)를

```
}
```

코드예시

```
DispatchQueue.global( ).async {
    작업의 한 단위
}
```

```
DispatchQueue.global( ).async {
    작업의 한 단위
}
```

실제 코드예시

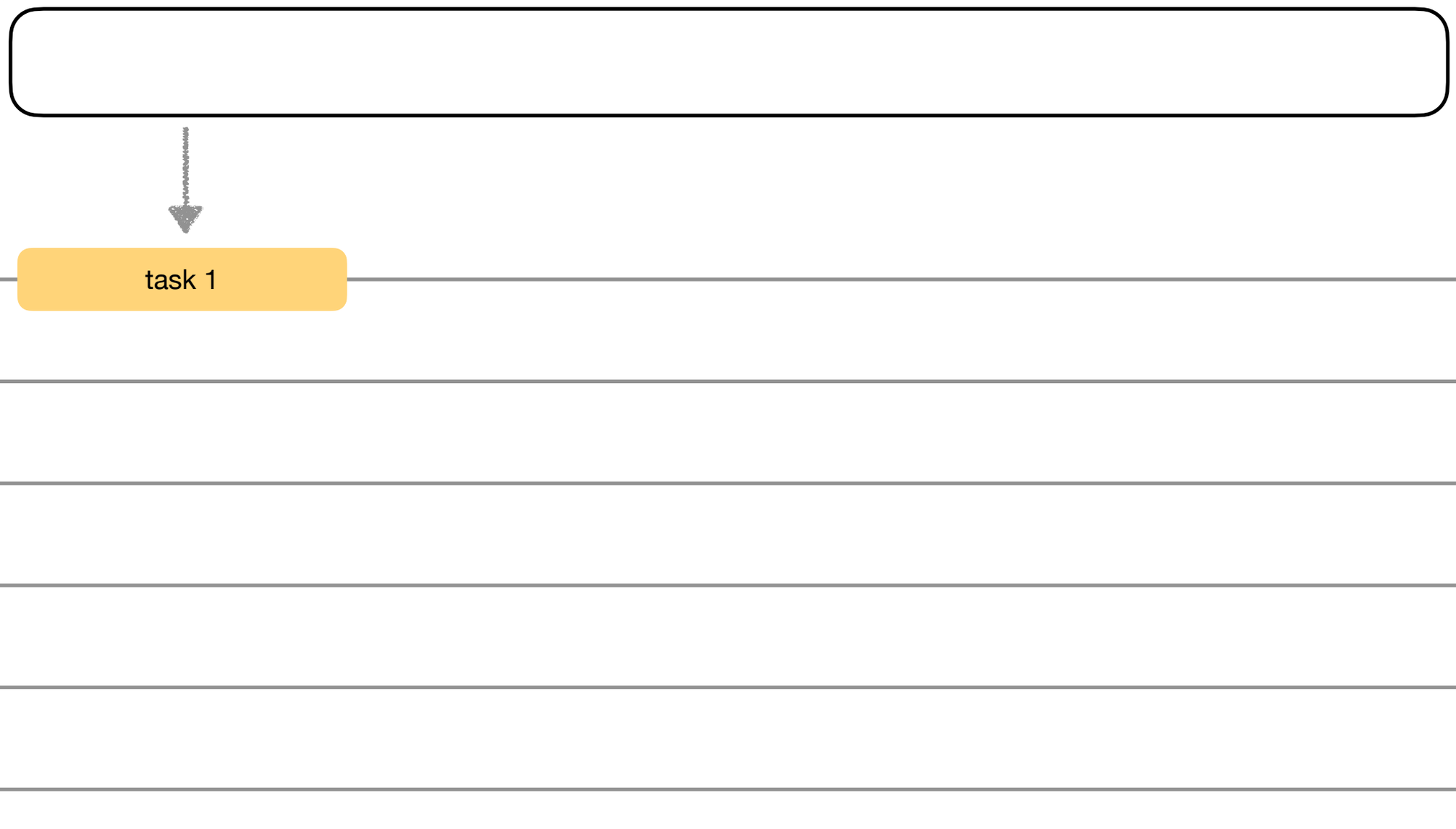
1-1.SimpleExample

이제 시스템이 알아서 보낸다.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

대기열 개념을 이용

Queue



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

GCD

- 간단한 일(커뮤니케이션의 양)
- 함수를 사용하는 작업
  (메서드 위주)

/

Operation

- 복잡한 일(커뮤니케이션의 양)
- 데이터와 기능을 캡슐화한 객체

기반

(+ 여러가지 기능)

취소 / 순서지정 / 일시중지 (상태추적)

프로젝트의 효율성, 사례 적합성 등

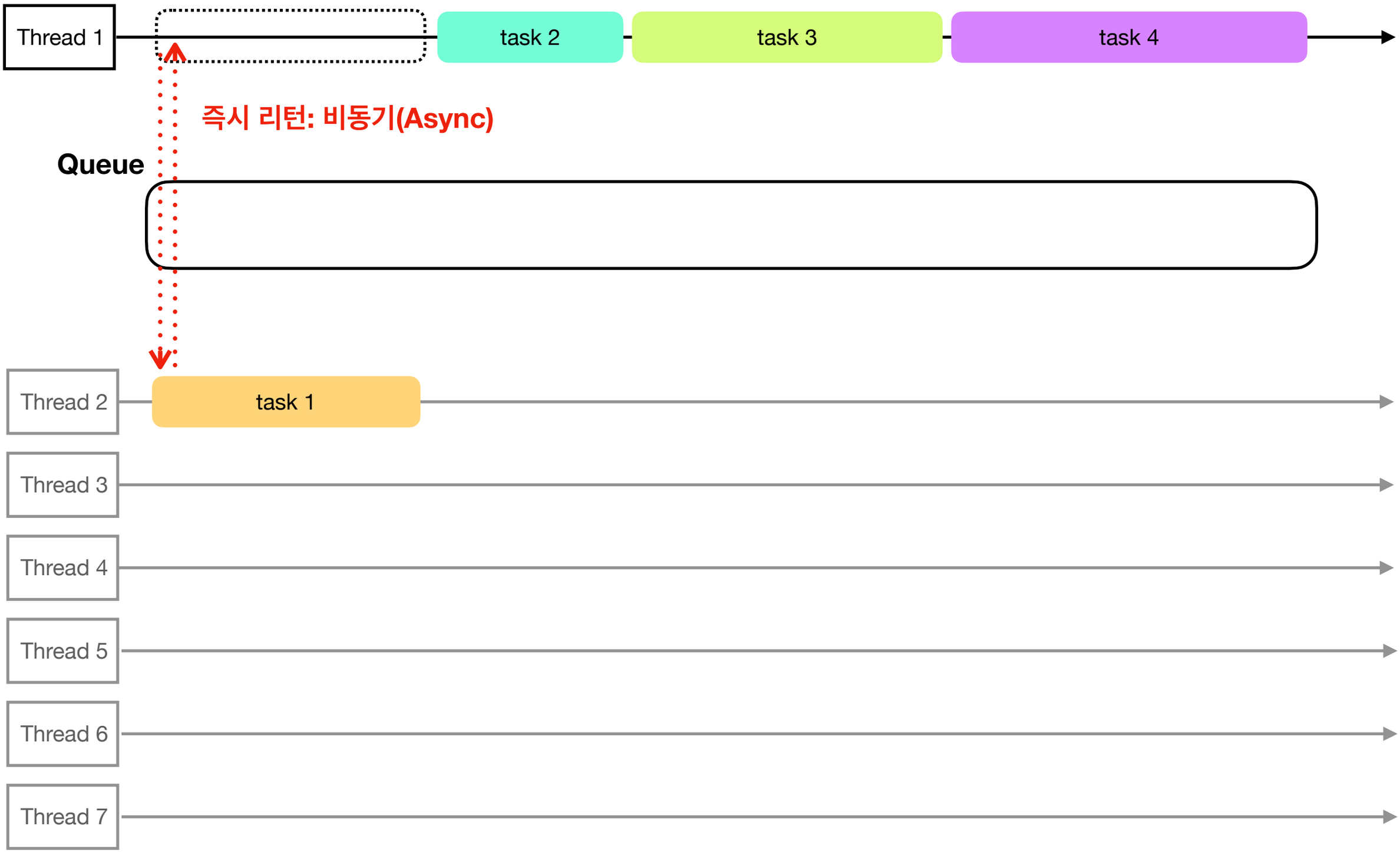
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

3) Synchronous(동기) VS Asynchronous(비동기)

동기(Sync) vs 비동기(Async) 차이 그림으로 이해하기

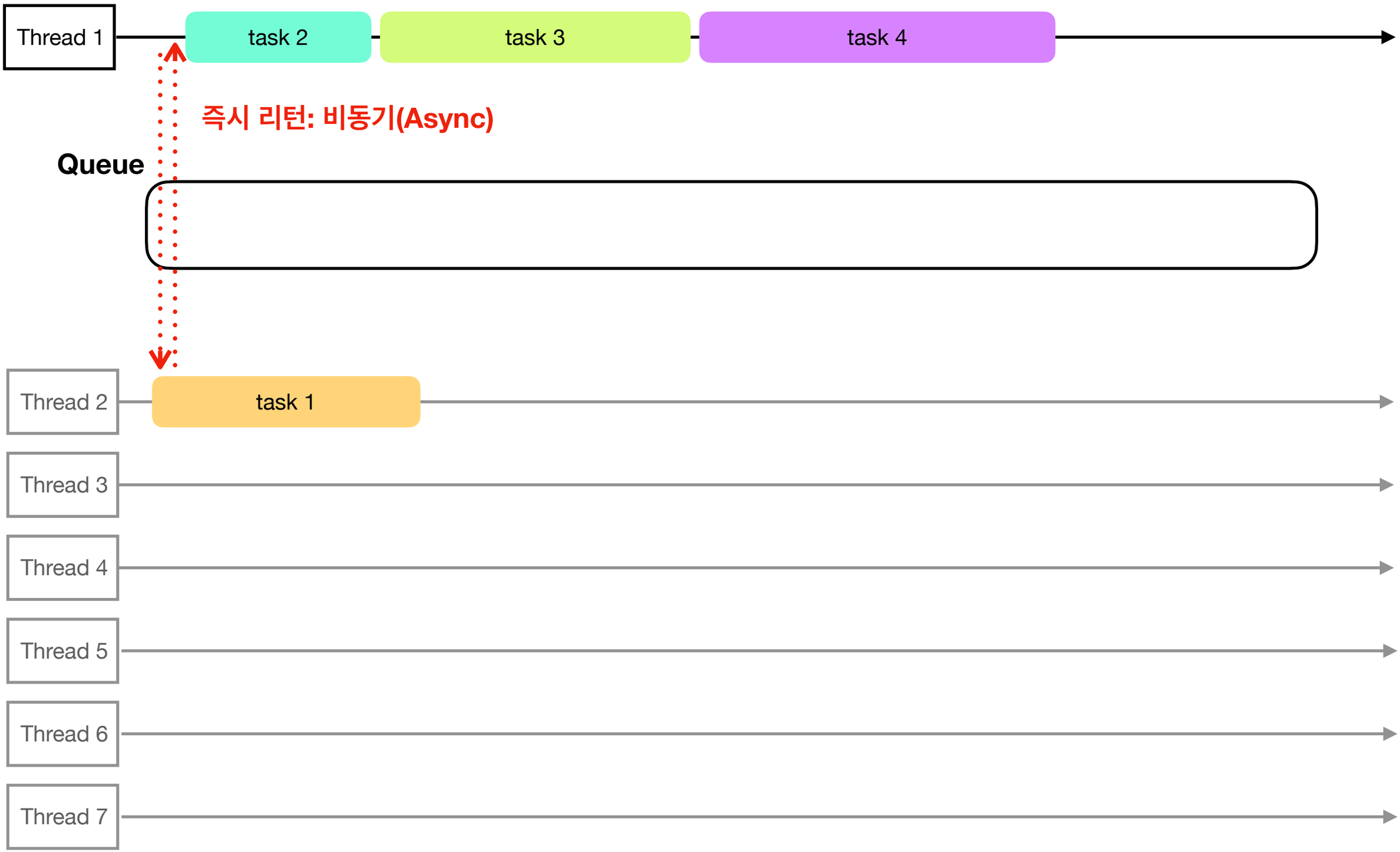
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

비동기 개념



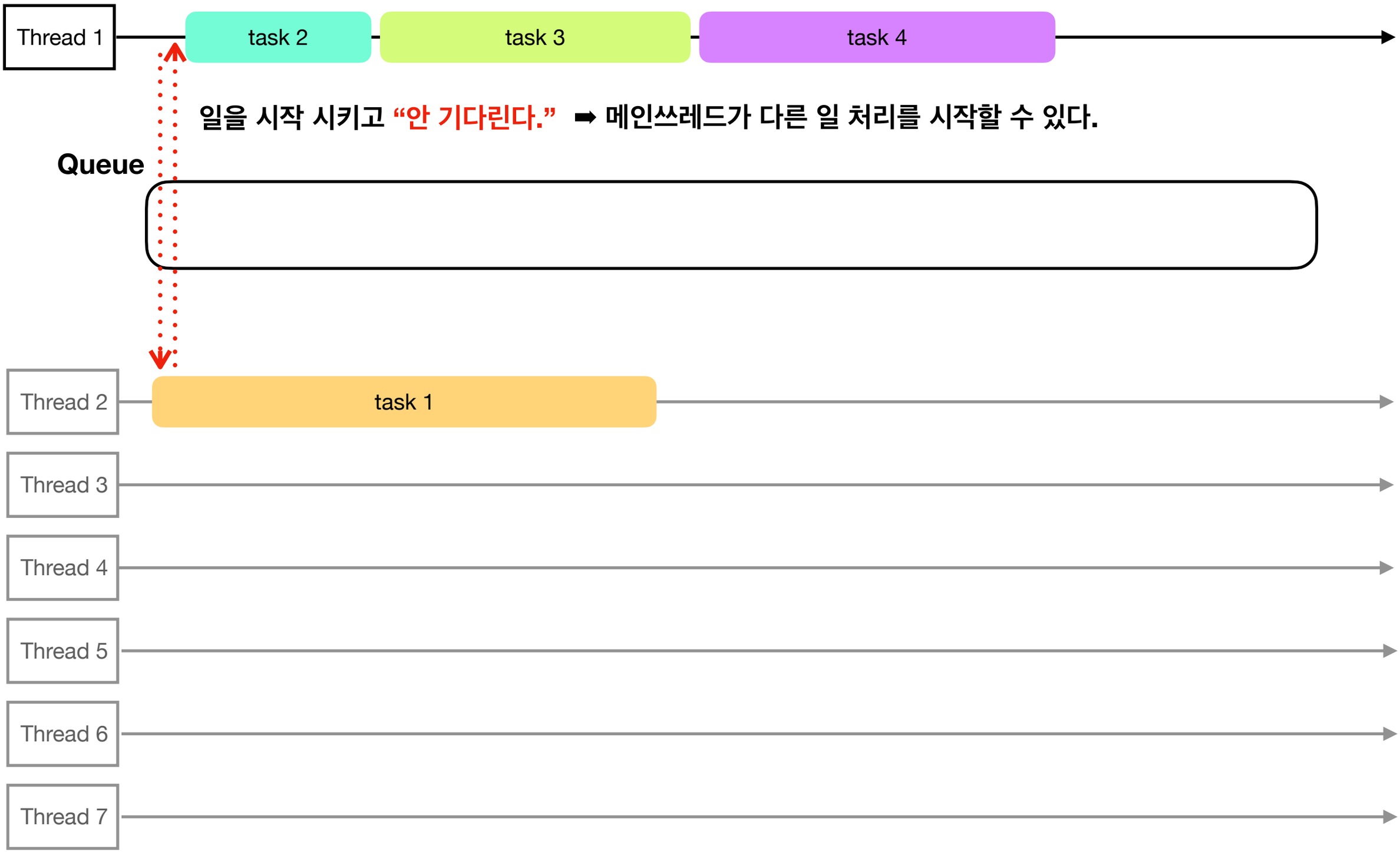
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

비동기 개념



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

비동기 개념



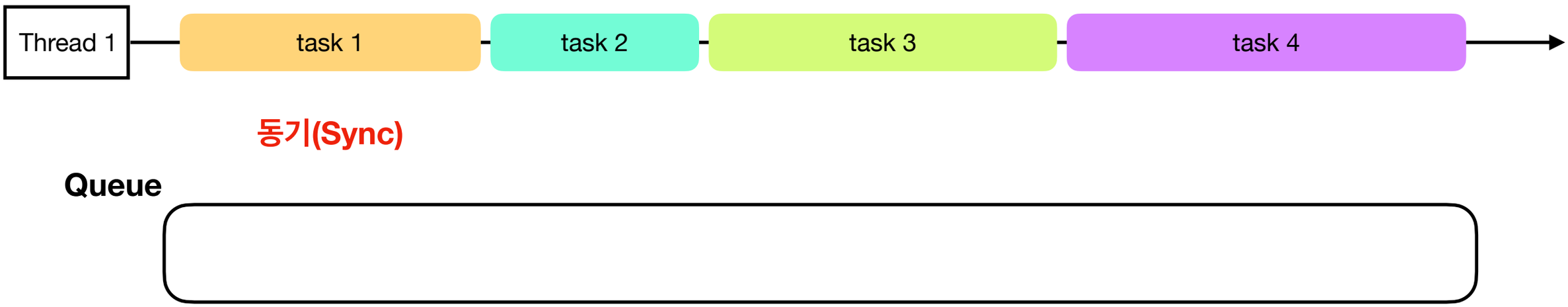
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
DispatchQueue.global( ).async {



}
```

원래의 작업이 진행되고 있던 곳(메인스레드)에서
디스패치 글로벌 큐로 보낸 작업을
안 기다린다.

동기 개념



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

동기 개념

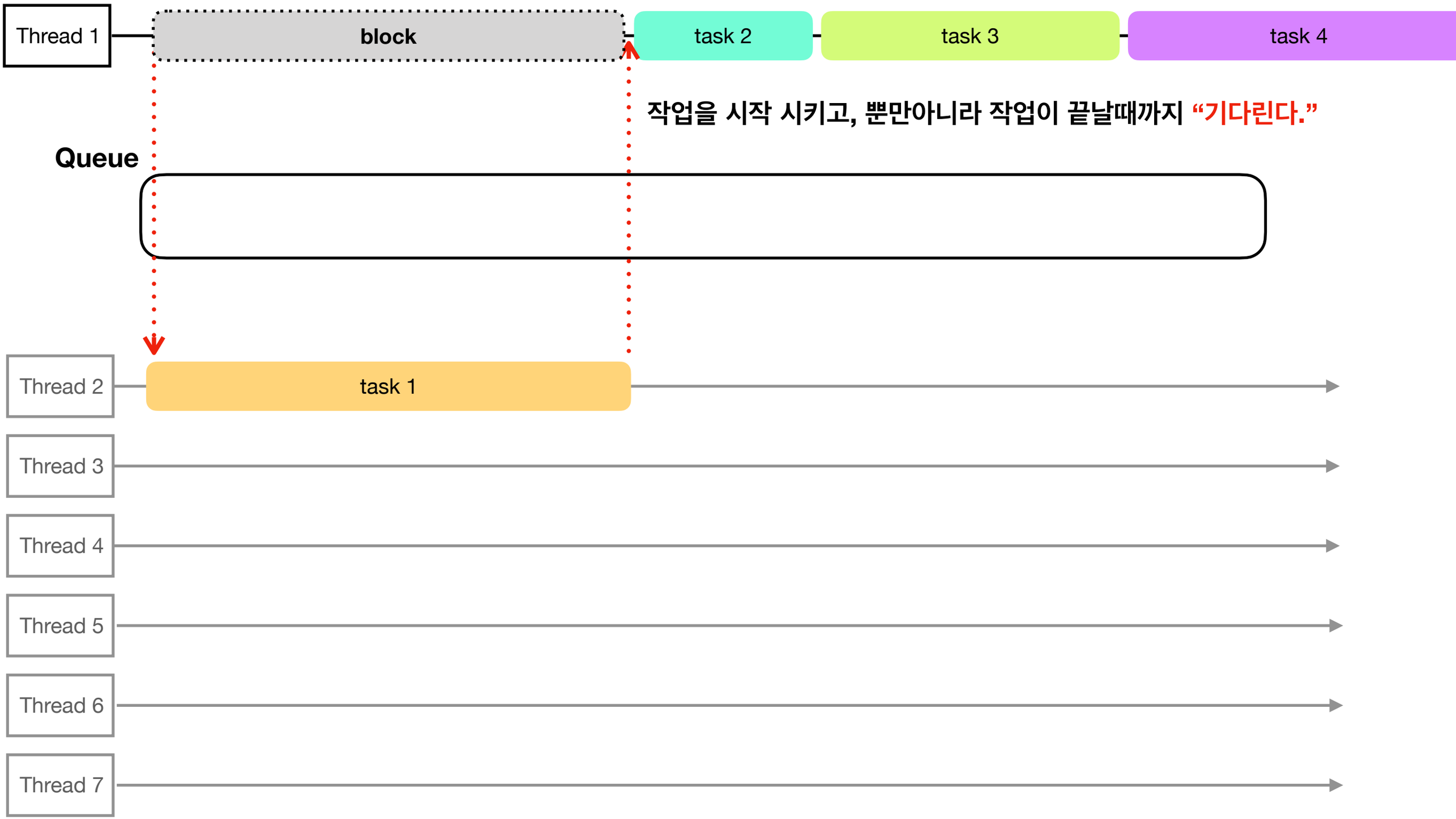
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
DispatchQueue.global( ).sync {


}
```

원래의 작업이 진행되고 있던 곳(메인스레드)에서
디스패치 글로벌 큐로 보낸 작업을
동기적으로 기다린다.

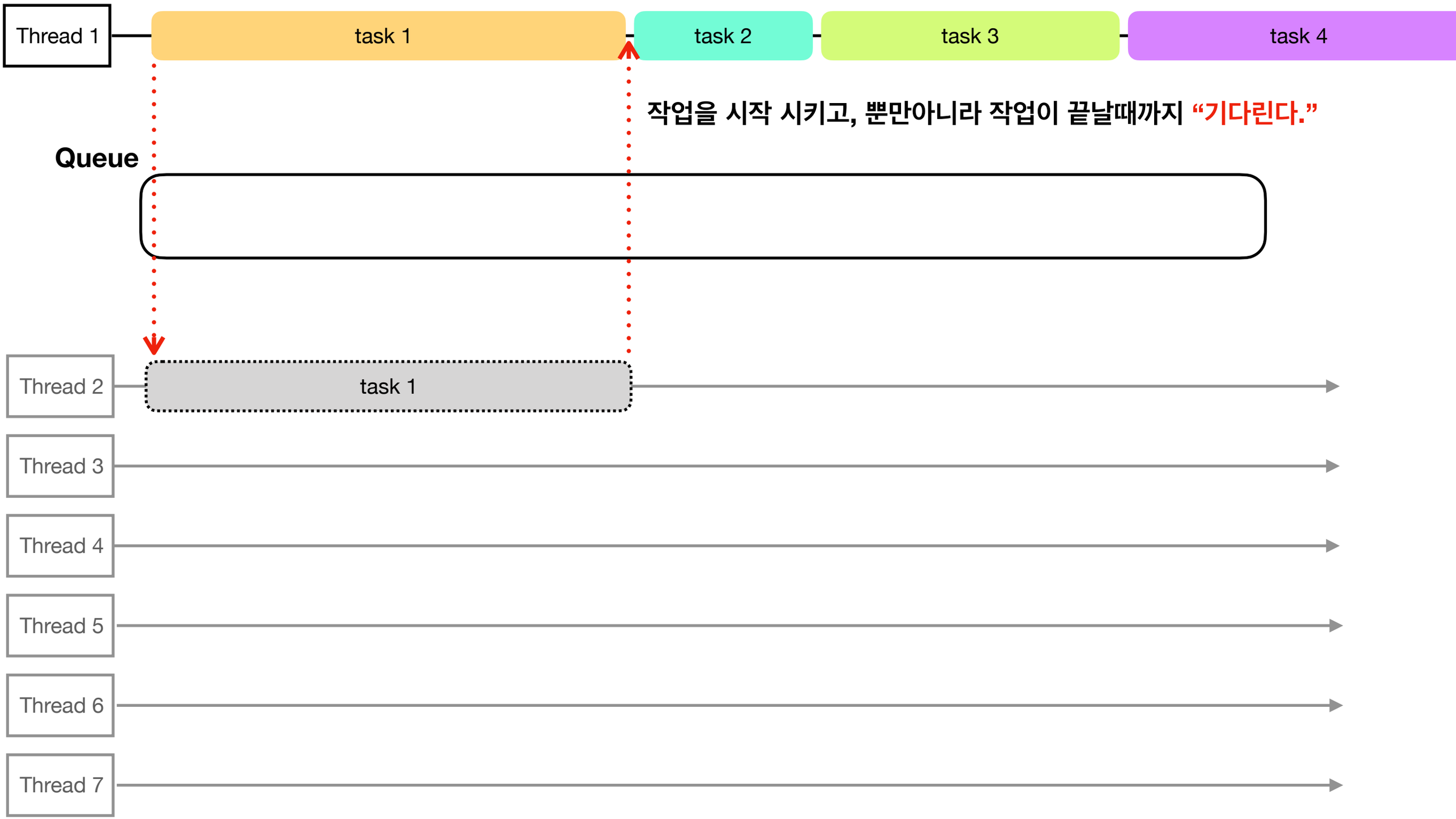
동기 개념



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

동기 개념

(동기적으로 보내는 코드를 짜더라도) 실질적으로는 메인쓰레드에서 일한다.



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
DispatchQueue.global( ).sync {
                                async


}
```

비동기(Async) ····· **작업을 다른 쓰레드에서 하도록 시킨 후,**
그 작업이 끝나길 "안 기다리고" 다음일을 진행한다.
(안 기다려도 다음 작업을 생성할 수 있다.)

동기(Sync) ····· **작업을 다른 쓰레드에서 하도록 시킨 후,**
그 작업이 끝나길 "기다렸다가" 다음일을 진행한다.
(기다렸다가 다음 작업을 생성할 수 있다.)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

1-2.AsyncVsSync

이런 “비동기”라는 개념이 일반적으로 필요한 이유는?

대부분은 서버와의 통신(네트워크 작업) 때문

(네트워크와 관련된 작업들은 내부적으로 비동기적으로 구현)

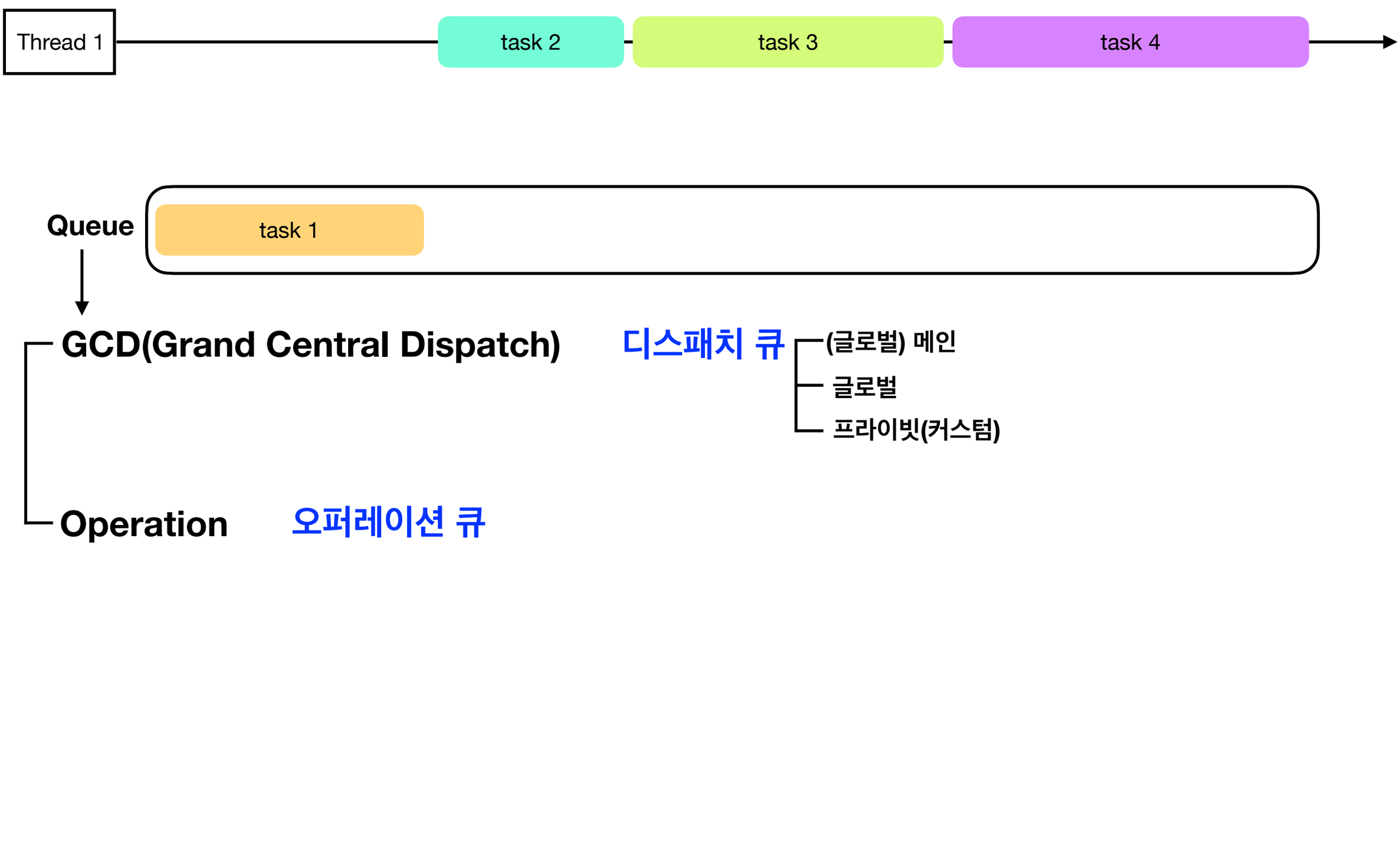
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
URLSession(configuration: .ephemeral).dataTask(with: url) {
    data, response, error in
    self.image = UIImage(data: data!)

    ....
}
```

4) Serial(직렬) VS Concurrent(동시)

큐의 종류



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Serial(직렬) vs Concurrent(동시) 차이 그림으로 이해하기

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Thread 1

Serial VS Concurrent: 큐(Queue)의 특성에 관한 것

Queue

task 1 | task 2 | task 3 | task 4

Thread 2

Thread 3

Thread 4

Thread 5

Thread 6

Thread 7

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

큐의 특성-직렬큐

Thread 1

일단 Serial큐 먼저

Queue

Thread 2 — task 1 — task 2 — task 3 — task 4

Thread 3

Thread 4

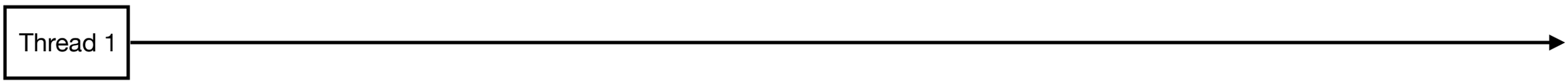
Thread 5

Thread 6

Thread 7

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

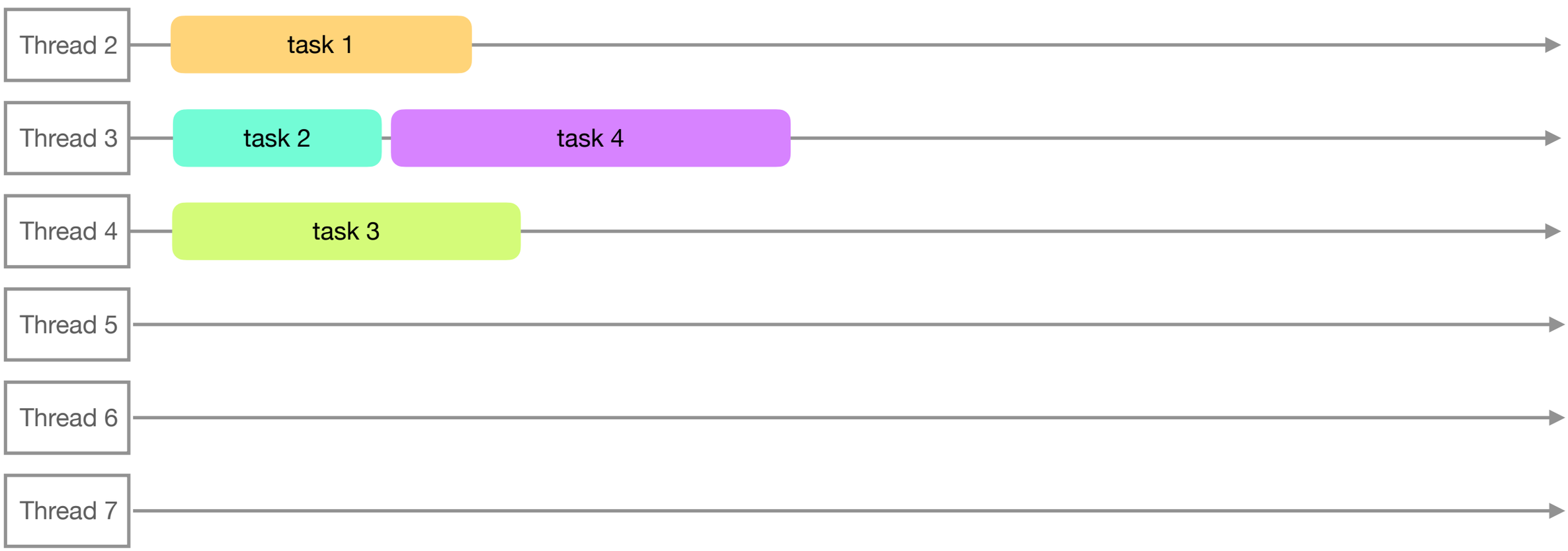
큐의 특성-동시큐



Concurrent큐

Queue

몇개의 쓰레드로 분산할지는 시스템이 알아서 결정, 다만 여러개의 쓰레드로 분산 처리



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

직렬(Serial) 큐 ..... **(보통 메인에서) 분산처리 시킨 작업을 “다른 한개의 쓰레드에서” 처리하는 큐**

동시(Cocurrent) 큐 ..... **(보통 메인에서) 분산처리 시킨 작업을 “다른 여러개의 쓰레드에서” 처리하는 큐**

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

분산처리 하려는 것이라면.. “Concurrent큐”가 무조건 좋아 보이는 것 같아보이는 데,

왜 직렬(Serial)큐가 필요할까?

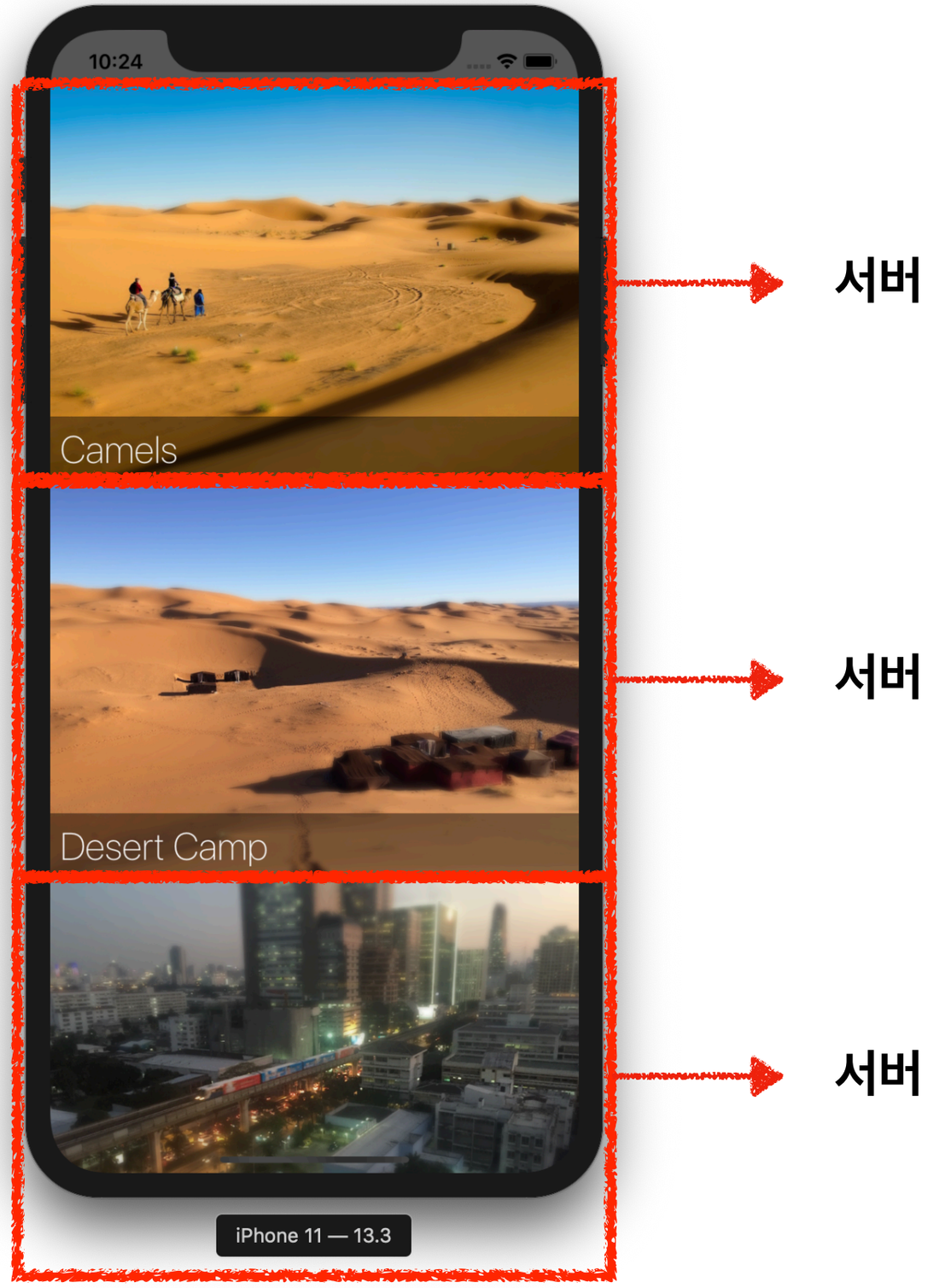
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

직렬(Serial) 큐 ····· **순서가 중요한 작업을 처리할때 사용**

동시(Cocurrent) 큐 ····· **각자 독립적이지만 유사한 여러개의 작업을 처리할때 사용**

(중요도나 작업의 성격등)

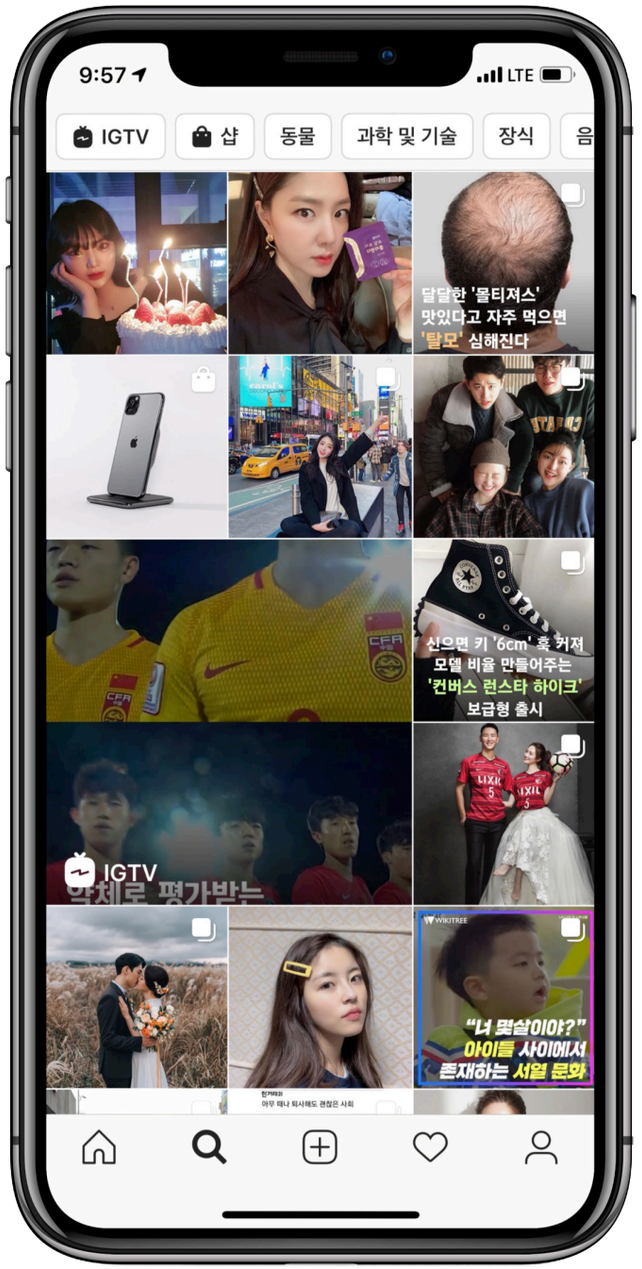
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com



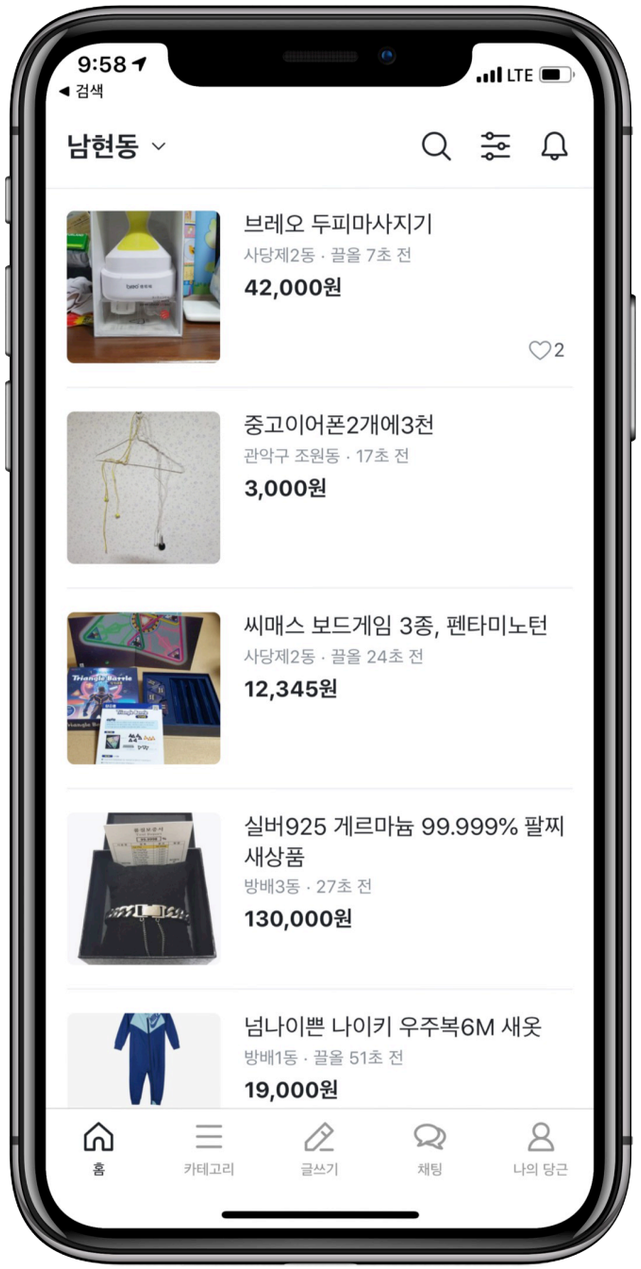
예제 출처: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

넷플릭스



인스타그램



당근마켓

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

비동기(Async)란 말과 동시(Concurrent)란 말이 같은 말인가?

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

2. 디스패치큐(GCD)의 종류와 특성

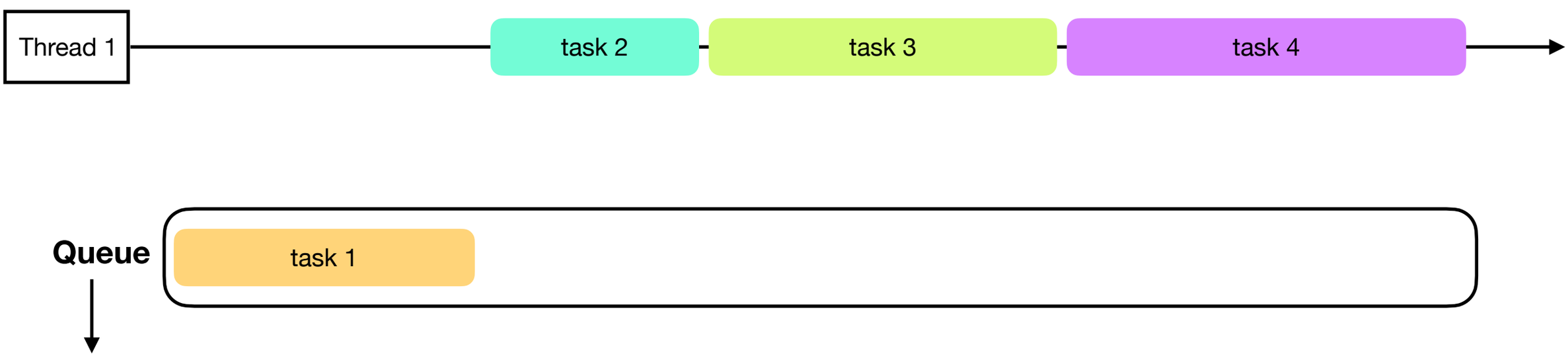
---

1) 메인 큐

2) 글로벌큐와 Qos의 이해

3) 프라이빗(커스텀) 큐

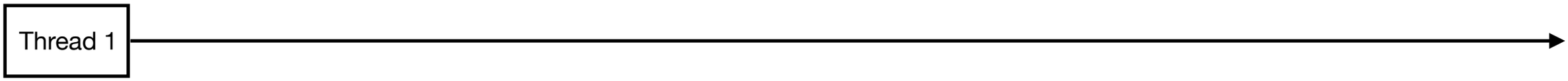
큐의 종류



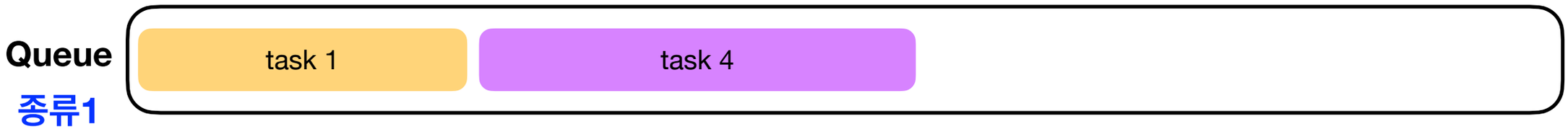
- **GCD(Grand Central Dispatch)** — 디스패치 큐
  - (글로벌) 메인
  - 글로벌
  - 프라이빗(커스텀)
- **OperationQueue** — 오퍼레이션 큐

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

큐의 종류



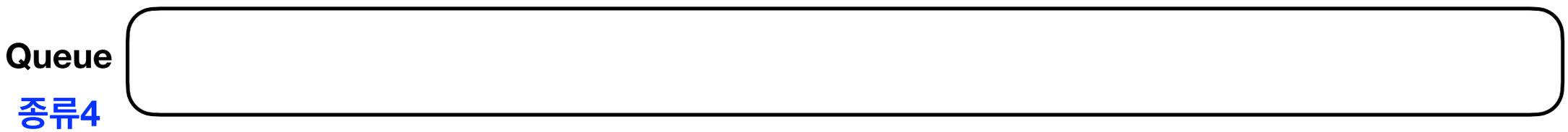
대기열마다 특성이 좀 다르다. 작업의 특성, 원하는 일처리에 따라 대기열(큐)의 특성에 맞게 작업을 보내면 된다.



종류1

종류2

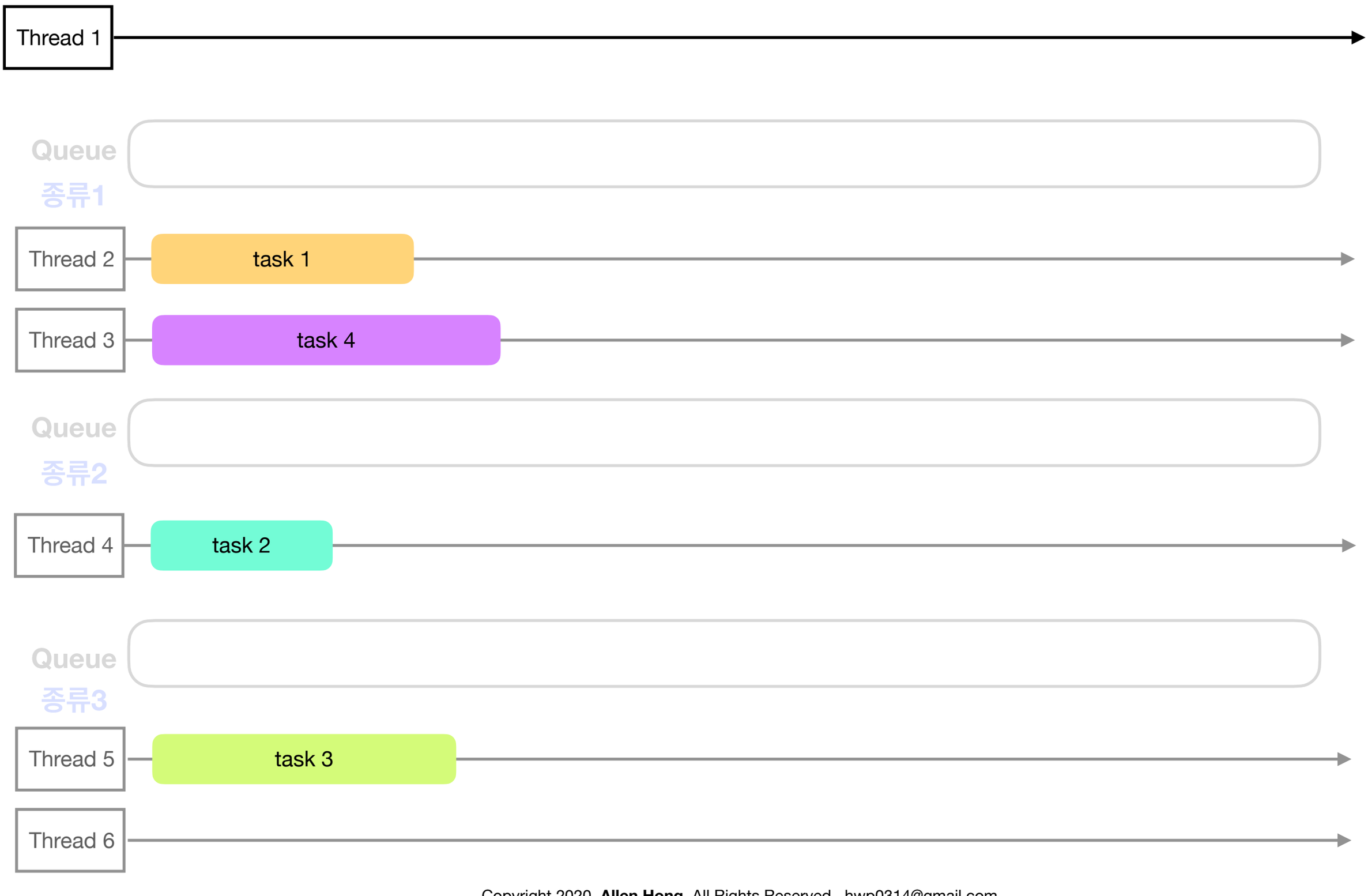
종류3



종류4

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

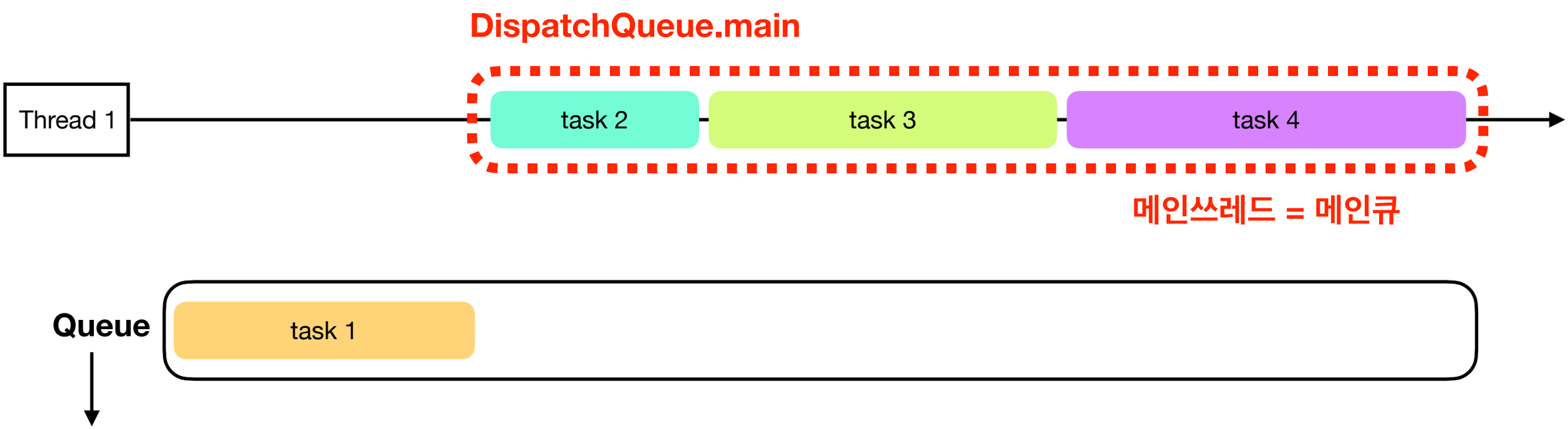
큐의 종류



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

1) 메인 큐

큐의 종류-메인큐



- GCD(Grand Central Dispatch) 디스패치 큐
  - (글로벌) 메인 (유일한)한개, 시리얼, 메인쓰레드
  - 글로벌
  - 프라이빗(커스텀)
- OperationQueue 오퍼레이션 큐

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
DispatchQueue.main.async {

}
```

코드예시

```
DispatchQueue.main.sync {
    print("print something")
}
```

(사실은 이런 의미가 숨어있었다.)

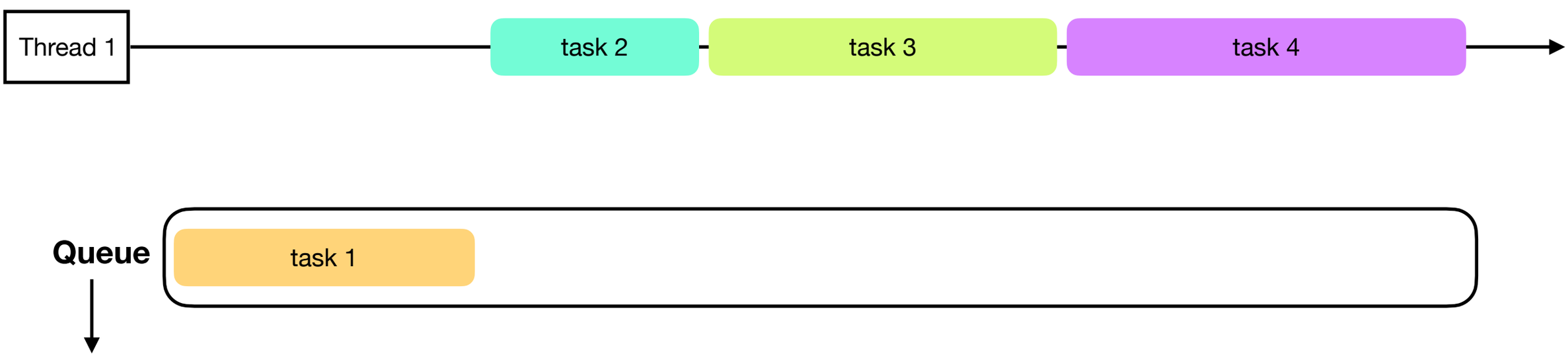
→ 1번 스레드에서 일을 한다.

코드예시 단순 참고

```
DispatchQueue.main.asyncAfter( .now( ) + 2 ) {



}
```

2) 글로벌큐와 Qos의 이해

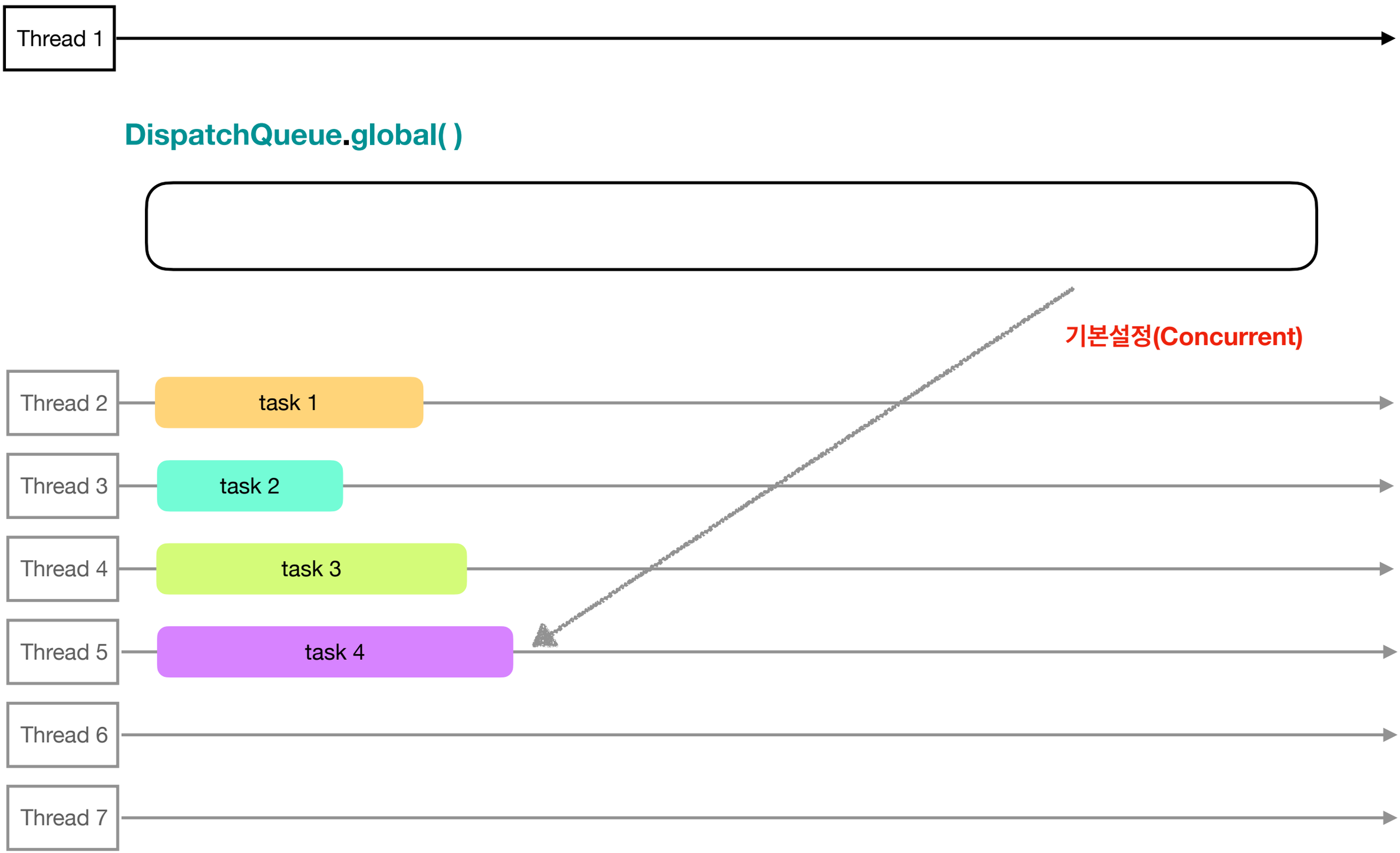
큐의 종류-글로벌큐



- GCD(Grand Central Dispatch) 디스패치 큐
  - (글로벌) 메인
  - 글로벌 종류 여러개, 기본설정(Concurrent), QoS (6종류)
  - 프라이빗(커스텀)
- OperationQueue 오퍼레이션 큐

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

큐의 종류-글로벌큐



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
DispatchQueue.global( ).async {



}
```

큐의 서비스품질

Thread 1

DispatchQueue.global(qos: .userInteractive) 거의 즉시

유저와 직접적 인터렉티브: UI업데이트, 애니메이션, UI반응관련 어떤 것이든 (사용자와 상호 작용)

DispatchQueue.global(qos: .userInitiated) 몇초

유저가 즉시 필요하긴 하지만, 비동기적으로 처리된 작업 (ex. 앱내에서 pdf파일을 여는 것과 같은)

DispatchQueue.global( ) 디폴트

일반적인 작업

DispatchQueue.global(qos: .utility) 몇초에서 몇분

보통 Progress Indicator와 함께 길게 실행되는 작업, 계산, IO, Networking, 지속적인 데이터 feeds

DispatchQueue.global(qos: .background) 속도보다는 에너지효율성 중시, 몇분이상

유저가 직접적으로 인지하지 않고(시간이 안 중요한) 작업, 데이터 미리가져오기, 데이터베이스 유지

DispatchQueue.global(qos: .unspecified)

legacy API

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

큐의 서비스품질

구 분	사용해야하는 상황	소요 시간
.userInteractive	유저와 직접적 인터렉티브: UI업데이트, 애니메이션, UI반응관련 어떤 것이든 (사용자와 직접 상호 작용하는 작업에 권장. 작업이 빨리 처리되지 않으면 상황이 멈춘 것처럼 보일만한)	거의 즉시
.userInitiated	유저가 즉시 필요하긴 하지만, 비동기적으로 처리된 작업 (ex. 앱내에서 pdf파일을 여는 것과 같은, 로컬 데이터베이스 읽기)	몇초
.default	일반적인 작업	-
.utility	보통 Progress Indicator와 함께 길게 실행되는 작업, 계산 (ex. IO, Networking, 지속적인 데이터 feeds)	몇초에서 몇분
.background	유저가 직접적으로 인지하지 않고(시간이 안 중요한) 작업 (ex.데이터 미리가져오기, 데이터베이스 유지보수, 원격 서버 동기화 및 백업 수행)	몇분이상 (속도보다는 에너지효율성 중시)
.unspecified	legacy API 지원 (스레드를 서비스 품질에서 제외시키는)	**-**

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
DispatchQueue.global( ).async {
    task1
}

DispatchQueue.global(qos: .userInitiated ).async {
    task2
}
```

큐의 서비스품질

Thread 1

DispatchQueue.global(qos: .userInteractive)

DispatchQueue.global(qos: .userInitiated)

task 2

DispatchQueue.global( ) 디폴트

task 1

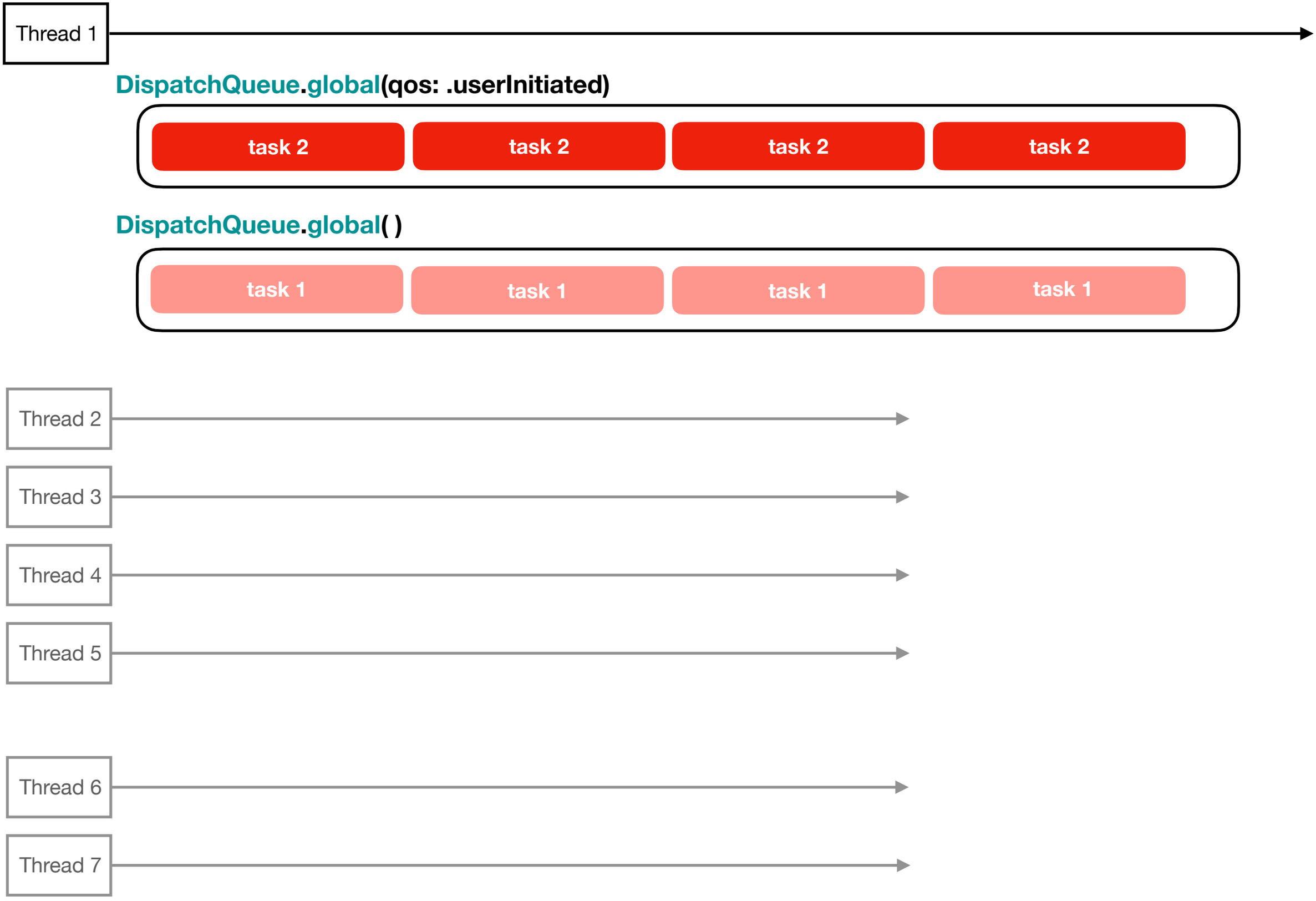
DispatchQueue.global(qos: .utility)

DispatchQueue.global(qos: .background)

DispatchQueue.global(qos: .unspecified)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

큐의 서비스품질



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

큐의 서비스품질

Thread 1

DispatchQueue.global(qos: .userInitiated)

DispatchQueue.global( )

Thread 2 — task 2

Thread 3 — task 2

Thread 4 — task 2

Thread 5 — task 2

Thread 6 — task 1 — task 1

Thread 7 — task 1 — task 1

iOS가 알아서
우선적으로 중요한 일임을
인지하고 쓰레드에
우선순위를 매겨
더 여러개의 쓰레드를 배치하고
배터리를 더 집중해서 사용
하도록 함

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

큐의 서비스품질

Thread 1

task 2

DispatchQueue.global(qos: .userInteractive)

DispatchQueue.global(qos: .userInitiated)

DispatchQueue.global( ) 디폴트

DispatchQueue.global(qos: .utility)

DispatchQueue.global(qos: .background)

task 1

async(qos: .utility)

DispatchQueue.global(qos: .unspecified)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

백그라운드로 정의

```
let queue = DispatchQueue.global(qos: .background)
```

작업을 보낼때 더 높은 수준으로 보냄

```
queue.async(qos: .utility) {

}
```

큐의 서비스품질

Thread 1

task 2

DispatchQueue.global(qos: .userInteractive)

DispatchQueue.global(qos: .userInitiated)

DispatchQueue.global( ) 디폴트

DispatchQueue.global(qos: .utility)

DispatchQueue.global(qos: .background) → .utility

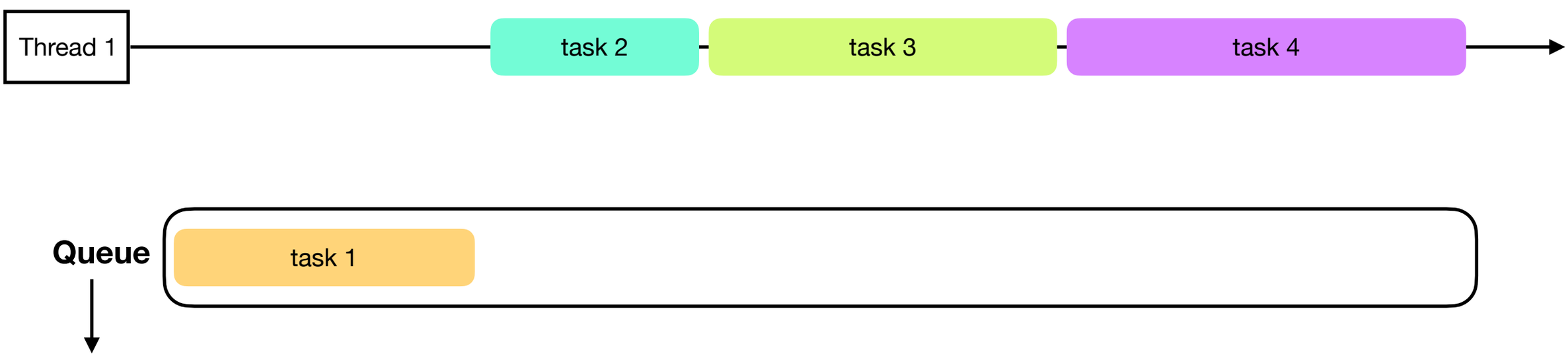
task 1 async(qos: .utility)

DispatchQueue.global(qos: .unspecified)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

3) 프라이빗(커스텀) 큐

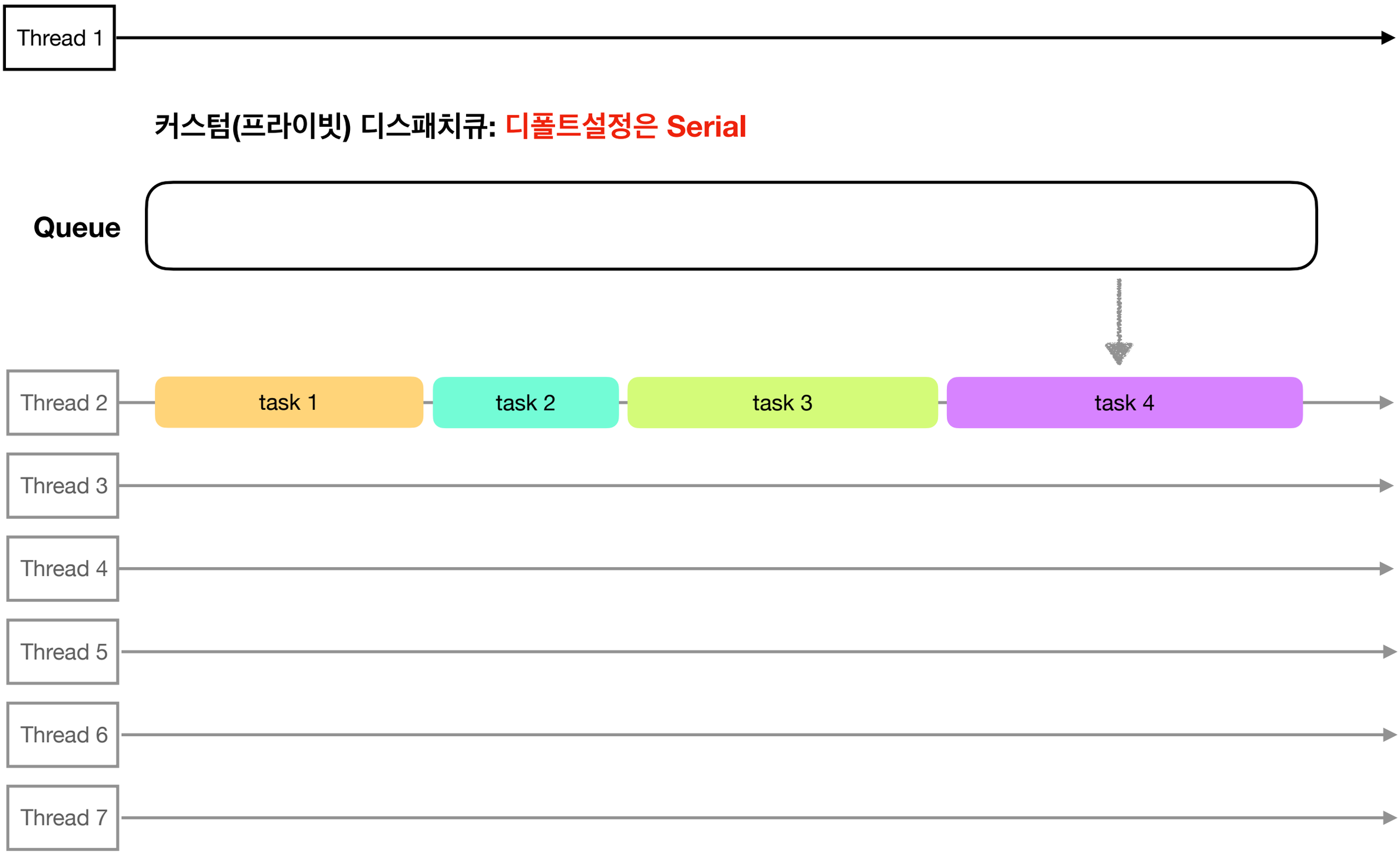
큐의 종류



- **GCD(Grand Central Dispatch)** — 디스패치 큐
  - (글로벌) 메인
  - 글로벌
  - 프라이빗(커스텀) — 커스텀으로 만듦, 디폴트설정(Serial), (Concurrent설정 가능), Qos 설정도 가능
- **OperationQueue** — 오퍼레이션 큐

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

큐의 종류



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

레이블만 붙이면 됨

```
let queue = DispatchQueue(label: “com.inflearn.serial”)
```

커스텀/프라이빗 큐는 동시큐로 설정도 가능

```
let queue2 = DispatchQueue(label: “com.inflearn.concurrent”, attributes: .concurrent)
```

OS가 QoS를 알아서, 추론

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

2.GCD-Queues

큐의 종류 정리

큐의 종류		생성 코드	특 징	직렬/동시
Dispatch Queue (GCD) (Class)	**.main**	**DispatchQueue.main**	메인큐 = 메인쓰레드 (1번쓰레드) (UI업데이트 내용 처리하는 큐)	Serial (직렬)
	.global( )	**DispatchQueue.global(qos:)**	6가지 Qos (작업에 따라 Qos상승가능) (시스템이 우선순위에 따라 더 많은 쓰레드를 배치하고, 배터리를 더 집중해서 사용하도록 함)	Concurrent (동시)
	custom (프라이빗)	**DispatchQueue(label: "…")**	Qos추론 / Qos설정가능.	디폴트: Serial 둘다 가능 (attributes로 설정)
OperationQueue (Class) (내부적으로 GCD기반으로 구현)		**let opQ = OperationQueue( )**	디폴트: .background. 기반(underlying) 디스패치큐에 영향 받음 (unspecified를 제외한 5가지)	디폴트: Concurrent 둘다 가능 (maxConcurrentOperationCount로 사용할 쓰레드 갯수 설정 가능)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

3. 디스패치큐(GCD) 사용시 주의해야할 사항

---

1) 반드시 메인큐에서 처리해야하는 작업

2) sync 메서드에 대한 주의사항

3) weak, strong캡처 주의

4) 컴플리션핸들러의 존재이유

5) 동기적 함수를 비동기함수 처럼 만드는 방법

1) 반드시 메인큐에서 처리해야하는 작업

중요하게 알아야할 한가지 개념

UI관련일들은 “메인큐”에서 처리해야 한다.

(화면)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
DispatchQueue.global( ).async {
        // 이미지 다운로드 등 관련 코드
        코드 1
        코드 2
        코드 3
        코드 4

        DispatchQueue.main.async {
                // 다운로드한 이미지 표시 코드
                self.imageView.image = image
        }

}
```

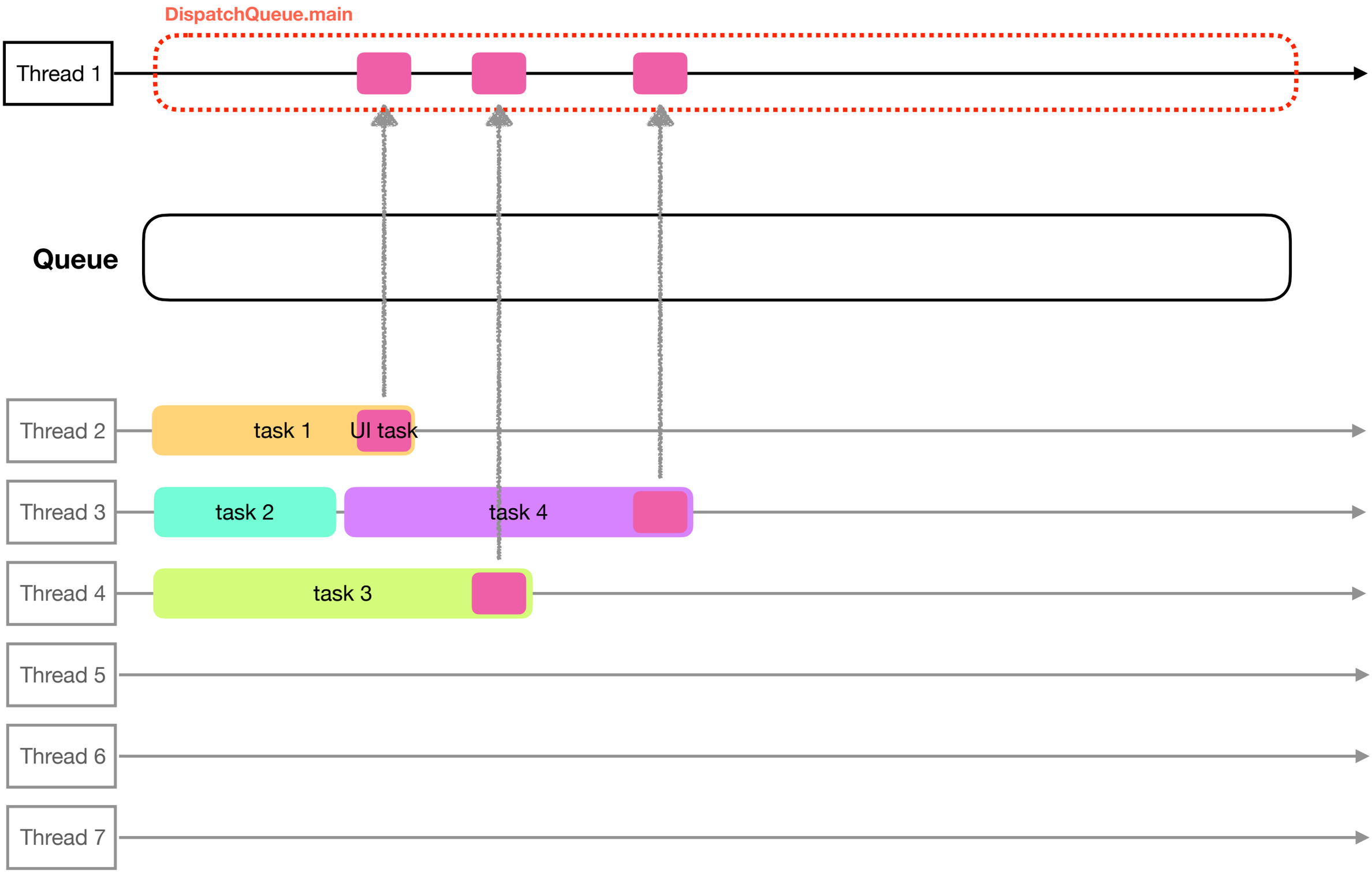
오래 걸리는 작업이니,
비동기로 분산해서 작업을
처리하고 싶다.
(예, (글로벌큐로 보내서)
다른 쓰레드로 보낸다.)

UI관련 작업은 다시
메인큐로 보낸다.

코드예시

```
URLSession.shared.dataTask(with: url) {
        // 이미지 다운로드 등 관련 코드
        코드 1
        코드 2
        코드 3
        코드 4

        DispatchQueue.main.async {
                // 다운로드한 이미지 표시 코드
                self.imageView.image = image
        }

}
```

UI업데이트



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

3-1.UI-update

2) sync메서드에 대한 주의사항

(sync메서드와 관련해 절대 해서는 안되는 코드 2가지)

1) 메인큐에서는 다른큐로 보낼때 sync메서드를 부르면 절대 안된다.

“메인큐에서는 항상 비동기적으로 보내야한다.”

즉, UI와 관련되지 않은 오래걸리는 작업(네트워크)들은
다른 쓰레드에서 일을 할 수 있도록 **“비동기적(async)”으로 실행**하여야 하며, 동기적으로 시키면 UI가 멈춰서
유저한테 반응을 늦게 하고 버벅거린다.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

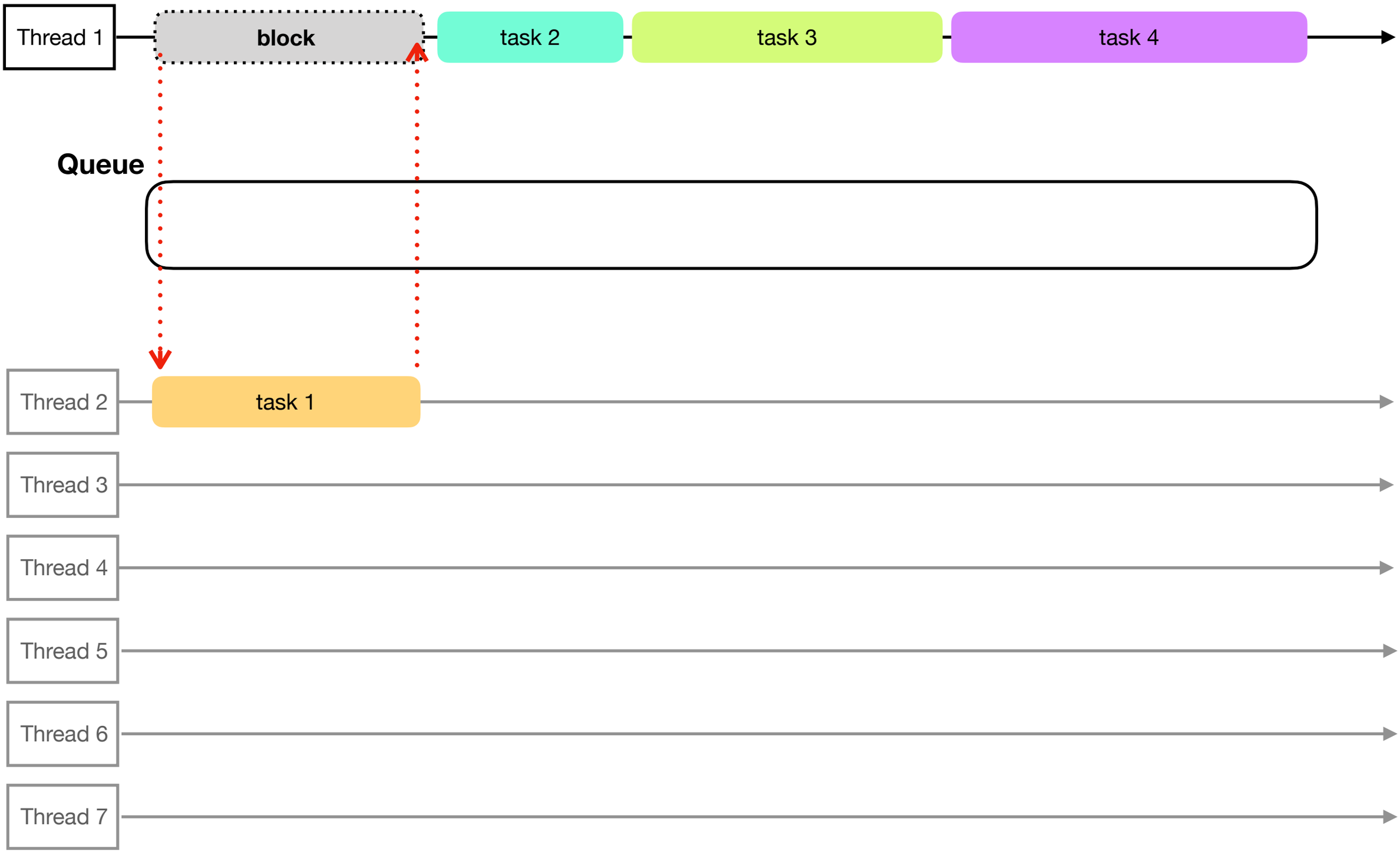
코드예시

(현재 메인에서 일하고 있다면)　절대 하면 안됨(never)

```
DispatchQueue.global( ).sync {
                        async


}
```

sync메서드 사용

UI 반응이 멈출 수 밖에 없다.



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

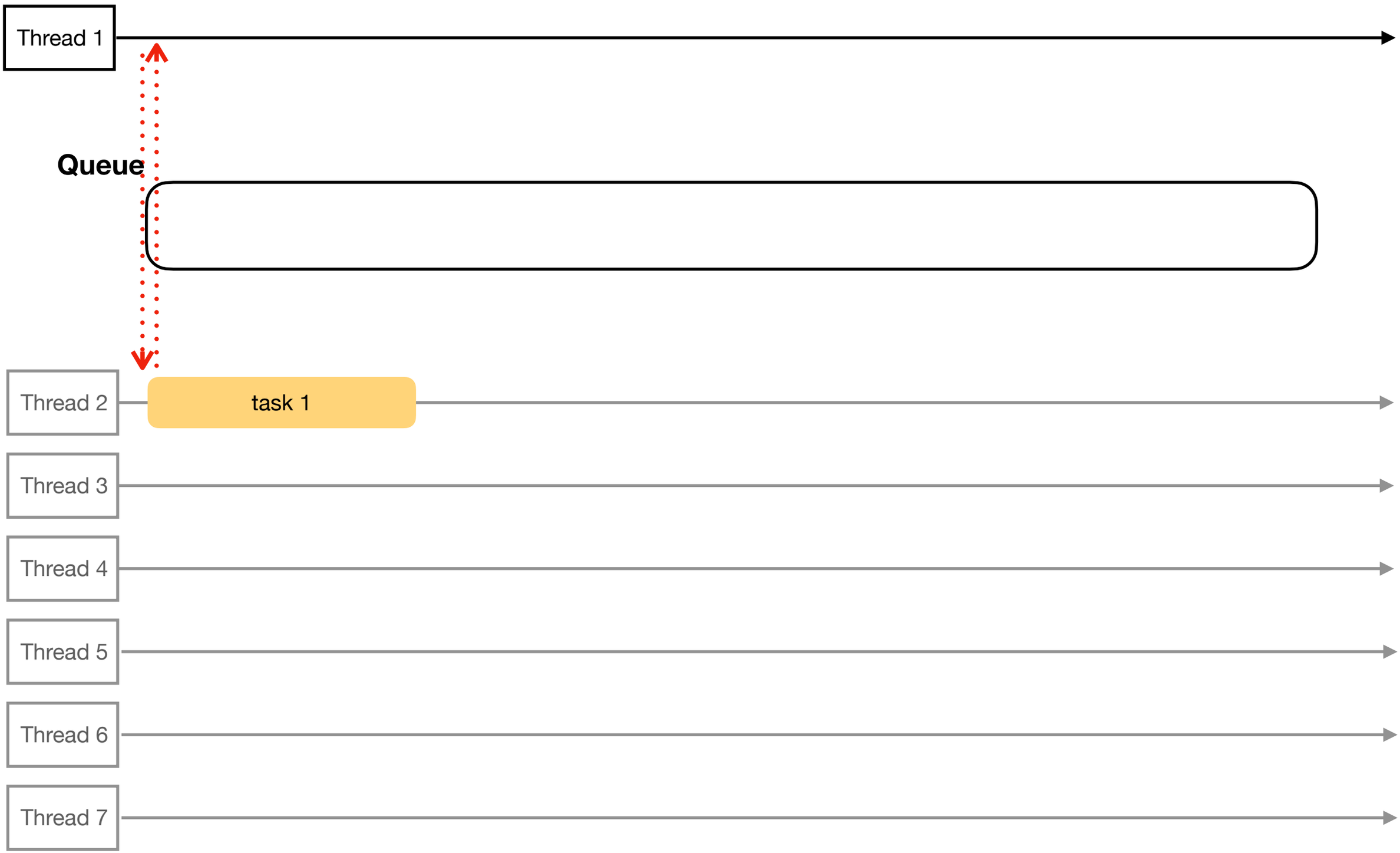
2) 현재의 큐에서 현재의 큐로 “동기적으로” 보내서는 안된다.

현재의 큐를 블락하는 동시에 다시 현재의 큐에 접근하기 때문에 **교착상황(DeadLock)이 발생**한다.

어떠한 이유에서간에 작업이 진행이 멈춰, 진행되지 못하는 상황

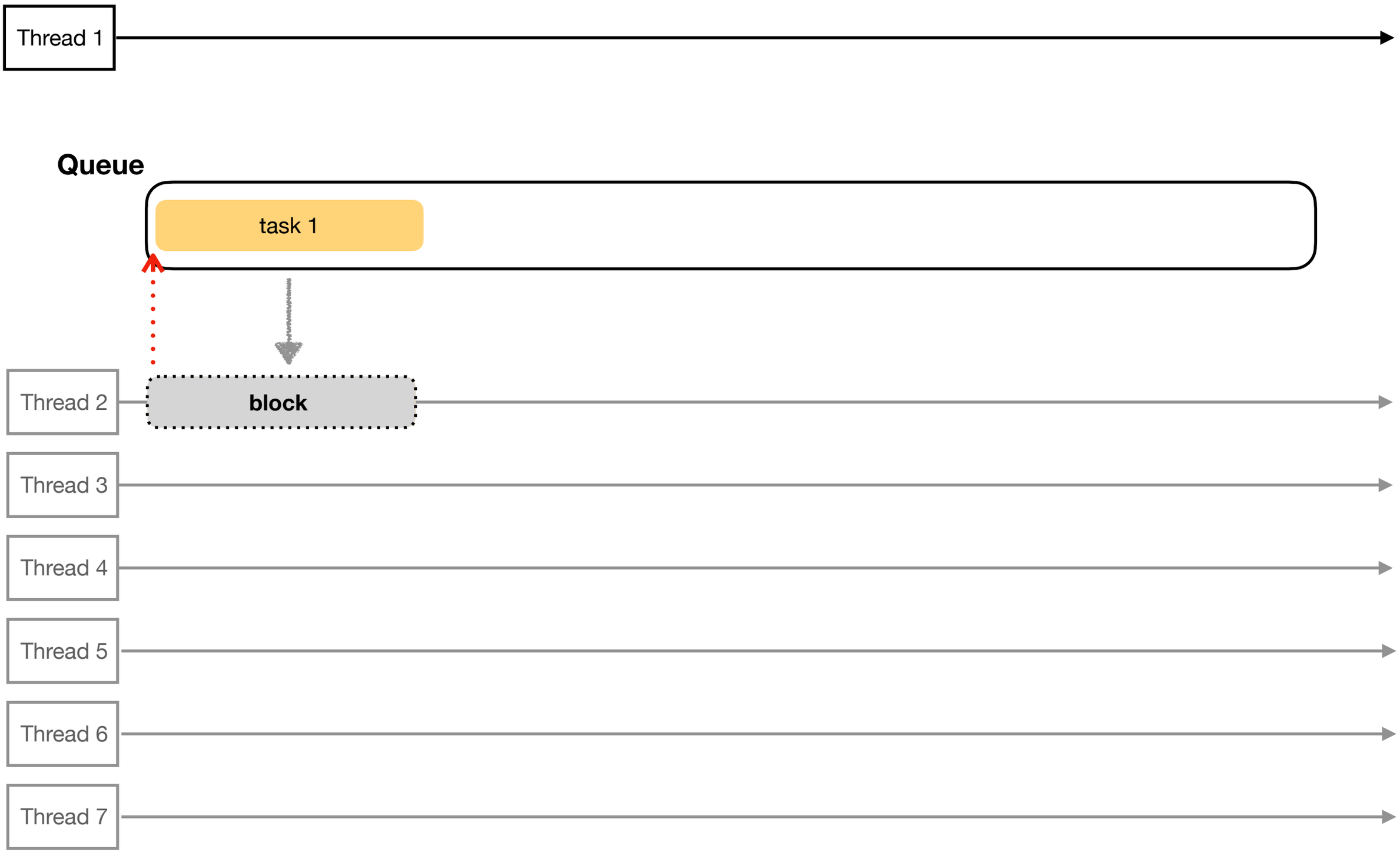
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

sync메서드 사용 **교착상황(Deadlock) 발생 상황**



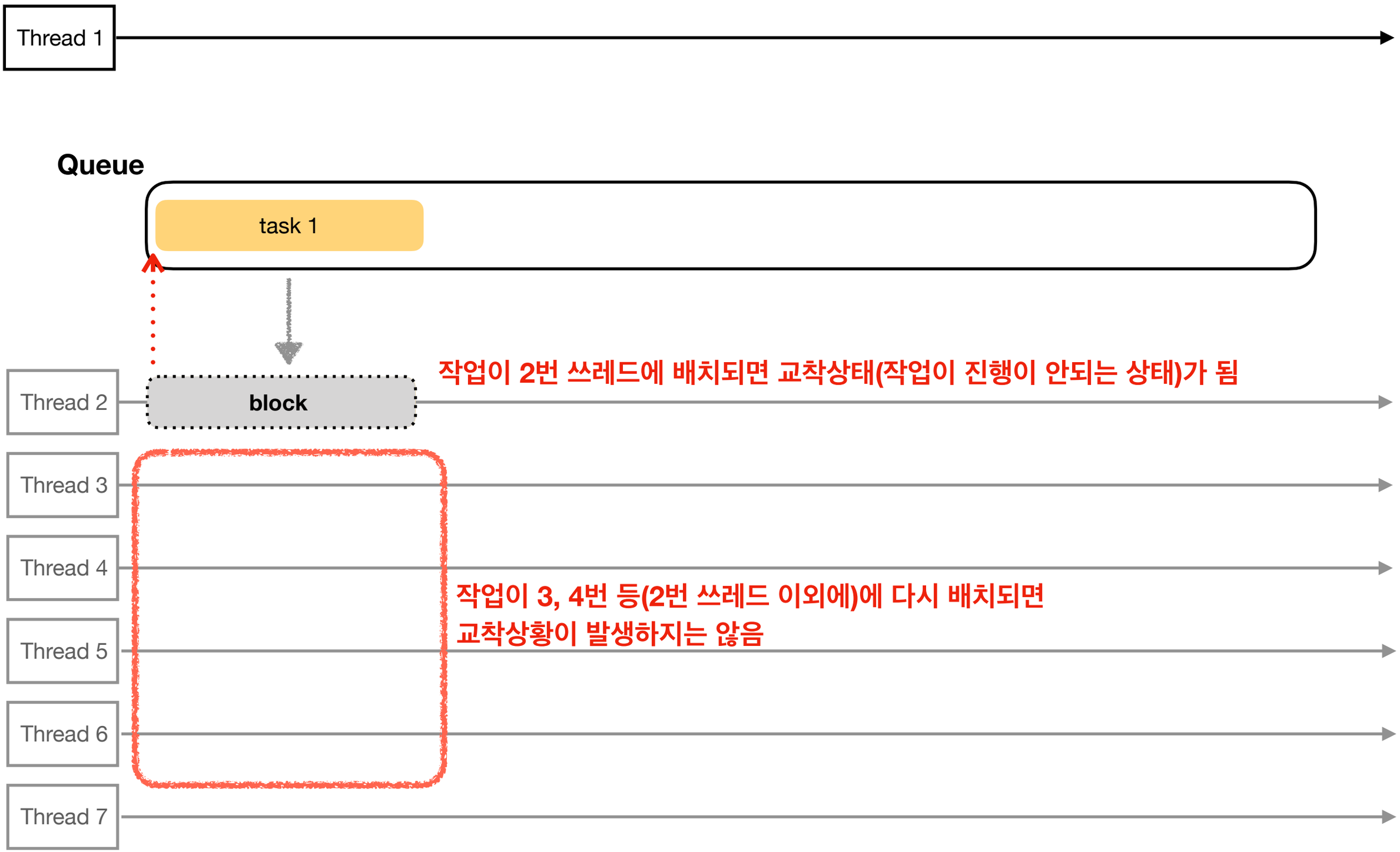
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

sync메서드 사용 **교착상황(Deadlock) 발생 상황**



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

sync메서드 사용 **교착상황(Deadlock) 발생 상황**



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

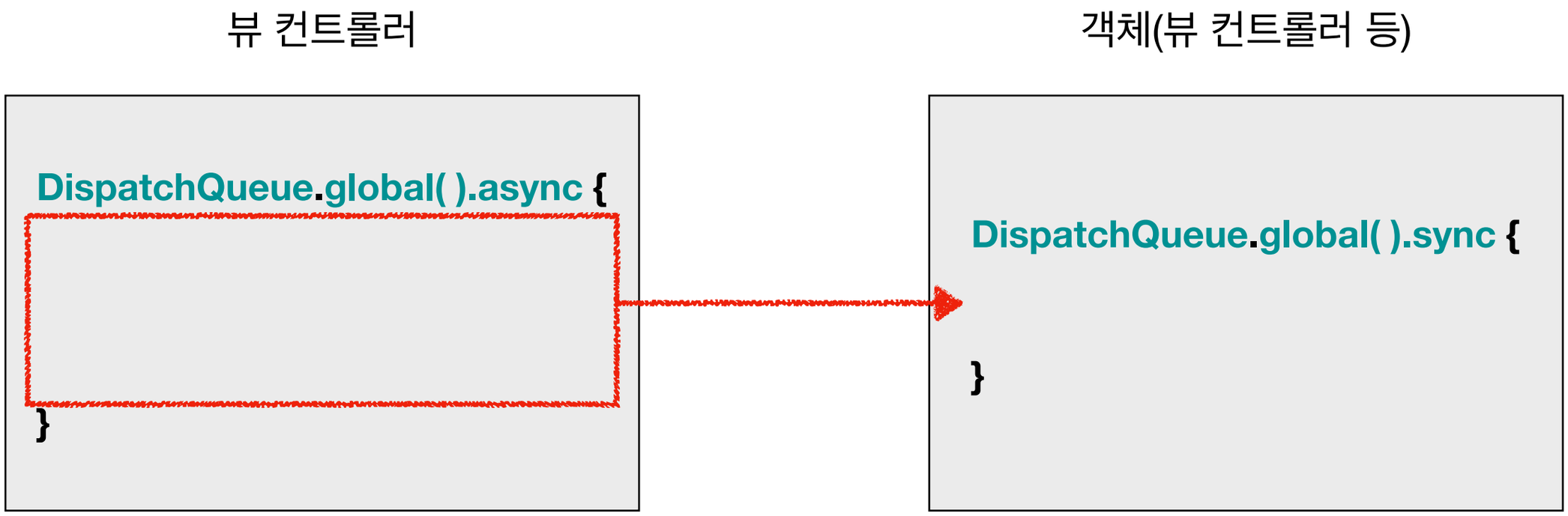
코드예시

실제로는 이런 코드 보다는...

```
DispatchQueue.global( ).async {

        DispatchQueue.global( ).sync {

        }

}
```

2) 현재의 큐에서 현재의 큐로 “동기적으로” 보내서는 안된다.

현재의 큐를 블락하는 동시에 다시 현재의 큐에 접근하기
때문에 **교착상황(DeadLock)**이 발생한다.



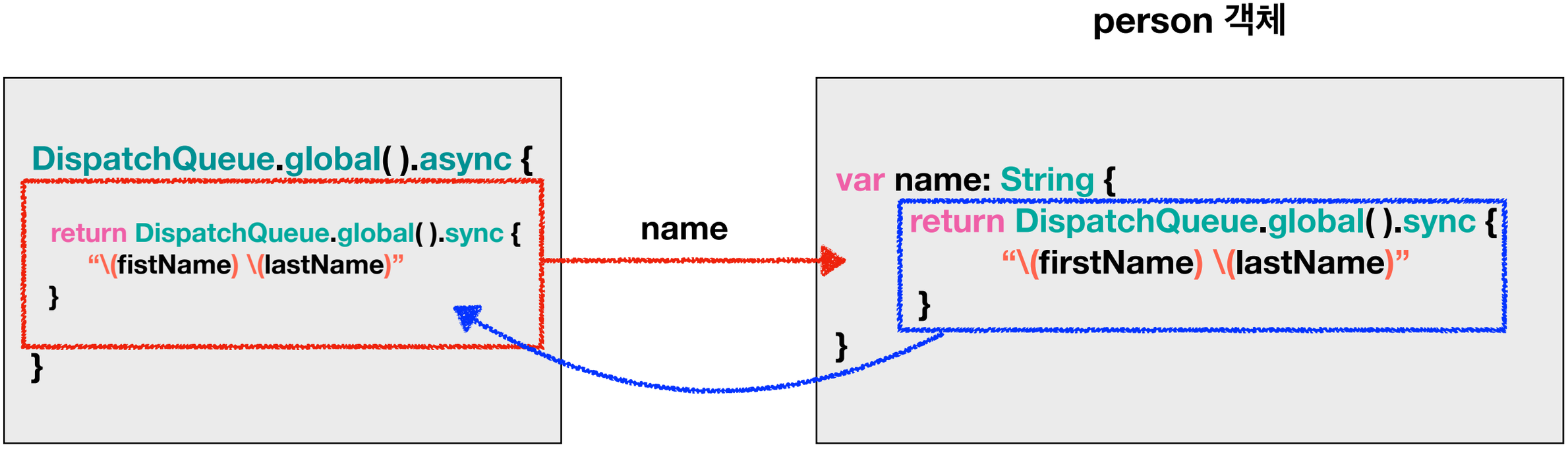
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

3-2.SyncMethod

코드 그림표현

3-2.SyncMethod



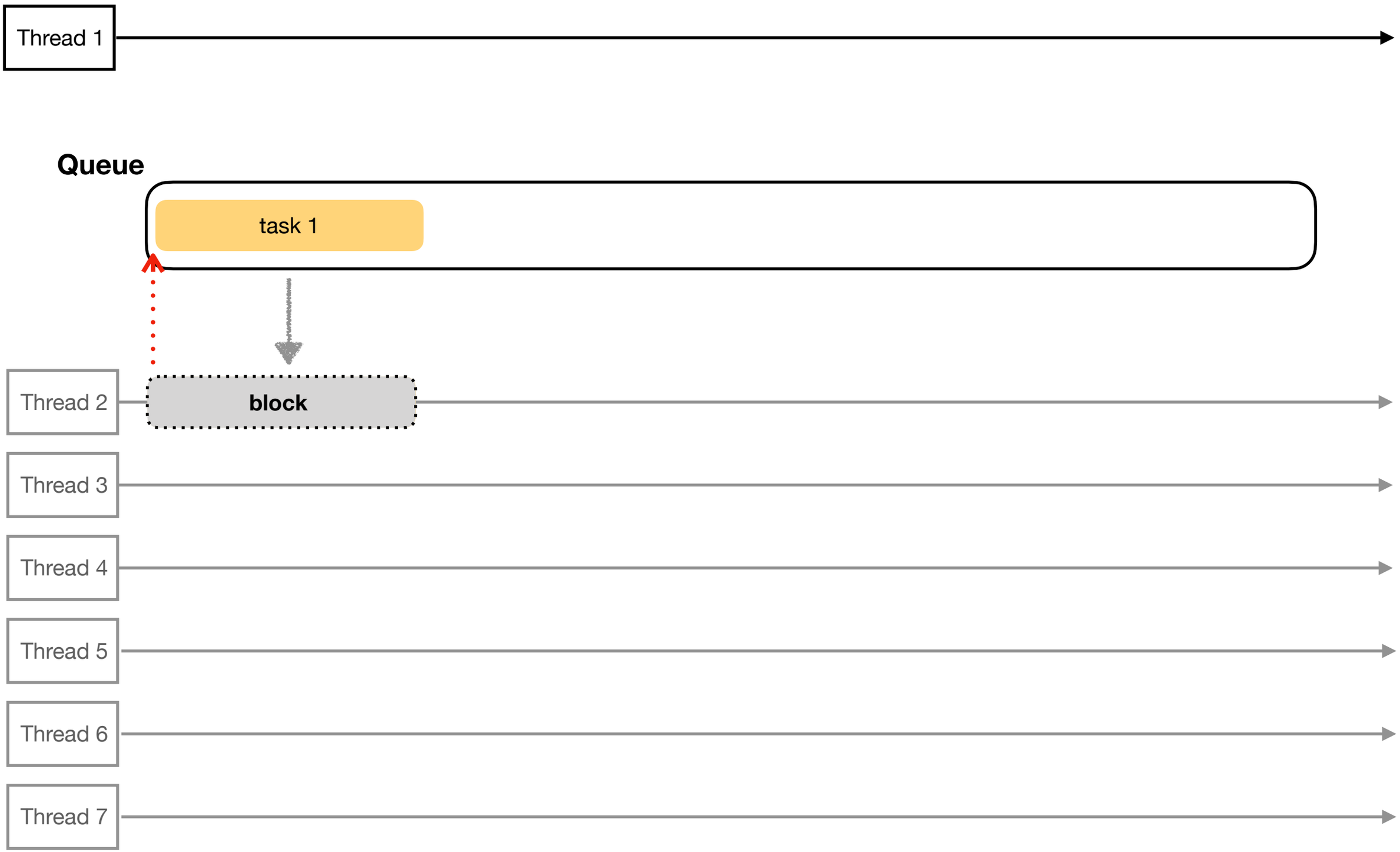
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
DispatchQueue.global( ).async {

    person 객체
    DispatchQueue.global( ).sync {


    }

}
```

결론적으로 이런 코드가 되버리는 것과 마찬가지임

sync메서드 사용 **교착상황(Deadlock) 발생 상황**



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

3) weak, strong캡처 주의

코드예시

```
DispatchQueue.global(qos: .utility).async { [weak self] in
    guard let self = self else { return }

    DispatchQueue.main.async {
        self.textLabel.text = “New posts updated!”
    }
}
```

뷰 컨트롤러

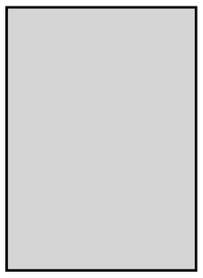
New posts updated!

```
DispatchQueue.global(qos: .utility).async { [weak self] in
    guard let self = self else { return }


      DispatchQueue.main.async {
       self.textLabel.text = “New posts updated!”
      }
}
```

ARC ➡ +1

let viewcon = ViewController( )

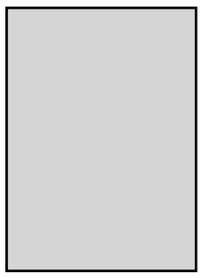
weak ➡ +0



뷰컨 dismiss

➡ 큐로 보낸 클로저(작업)도 중단됨

strong ➡ +1



뷰컨 dismiss

➡ 큐로 보낸 클로저(작업) 여전히 동작

(레퍼런스 사이클은 일어나지 않지만)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

4) (비동기 작업에서) 컴플리션핸들러의 존재이유

(completionHandler)

비동기로 작업을 시키고 나서, 작업에 해당하는 값을 바로 사용하면 안된다.

작업이 아직 종료하지 않았는데,
해당 값에 접근하면, 잘못된 값을 사용할 확률이 높다.
그래서, 해당 **비동기 작업이 끝났다는 것을 정확히**
알려주는 시점이 **컴플리션핸들러(completionHandler)**이다.

비동기 함수와 관련된 작업들은 모두 **컴플리션핸들러**를 가지고 있다.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

completion

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

5) 동기적 함수를 비동기함수 처럼 만드는 방법

동기함수를 비동기함수처럼 만드는 방법
(여러번 재활용 위해)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

3-5.SyncToAsync

URLSession과 같은 이미 비동기함수는

GCD/Operation이 필요없다.

(내장되어 있기에)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

4. 디스패치 그룹

---

1) 디스패치 그룹의 개념

2) 디스패치 그룹의 사용

- 디스패치 그룹과 동기함수
- 디스패치 그룹과 비동기함수

3) (참고) Dispatch workItem

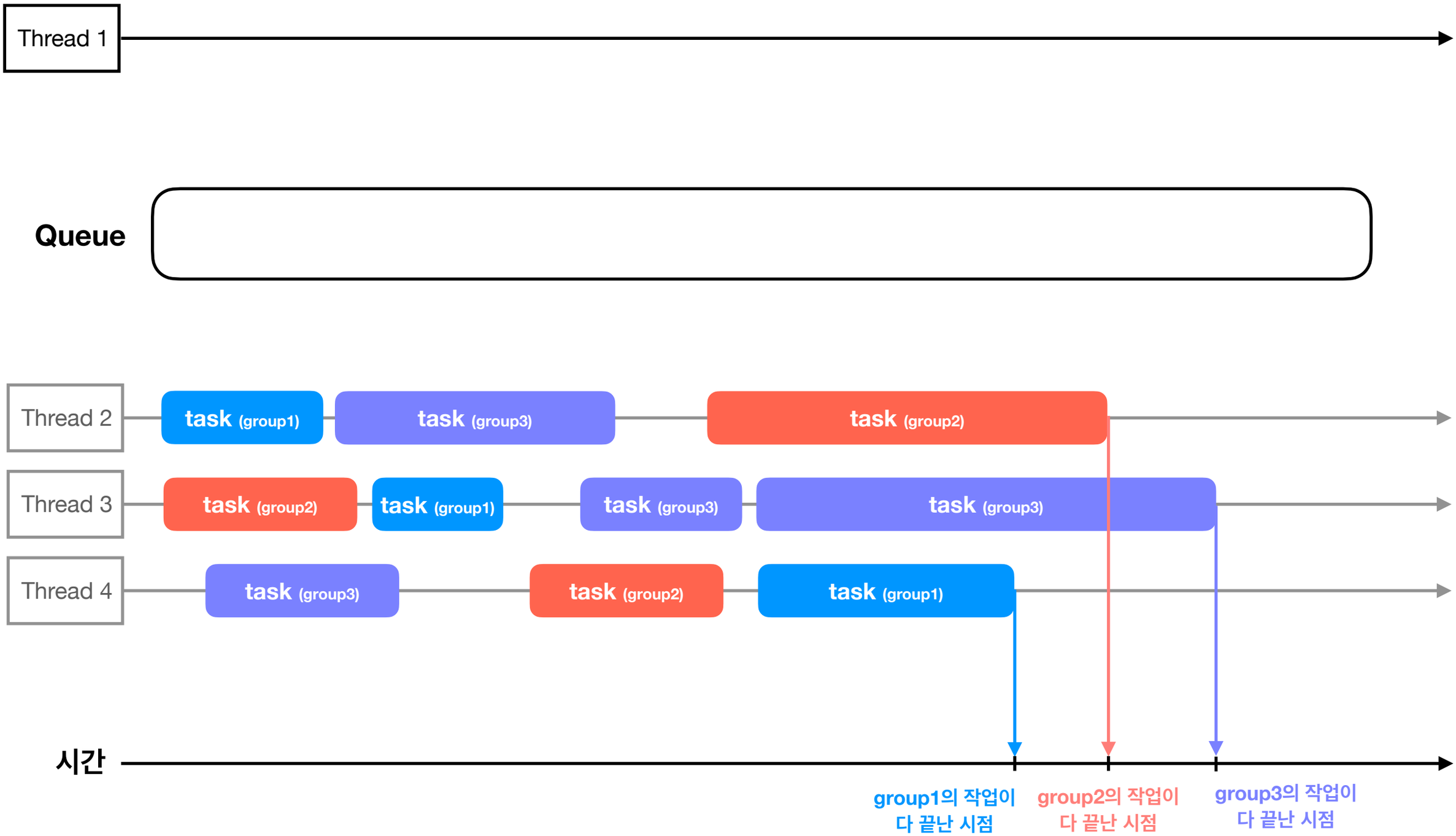
4) (심화) Semaphore의 이해

1) 디스패치 그룹의 개념

그룹이라는 개념이 왜 필요할까?

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

DispatchGroup

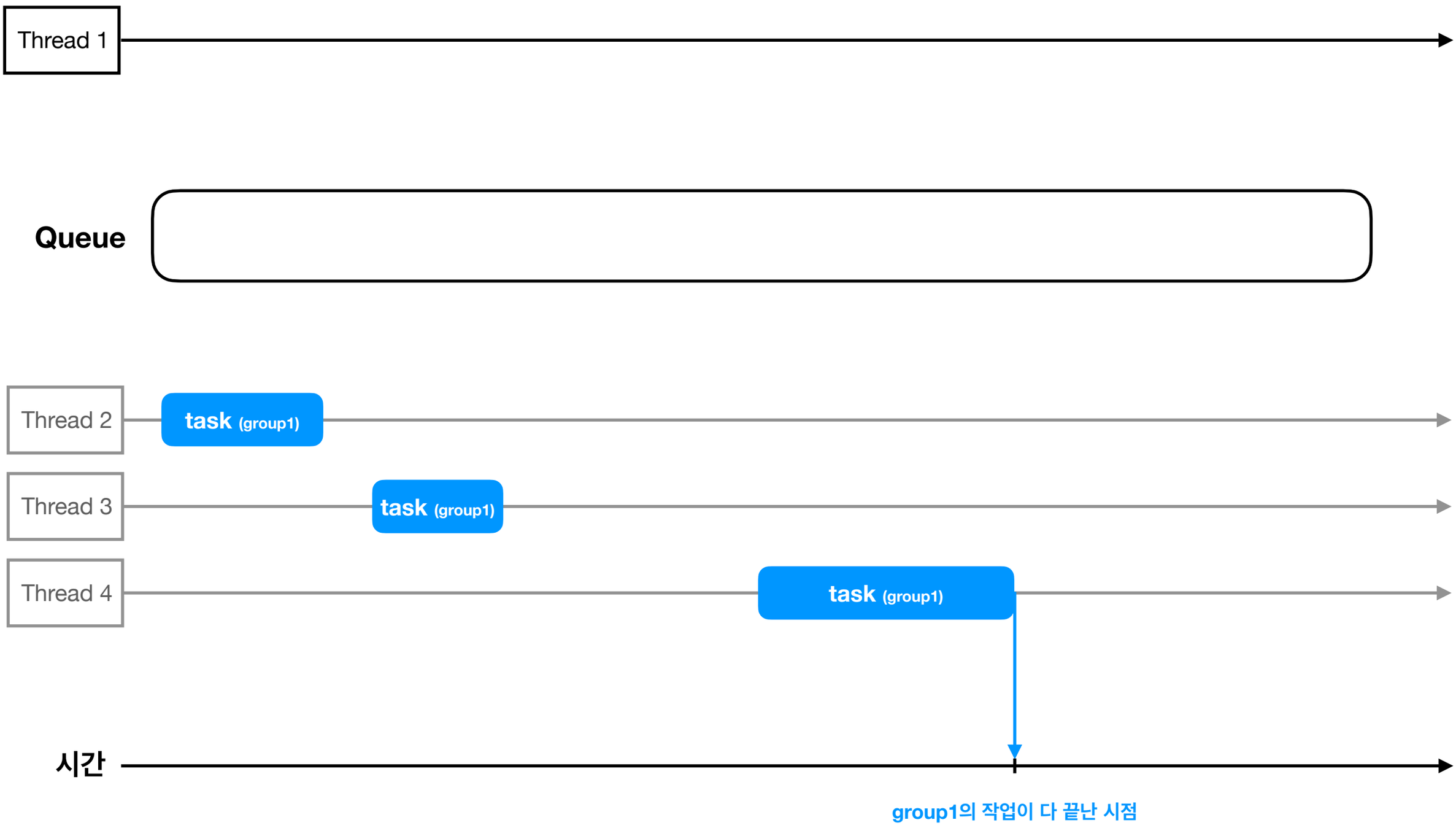


작업을 그룹 짓고, 각 그룹별로 끝나는 시점을 알고싶다.

그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

DispatchGroup



특정 그룹의 모든 작업 완료 시점을 활용

그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

(예) 여러 애니메이션 효과가 겹쳐있을때,
애니메이션이 모두 종료된 시점을 알고싶다.

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

(예) 런치스크린과 첫화면 예시

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

런치스크린 **앱의 첫화면**

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

런치스크린

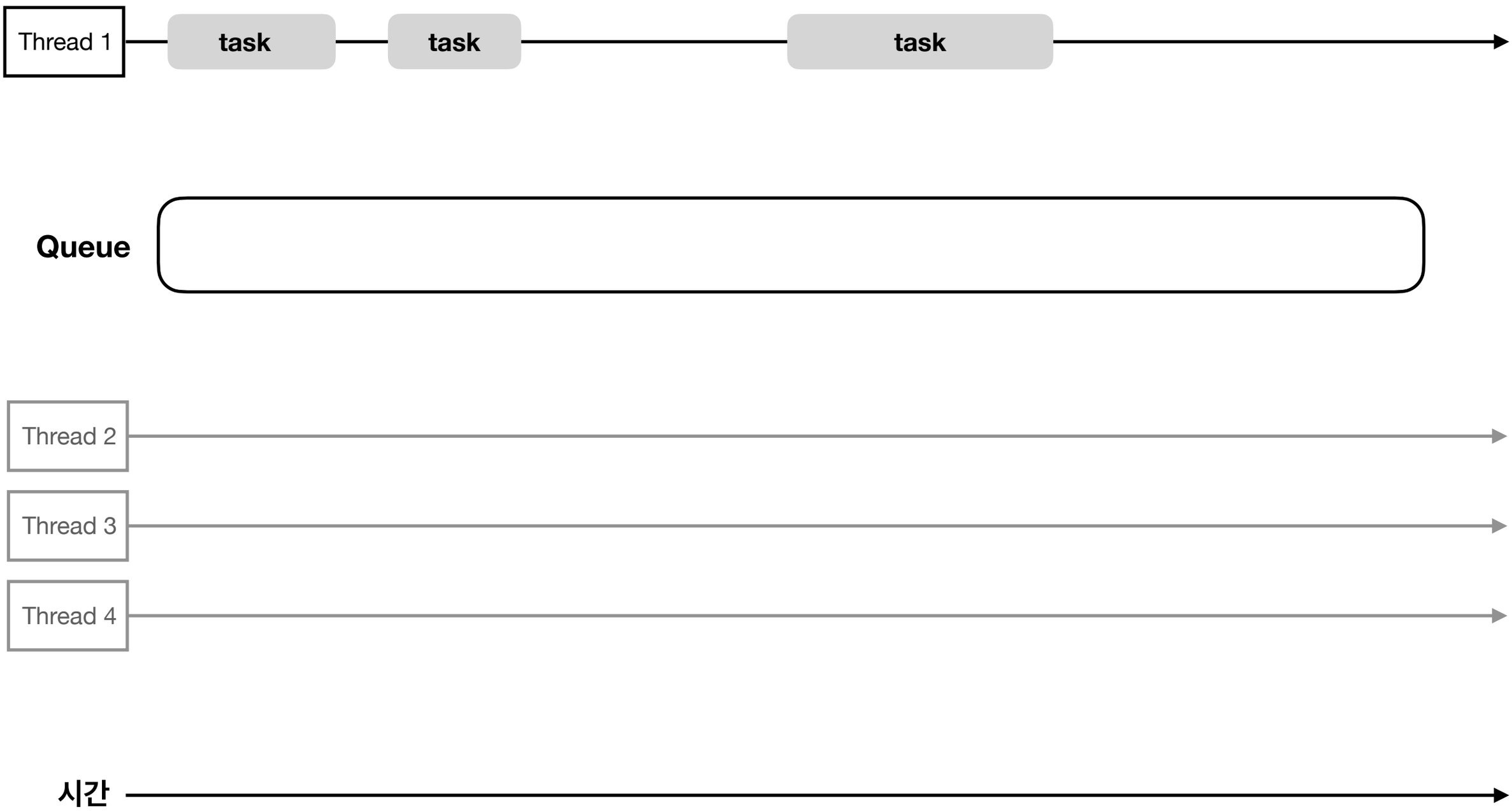
앱의 첫화면

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

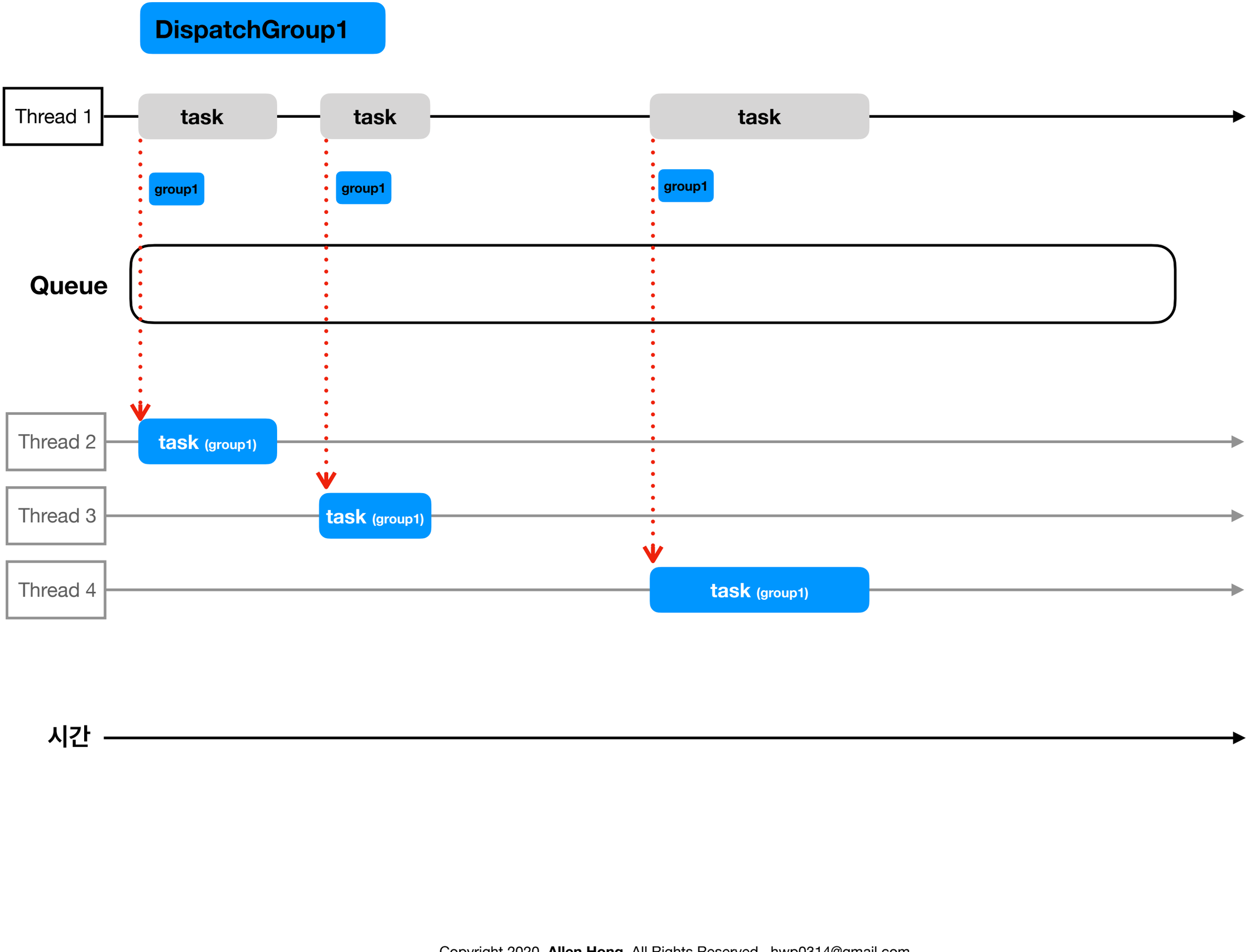
코드예시

그룹 생성하기

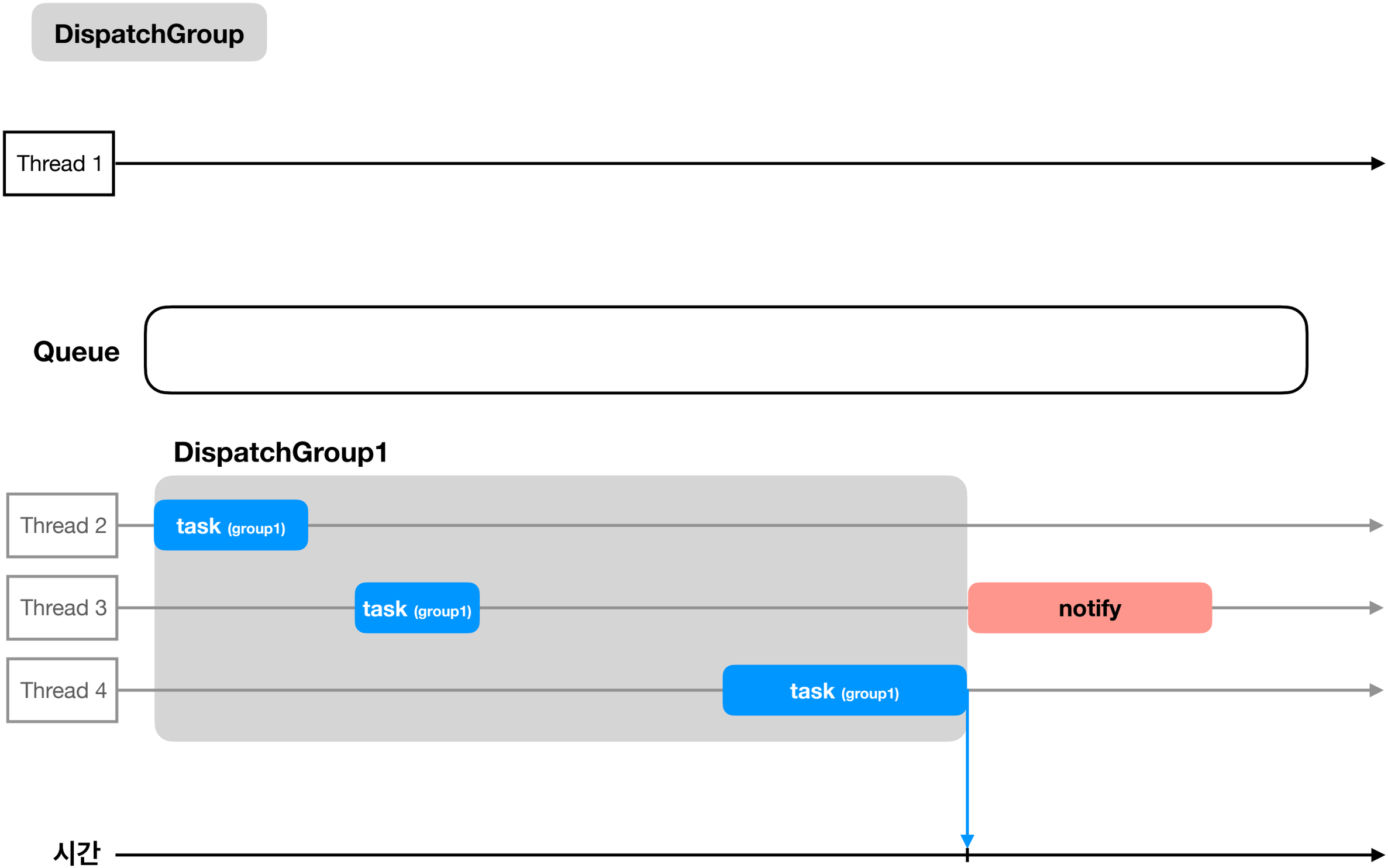
```
let group1 = DispatchGroup( )

                                        큐로 보낼때, 어떤 그룹에 넣을 것인지 정해주기
DispatchQueue.global(qos:  ).async(group: group1) {        }

DispatchQueue.global(qos:  ).async(group: group1) {        }

DispatchQueue.global( ).async(group: group1) {        }


group1.notify(queue: DispatchQueue.main) { [weak self] in
      self?.textLabel.text = “모든 작업이 완료되었습니다.”
}
```



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com



해당 그룹의 모든 작업이 끝났음을 알린다.(notify)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
let group1 = DispatchGroup( )

DispatchQueue.global(qos:  ).async(group: group1) {          }

DispatchQueue.global(qos:  ).async(group: group1) {          }

DispatchQueue.global( ).async(group: group1) {          }


group1.notify(queue: DispatchQueue.main) { [weak self] in
      self?.textLabel.text = “모든 작업이 완료되었습니다.”
}
```

DispatchGroup

Thread 1

notify

Queue

DispatchGroup1

Thread 2

task (group1)

Thread 3

task (group1)

Thread 4

task (group1)

시간

해당 그룹의 모든 작업이 끝났음을 알린다.(notify)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

4.DispatchGroup

동기적인 기다림

어떤 이유로 그룹의 완료 알림에 비동기적으로 응답 할 수 없는 경우,

대신 디스패치 그룹에서 **wait메서드를 사용**할 수 있음.

이것은 모든 작업이 완료 될 때까지 현재 대기열을 차단하는 동기적 방법.

작업이 완료 될 때까지, 얼마나 오래 기다릴지 기다리는 시간을 지정하는 선택적(optional)

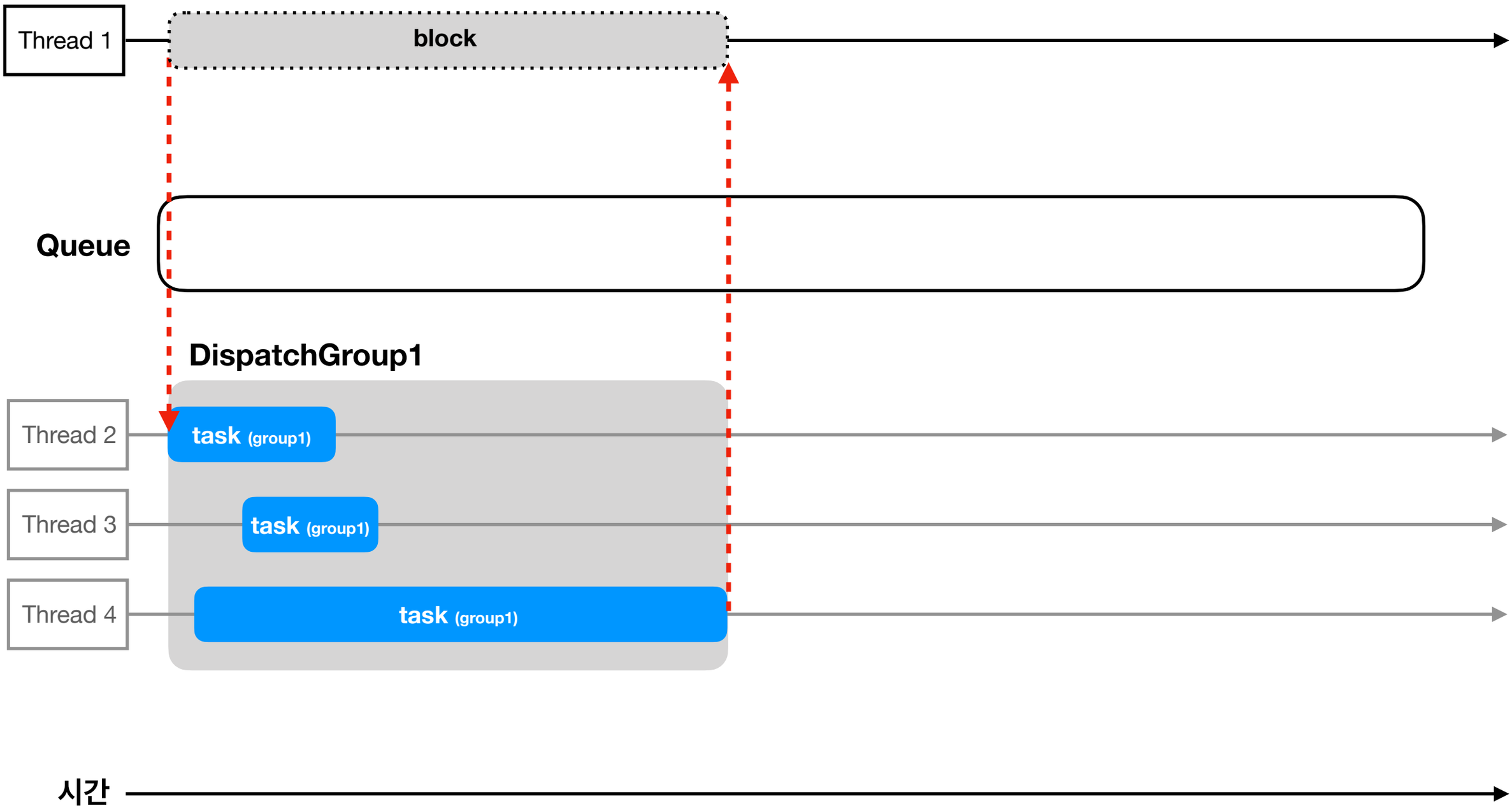
파라미터가 필요함(지정하지 않으면 무제한으로 대기)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

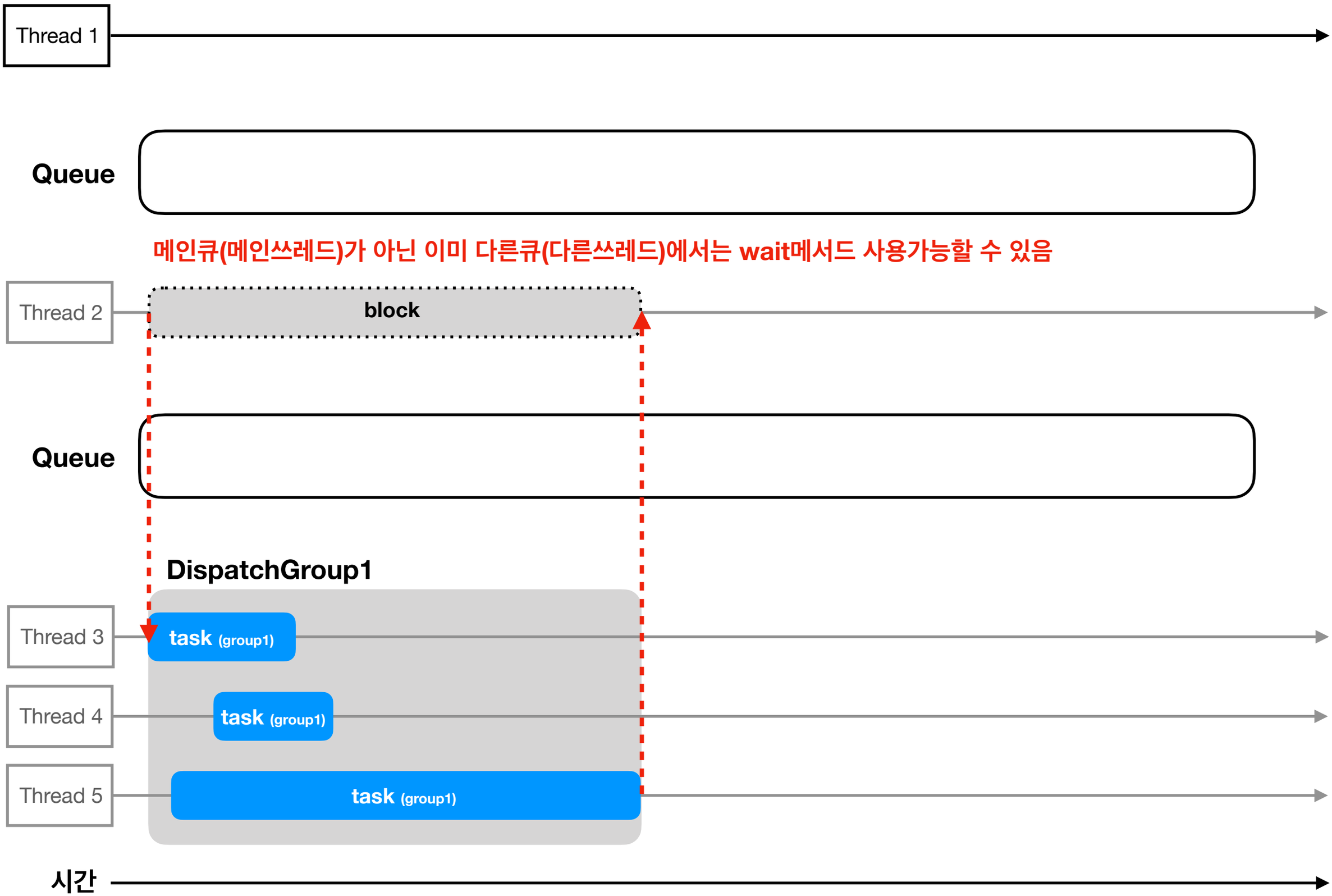
```
let group1 = DispatchGroup( )

DispatchQueue.global(qos:  ).async(group: group1) {        }

DispatchQueue.global(qos:  ).async(group: group1) {        }

DispatchQueue.global( ).async(group: group1) {          }

group1.wait(timeout:  DispatchTime.distantFuture)
```

DispatchGroup wait메서드의 사용(잘못된 사용의 경우) 실제앱에서 메인큐(메인쓰레드)에서는 wait메서드 사용하면 안됨



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

DispatchGroup wait메서드의 사용(제대로 사용하려면)

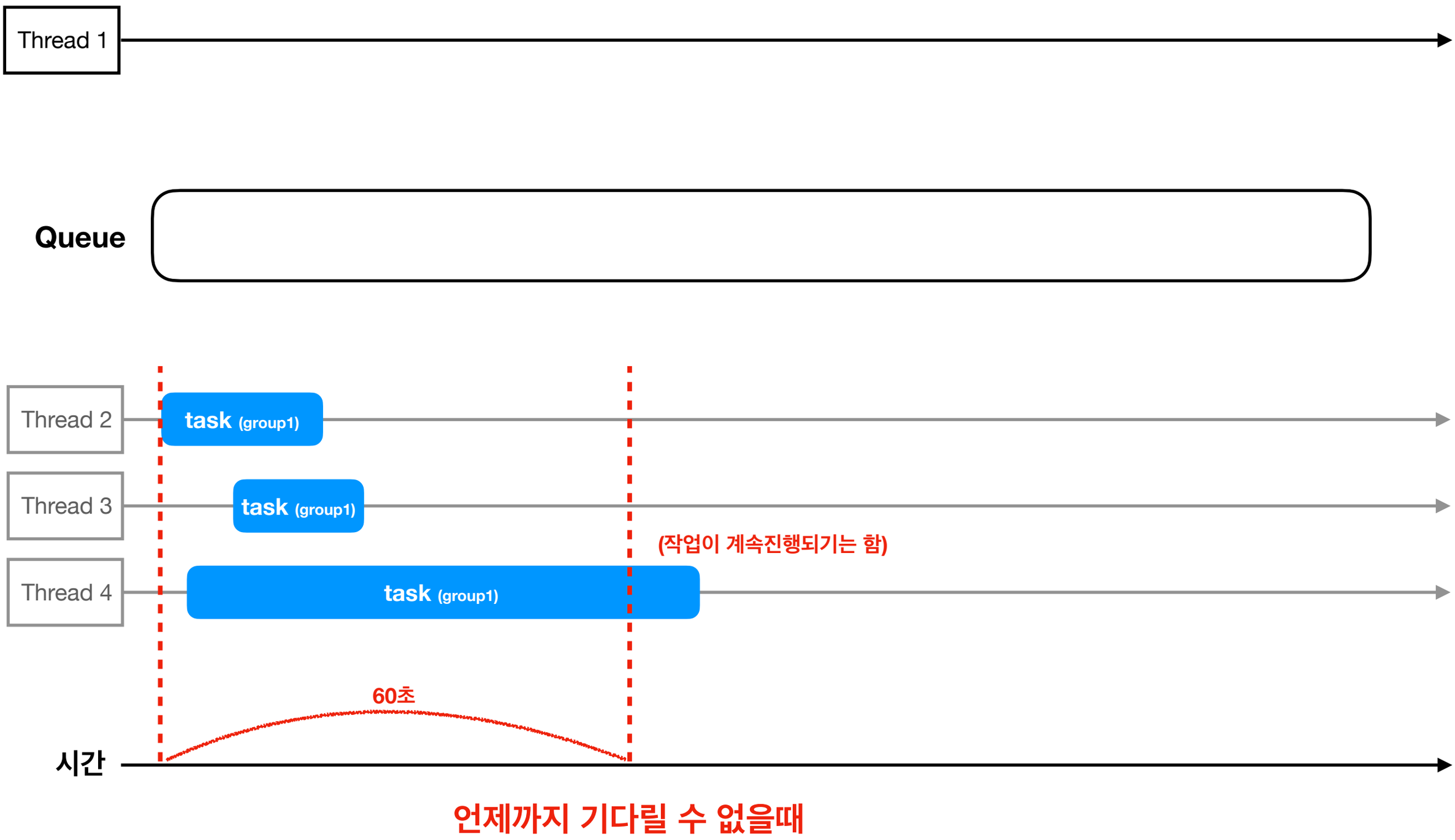


Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
let group1 = DispatchGroup( )

DispatchQueue.global(qos:  ).async(group: group1) {          }

DispatchQueue.global(qos:  ).async(group: group1) {          }

DispatchQueue.global( ).async(group: group1) {          }


if group1.wait(timeout: .now( )  +  60 ) == .timedOut {
    print(“작업이 60초안에 종료하지 않았습니다.”)
}
```

DispatchGroup



언제까지 기다릴 수 없을때

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

4.DispatchGroup

2) 디스패치 그룹의 사용

- 디스패치 그룹과 동기함수
- 디스패치 그룹과 비동기함수

코드예시

```
let group1 = DispatchGroup( )

DispatchQueue.global(qos:  ).async(group: group1) {  동기적 함수  }

DispatchQueue.global(qos:  ).async(group: group1) {  동기적 함수  }

DispatchQueue.global( ).async(group: group1) {  동기적 함수  }


group1.notify(queue: DispatchQueue.main) { [weak self] in
      self?.textLabel.text = “모든 작업이 완료되었습니다.”
}
```

코드예시

```
let group1 = DispatchGroup( )

DispatchQueue.global(qos:  ).async(group: group1) {
    task1
}

DispatchQueue.global(qos:  ).async(group: group1) {
    task2
}
```

디스패치 그룹 주의할점

(1) 동기(적인)함수

(2) 비동기(적인)함수 (주의)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

디스패치 그룹: 클로저 내에서 비동기함수를 호출할때 어떤 문제가 있을까?

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

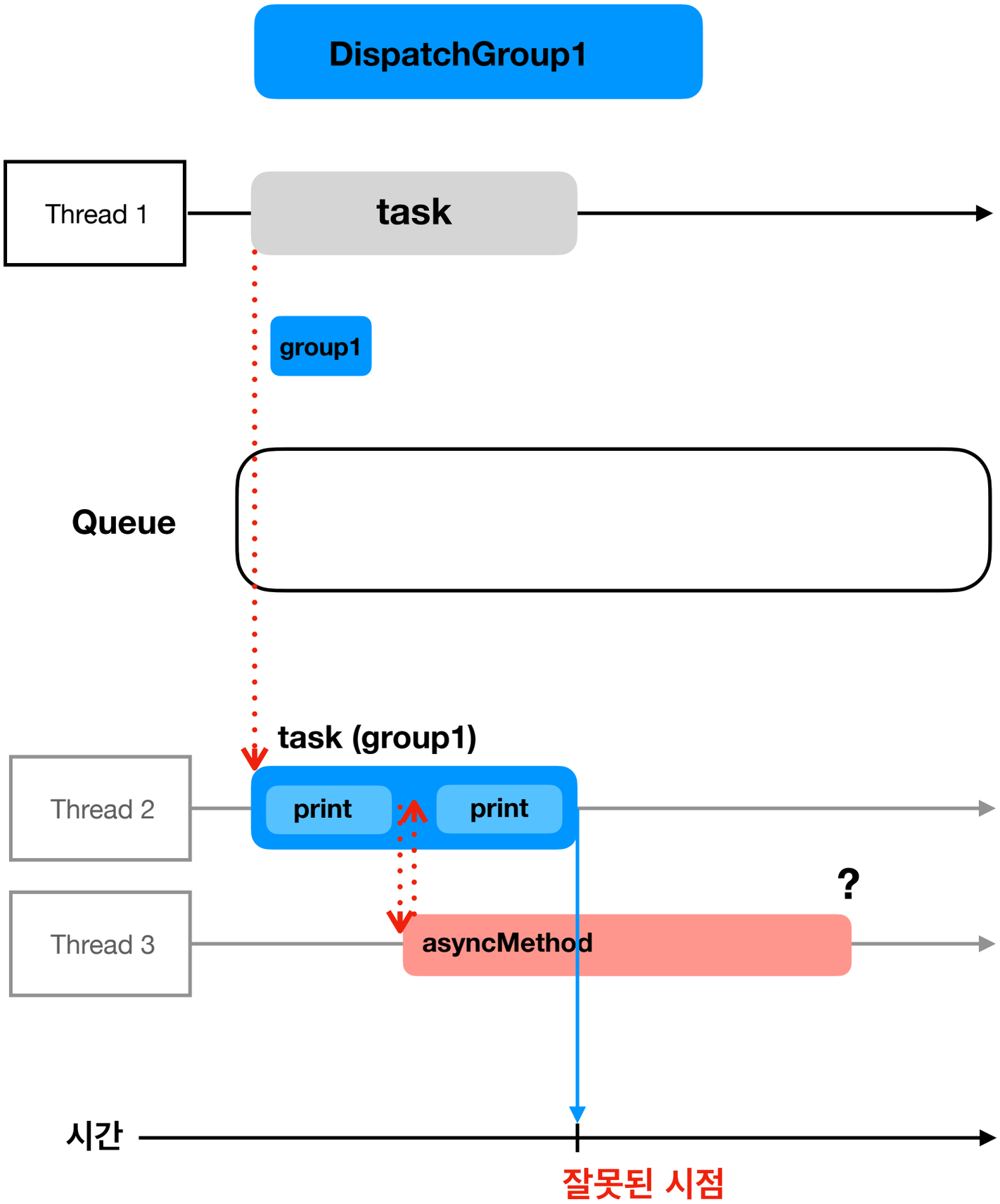
```
let group1 = DispatchGroup( )

DispatchQueue.global(qos:  ).async(group: group1) {
```

큐로 보내는 클로저 내부에

```
}
```

코드예시

```
let group1 = DispatchGroup( )

DispatchQueue.global(qos:  ).async(group: group1) {

        print(“async group task started”)
        asyncMethod(input: url) { result in
            ...

        }
        print(“async group task finished”)

}
```



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

DispatchGroup1

Thread 1

task

group1

Queue

group1.enter( )

입장1 : 퇴장1

task (group1)

Thread 2

print

print

Thread 3

asyncMethod

group1.leave( )

시간

제대로 일이 끝난 시점

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
queue.async(group: group1) {
    group1.enter( )
    someAsyncMethod {
        group1.leave( )
    }
}
```

입장 1

퇴장 1

종료

“비동기” 디스패치 그룹 함수를 만들 수 있다.

(그룹을 파라미터로 비동기 디스패치 그룹 함수화)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

4-2.AsyncDispatchGroup

실제 코드예시

4-2.GroupAnimationExample

실제 코드예시

4-2.AnotherExample

4) (참고) Dispatch workItem

<Dispatch workItem> 워크 아이템 (Class)

1) 작업을 미리 정의해 놓고 사용하는, 큐에 제출하기 위한 객체

2) 빈약한 <취소 기능>을 내장

3) 빈약한 <순서 기능>을 내장

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
let item1 = DispatchWorkItem(qos: .utility)  {
        print(“task1: 출력하기”)
        print(“task2: 출력하기”)
}

let item2 = DispatchWorkItem  {
        print(“task3: 출력하기”)
        print(“task4: 출력하기”)
}

let queue = DispatchQueue(label: “com.inflearn.serial”)

queue.async(execute: item1)
queue.async(execute: item2)
```

2) 빈약한 <취소 기능>을 내장

cancel( )메서드 존재

1. 작업이 아직 시작 안된 경우(아직 큐에 있을때) - 작업이 제거됨

2. 작업이 실행중인 경우 - isCancelled속성이 true로 설정됨
(직접적으로 실행중인 작업이 멈추는 것은 아님)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
let item1 = DispatchWorkItem(qos: .utility)  {
        print(“task1: 출력하기”)
        print(“task2: 출력하기”)
}

item1.cancel( )
```

```
let queue = DispatchQueue(label: “com.inflearn.serial”)

queue.async(execute: item1)
queue.async(execute: item2)
```

3) 빈약한 <순서 기능>을 내장

notify(queue: 실행할큐, execute: 디스패치아이템)메서드 존재

(직접적으로 실행 다음에, 실행할 아이템(작업)을 지정)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
let item1 = DispatchWorkItem(qos: .utility)  {
        print(“task1: 출력하기”)
        print(“task2: 출력하기”)
}

let item2 = DispatchWorkItem  {
        print(“task3: 출력하기”)
        print(“task4: 출력하기”)
}

item1.notify(queue: DispatchQueue.global( ), execute: item2)

queue.async(execute: item1)
```

실제 코드예시

4-3.DispatchWorkItem

4) (심화) Semaphore의 이해

<Semaphore> 수기 신호

공유 리소스에 접근가능한 작업 수를 제한해야할 경우

(예: 다운로드 숫자를 제한해야 할때)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
let semaphore = DispatchSemaphore(value: 3)

queue.async(group: group1) {
        group1.enter( )
        semaphore.wait( )
        someAsyncMethod {
                group1.leave( )
                semaphore.signal( )
        }
}
```

Semaphore

작업은 최대 한번에 3개씩만 실행 가능

Thread 1

3개

Queue

task1 wait( )

task2 wait( )

task3 wait( )

task4 wait( )

task5 wait( )

task6 wait( )

task7 wait( )

Thread 2

Thread 3

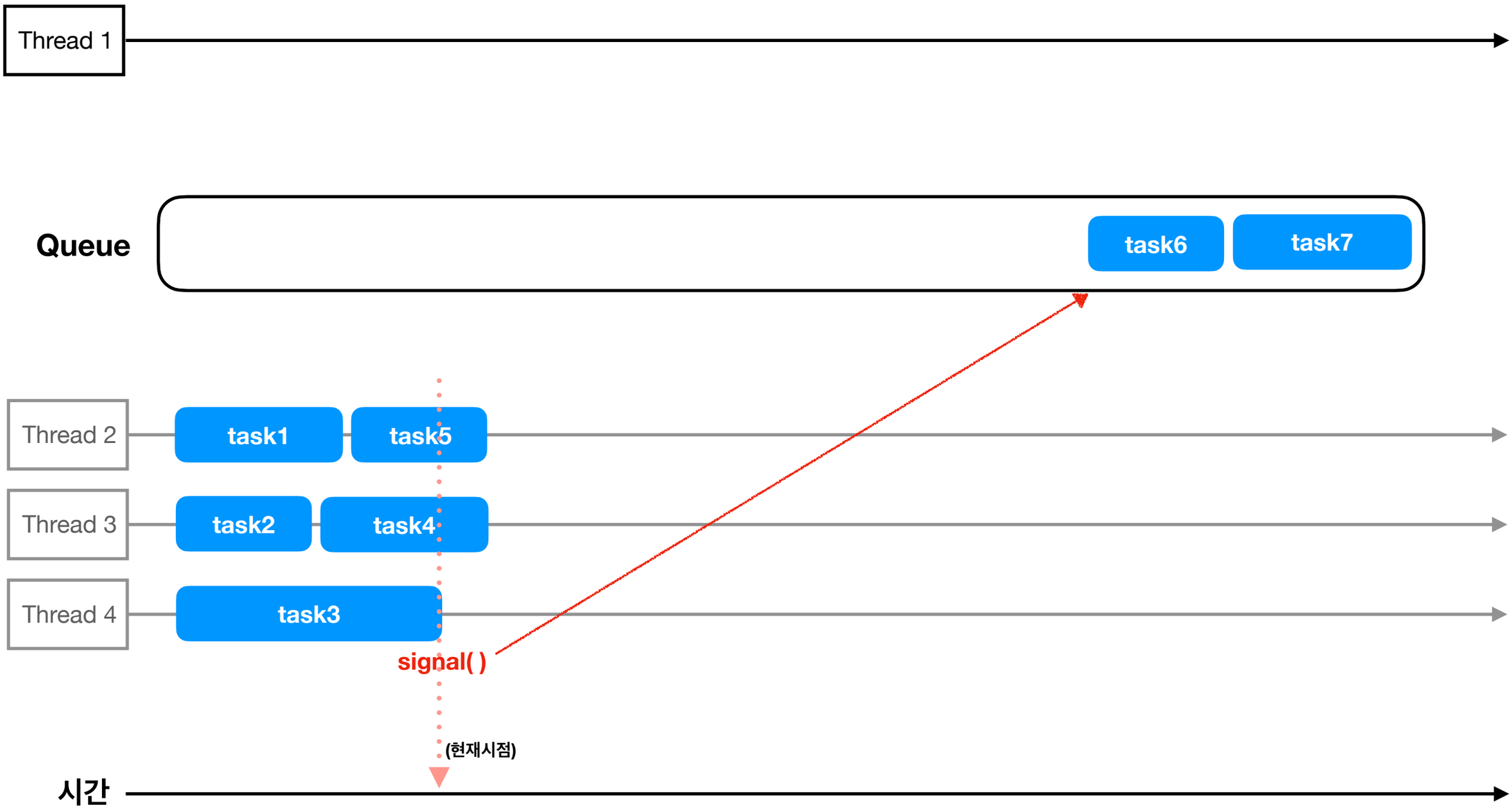
Thread 4

시간

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Semaphore

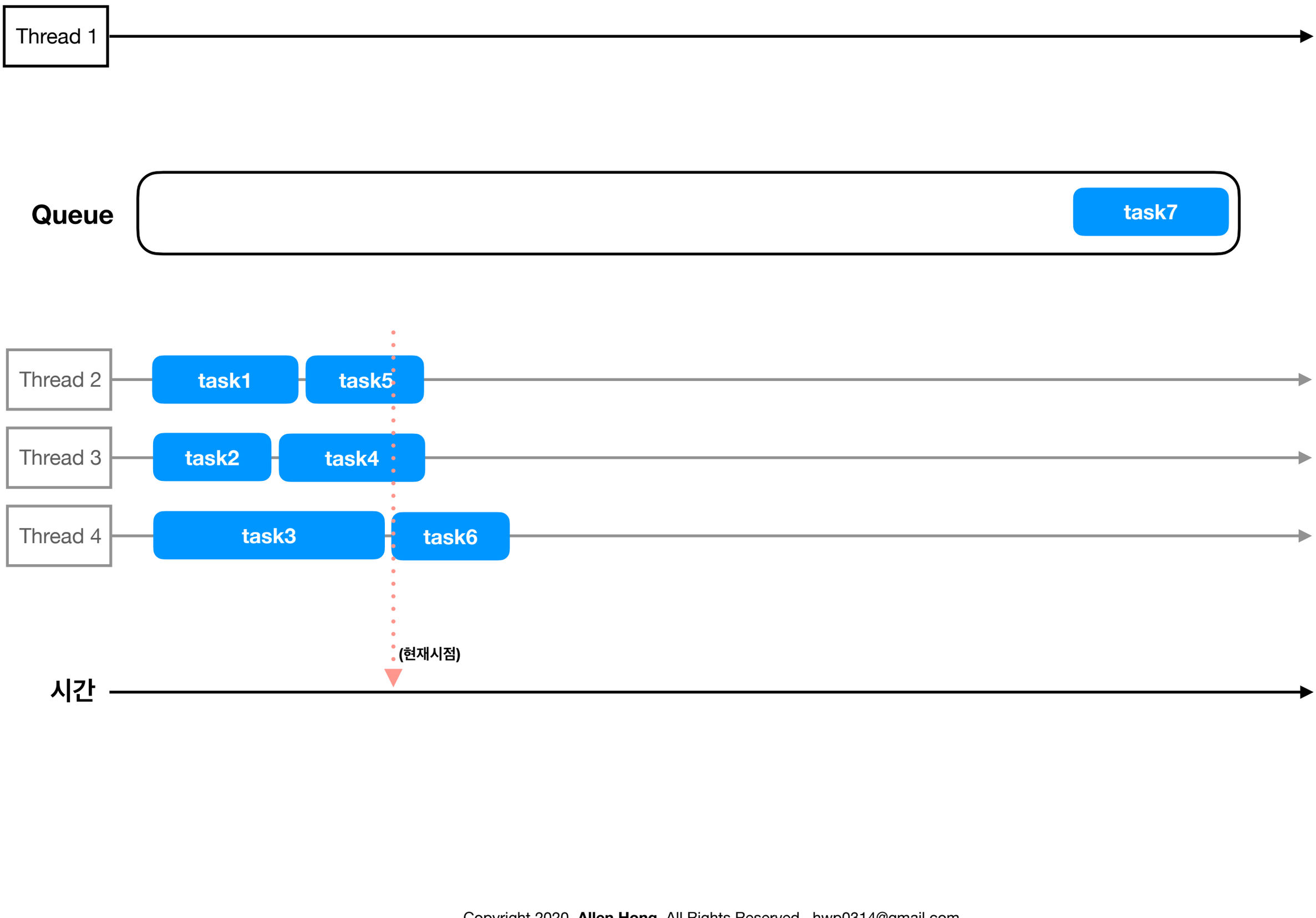
작업은 최대 한번에 3개씩만 실행 가능



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Semaphore

작업은 최대 한번에 3개씩만 실행 가능



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
let semaphore = DispatchSemaphore(value: 3)

queue.async(group: group1) {
    group1.enter( )
    semaphore.wait( )
    someAsyncMethod {
        group1.leave( )
        semaphore.signal( )
    }
}
```

일단기다려: - 1 = 2

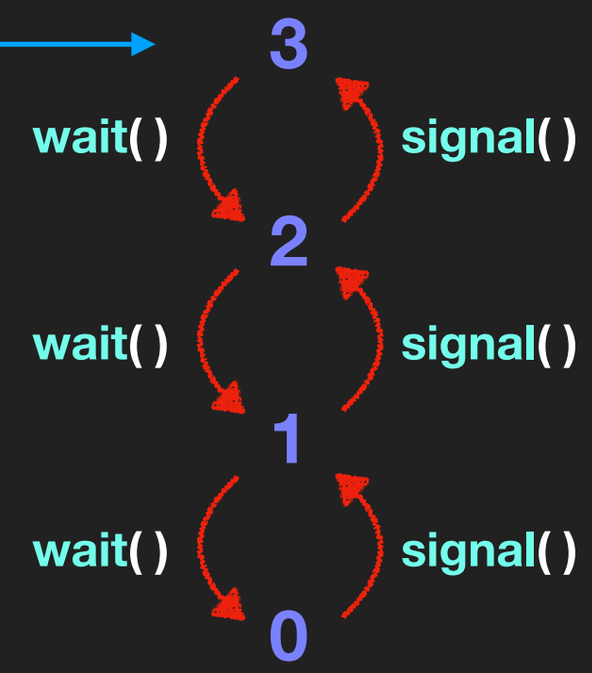
신호줄께: + 1 = 3

(이제 써도돼)

현재쓸 수 있는 자원 수

코드예시

```
let semaphore =
        DispatchSemaphore(value: 3)
```

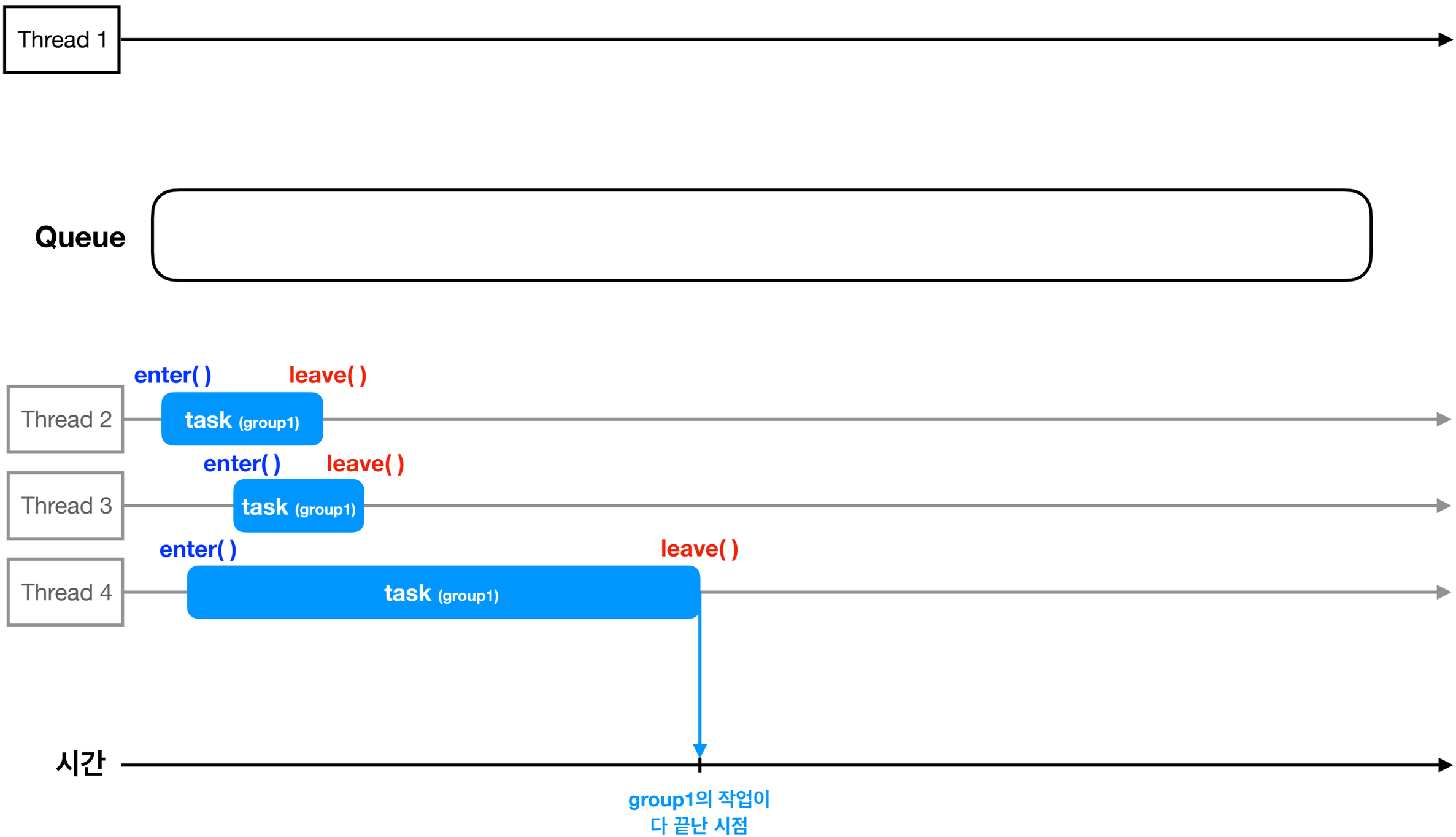


실제 코드예시

4-4.DispatchSemaphore

(비동기)디스패치 그룹과 세마포어 비교

(비동기)디스패치 그룹



group1에 관련된 모든 작업이 끝나는 시점

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

(비동기)디스패치 그룹과 세마포어 비교 세마포어

Thread 1

작업은 최대 한번에 3개씩만 실행 가능

let semaphore = DispatchSemaphore(value: 3)

Queue

task4 wait( )

task5 wait( )

task6 wait( )

task7 wait( )

Thread 2

wait( )

task1

Thread 3

wait( )

task2

signal( )

Thread 4

wait( )

task3

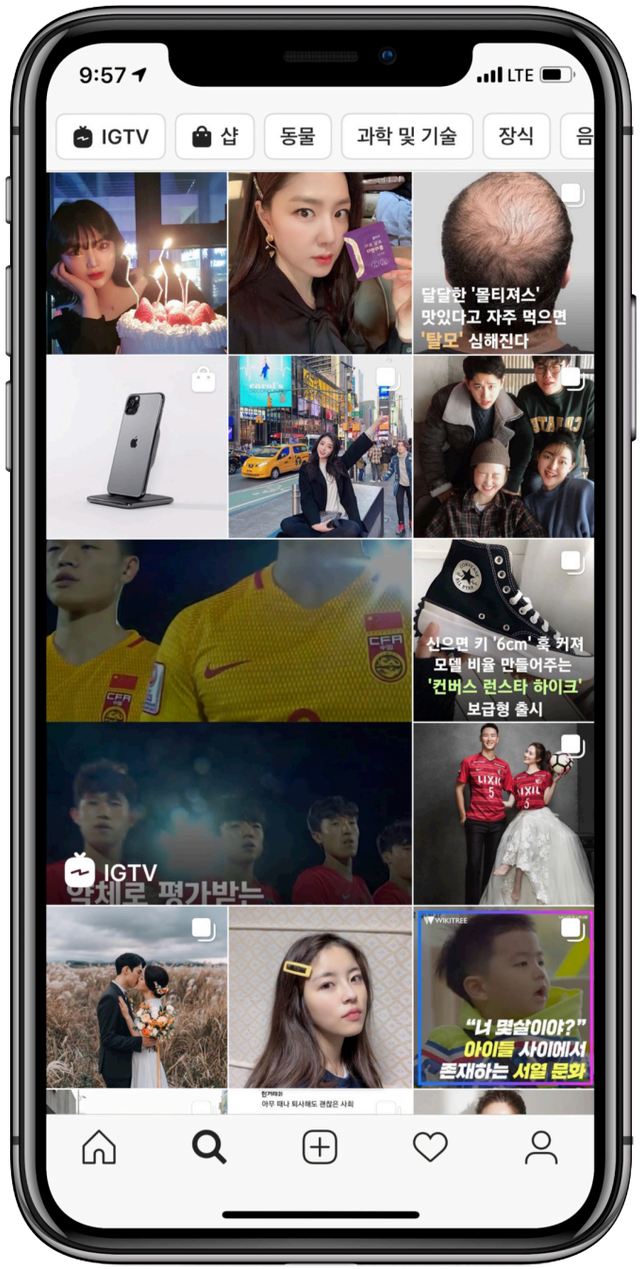
(현재시점)

시간

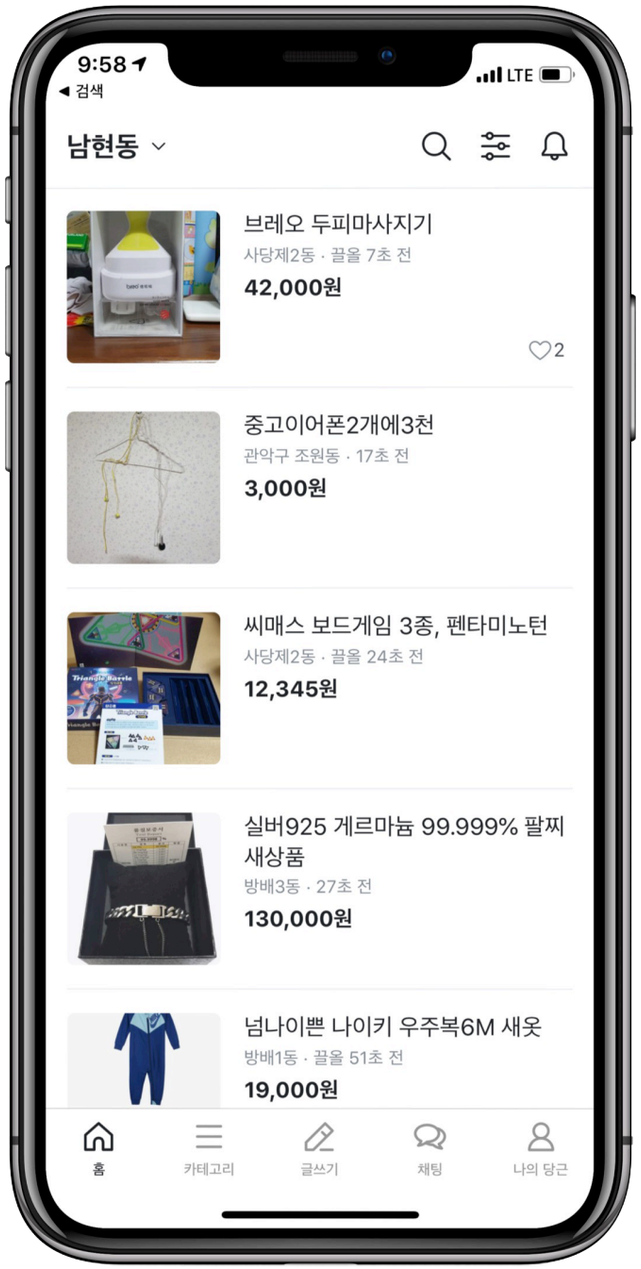
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

5. (참고) GCD 프로젝트 살펴보기

넷플릭스



인스타그램



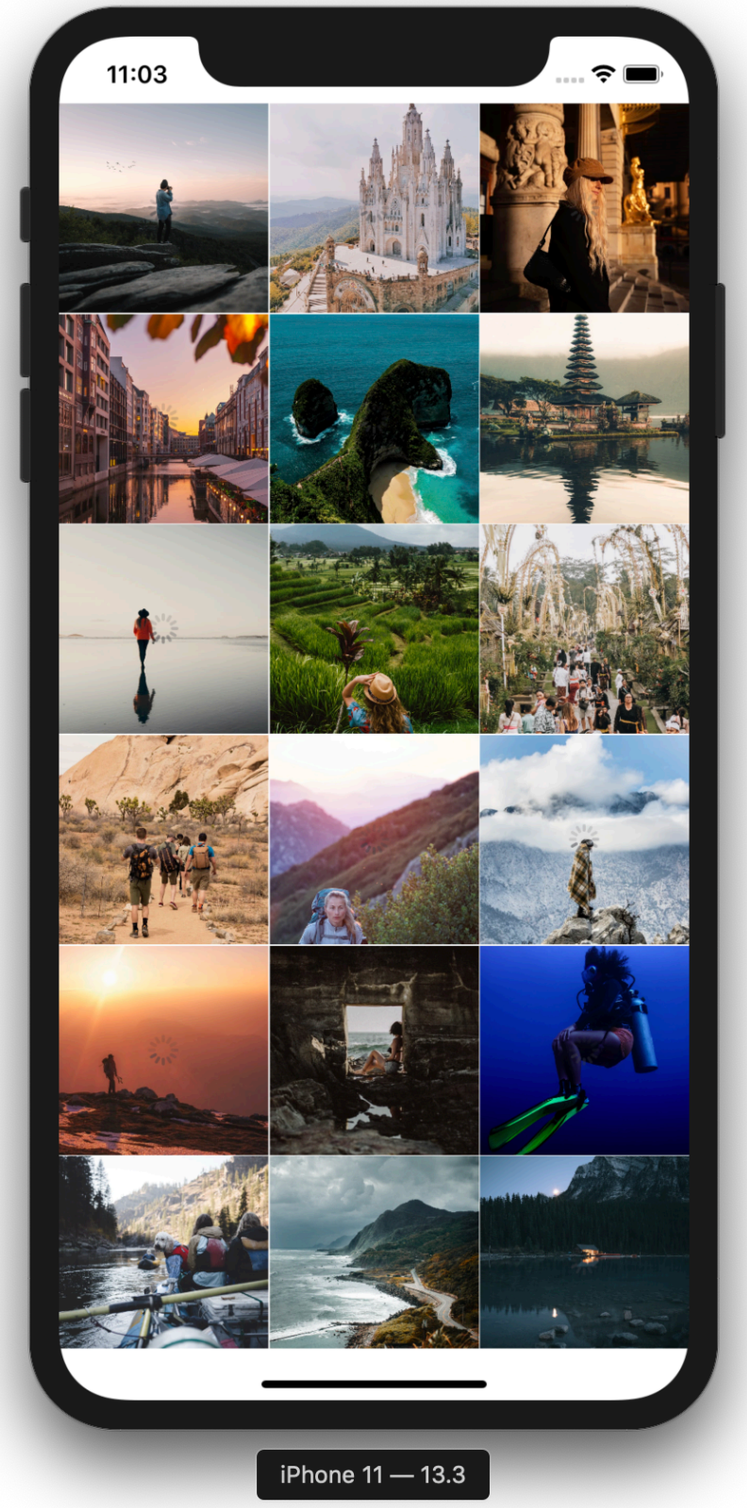
당근마켓

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

일단 각각의 셀이 네트워크에서 받아오는 이미지 등이면
설계 시, 무조건 주의해야한다는 염두를 해야함

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

GCD프로젝트 폴더 2.GCD-Project참고 / 3.Concurrency-Better



starter

동시성처리
못한다는 가정

cellForItemAt
메인쓰레드에서 처리중

(다운로드 및 이미지표시)

final

개 선

다른메서드에서
동시성 처리중
(IndexPath전달)

인디케이터 X

이미지 캐시처리 X

(동시성처리만의
목적에는 조금 부합)

3.Concurrency - Better

개 선

셀의 CustomImageView에서
다운로드 및
이미지표시

<동시성처리>

안함	(컬렉션)뷰컨트롤러	각 셀

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

폴더 2.GCD-Project참고 / 3.Concurrency-Better

6. (심화) 동시성과 관련된 문제들 (Concurrency Problems)

---

1) Race Condition (경쟁상황)

2) Deadlocks (교착상태)

3) Priority Inversion (우선 순위의 뒤바뀜)

2개이상의 쓰레드를 사용하면서, 동일한 메모리 접근 등으로 인해 발생할 수 있는 문제

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Thread-Safety

- Thread-safe(쓰레드 세이프)의 뜻 : 여러 쓰레드가 동시에 쓰여도 안전하다.(괜찮다.)
- 동시적 처리를 하면서(여러 스레드를 사용하면서도) 문제없이 스레드를 안전하게 사용

“데이터(객체나 변수 등)에 여러 쓰레드를 사용하여 접근하여도, 한번에 한개의 쓰레드만 접근가능하도록 처리하여 경쟁상황의 문제없이 사용”

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Concurrency Problems

1) Race condition (경쟁적인 상황)
2) Deadlocks (교착 상태)
3) Priority inversion (우선순위의 뒤바뀜)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실행시마다 항상 같은 순서로 발생하는 것이 아니기 때문에,

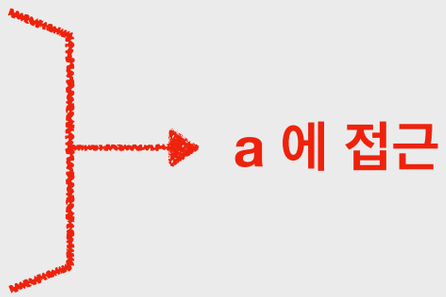
디버그 하기 어려움.

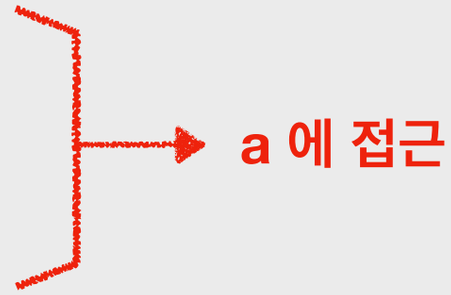
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

1) Race Condition (경쟁상황)

1) Race Condition(경쟁적인 상황)

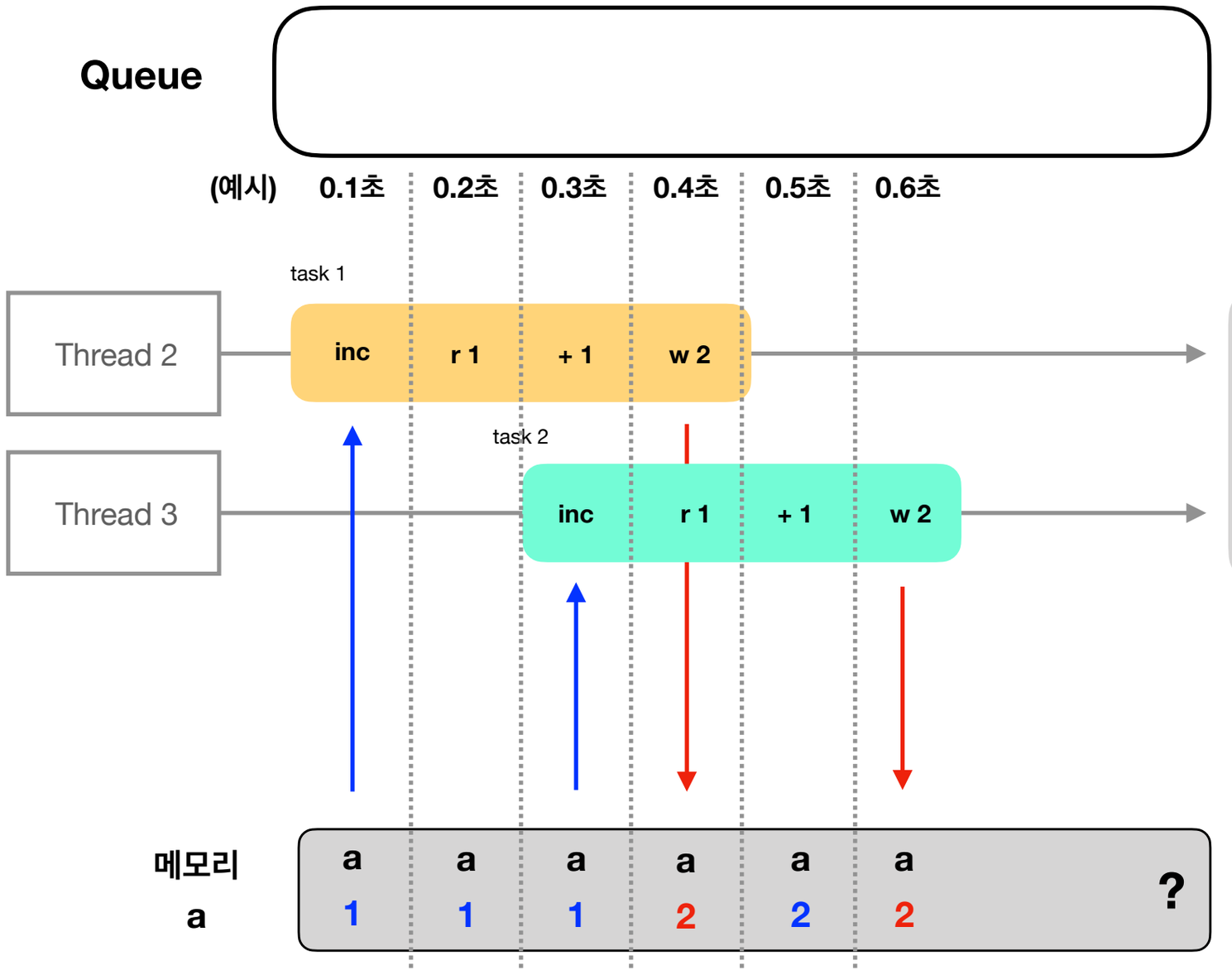
```
var a = 1

DispatchQueue.global( ).async {
    sleep(1)
    a += 1
}

DispatchQueue.global( ).async {
    sleep(1)
    a += 1
}

print(a)
```





Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

1) Race Condition(경쟁적인 상황)



두개이상의 쓰레드가
한 곳의 메모리(저장공간)에
동시에 접근하여 값을 사용하려고
할때 문제가 발생할 수 있다.

그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

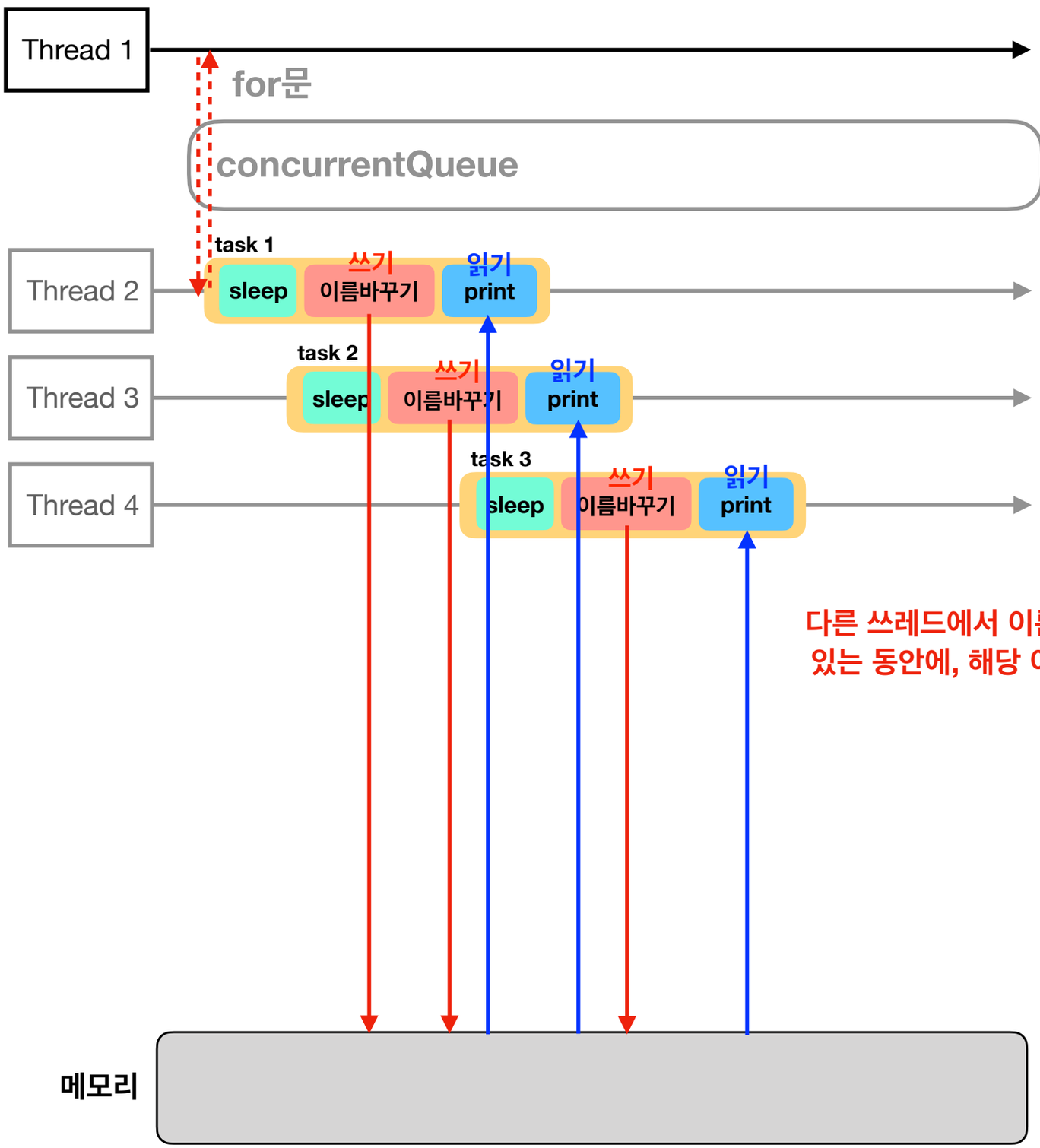
실제 코드예시

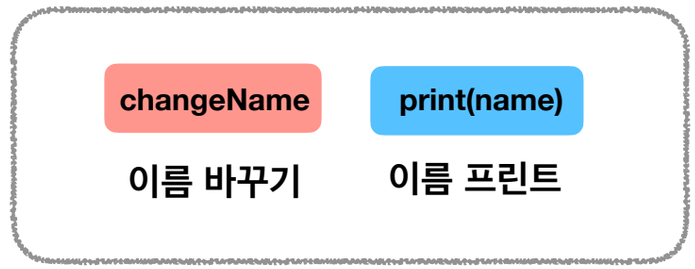
6-1.RaceCondition

실제 코드예시

6-1.AnotherRaceCondition

코드를 그림으로 표현 6-1.AnotherRaceCondition





적정한 프린트가 이루어지지 않는 이유:
여러쓰레드에서 동시다발적으로
쓰기(changeName) 및 읽기(print)가 발생하고 있음

여러쓰레드가 경쟁적으로 데이터에 접근(Race Condition)

Person 클래스의 내부는 동기적으로 구현된 것처럼 보이지만, 실제 해당 객체에 접근하는 쓰레드가 더 중요

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

<Race Condition(경쟁 상황) 문제 해결은?>

다음장의 Thread-safe한 코드의 구현과 방법에서 자세하게 해결 예정

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

2) Deadlocks (교착상태)

2) Deadlock(교착 상태)

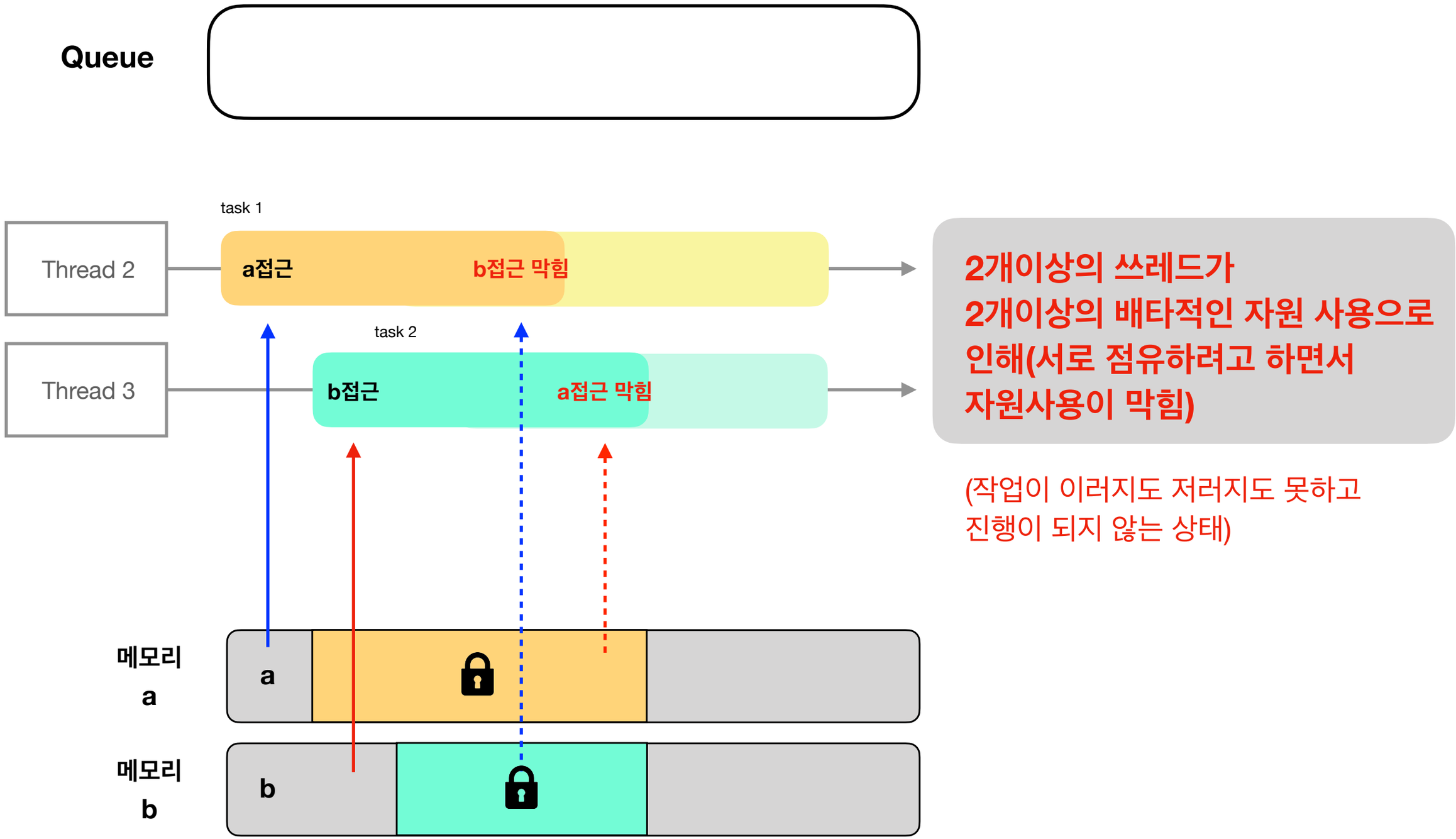


그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

< Deadlocks (교착상태)의 다양한 발생 가능성>

- 동기 작업이 현재의 쓰레드가 필요한 경우
  (3-2. sync 메서드에 대한 주의사항 참고)

- 앞선 작업이 현재의 쓰레드가 필요한 경우

- 여러개의 세마포어가 존재할때, 순서 잘못 설계 등

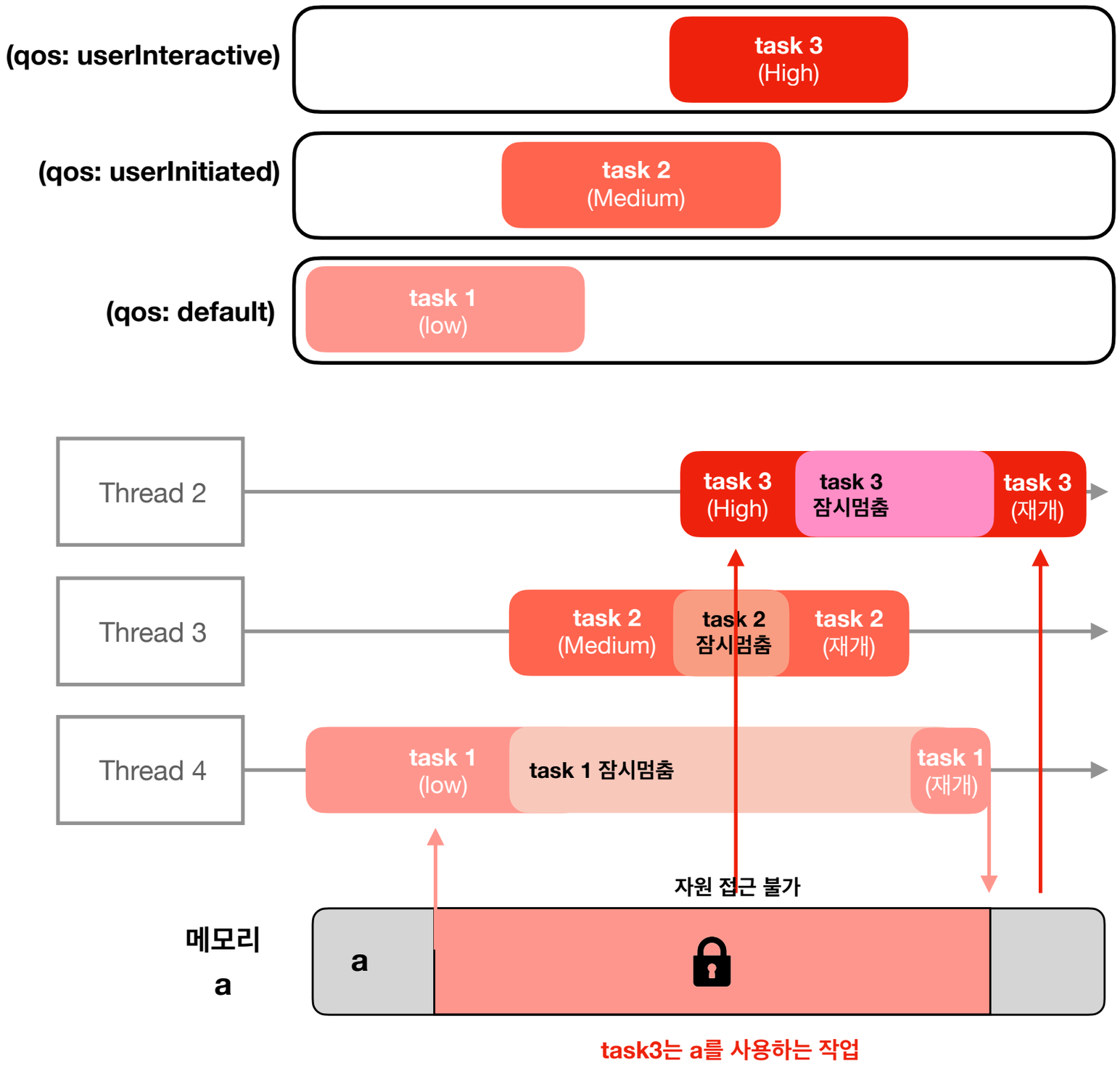
<문제 해결은?>

시리얼 큐로 해결 가능
(그러나, 세마포어나 제한된 리소스 순서 같은 것 조심히 사용해야하고, 객체 등 설계 시에 각별히 주의)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

3) Priority Inversion (우선 순위의 뒤바뀜)

3) Priority Inversion(우선 순위의 뒤바뀜)



task3는 a를 사용하는 작업

task2 ➡ task1 ➡ task3

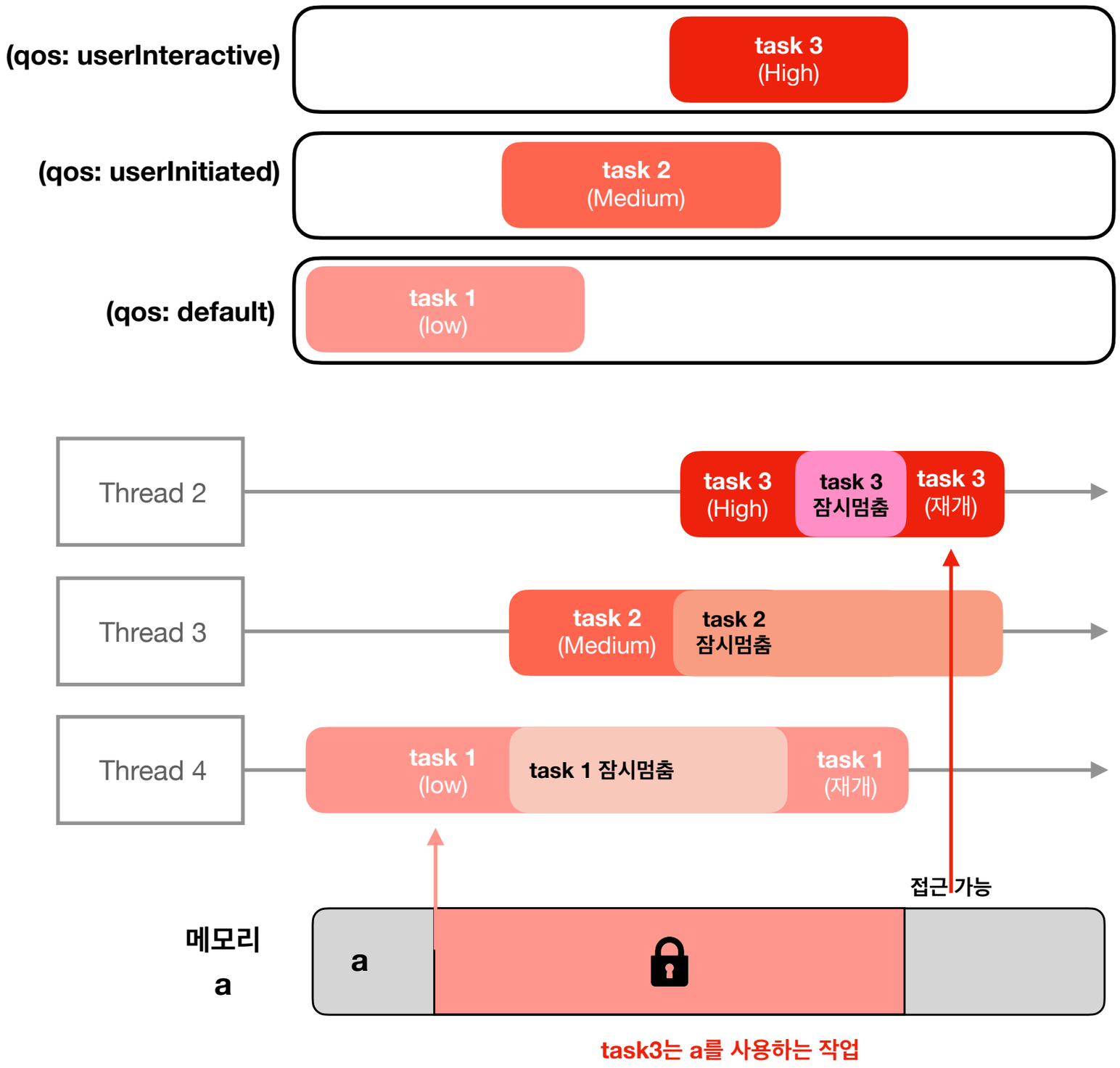
낮은 우선 순위의 작업이
자원을 배타적으로 사용하고
있을 때, (다른 작업이 자원을
사용하지 못하게 막고 있으므로)

작업의 우선 순위가 바뀔 수 있음

그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

3) Priority Inversion(우선 순위의 뒤바뀜)



(A) GCD가 알아서 우선 순위 조정

task3가 사용해야할 자원을 잠그고 있는 **task1**의 우선순위를 **High**(userInteractive)로 올려 **task1**가 우선적으로 일을 끝내도록 한다.

(실제로) task1 ➡ task3 ➡ task2

그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

< Priority Inversion(우선 순위 뒤바뀜)의 다양한 발생 가능성>

- 시리얼큐에서 높은 우선순위 작업이 낮은 우선순위의 뒤에 보내지는 경우

- 낮은 우선순위의 작업이 높은 우선순위가 필요한 자원을 잠그고 있는 경우
(예: lock코드, 세마포어 등) (설계시 주의)

- 높은 우선순위 작업이 낮은작업에 의존하는(디펜던시) 경우 **(Operation)**

<문제 해결은?>

1차적으로 GCD가 우선순위를 조정해서 알아서 해결

(낮은 우선순위 작업의 우선순위를 높여서 우선처리) **(실질적으로는) task1 ➡ task3 ➡ task2**

(안전하게) 공유된 자원 접근시 - 동일한 QoS 사용

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

6-3.PriorityInversion

Concurrency Problem

구 분	정 의	문제의 해결
(1) 경 쟁 상 황 (Race Condition)	2개이상의 쓰레드에서 공유된 데이터(변수나 객체)에 동시적으로 접근하는 경우	확인방법: TSan (editScheme -> Run -> Diagnostics -> Thread Sanitizer) **(1) 시리얼큐 + Sync (엄격한 Thread-safe)** **(2) 디스패치 배리어 작업 (조금더 효율적인 방법)** (추가적으로, Semaphore이용 동시실행자원도 제한가능) **다만, 객체 등 설계시에 주의해서 사용해야함**
(2) 교 착 상 태 (DeadLock)	한정된 자원을 여러 쓰레드에서 사용하려고 할때, 자원을 얻지 못해 다음 처리를 위한 진행을 못하고 있는 상태	**(1)** (단순한 처리방법으로) **시리얼큐로 해결 가능** **(2) 세마포어 등 제한된 리소스 순서 조심히 사용/ 객체 등 설계시 주의**
(3) 우선순위 뒤바뀜 (Priority Inversion)	다양한 경우에서, 작업의 Qos(서비스 품질)이 뒤바뀌어서 작업이 진행되는 경우	**(1) 1차적으로, GCD가 자원을 점유하고 있는 낮은 우선순위 작업의 우선순위를 높여서 처리.** **(2) 공유된 자원에 대한 접근시 동일한 Qos를 사용해야함.**

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Thread-Safety

- Thread-safe(쓰레드 세이프)의 뜻 : 여러 쓰레드가 동시에 쓰여도 안전하다.(괜찮다.)
- 동시적 처리를 하면서(여러 스레드를 사용하면서도) 문제없이 스레드를 안전하게 사용

“데이터(객체나 변수 등)에 여러 쓰레드를 사용하여 접근하여도, 한번에 한개의 쓰레드만 접근가능하도록 처리하여 경쟁상황의 문제없이 사용”

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

<Tread-safe한 코드의 구현과 방법>

Lock으로 올바르게 구현하는 것은 어렵기도하고, **교착상황(Deadlocks)**이 생길 가능성이 높음

(낮은 수준의 해결책)

참고: https://medium.com/@dmytro.anokhin/concurrency-in-swift-reader-writer-lock-4f255ae73422

참고: https://medium.com/cubo-ai/concurrency-thread-safety-in-swift-5281535f7d3a

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

7. (심화) Tread-safe한 코드의 구현과 방법

---

1) TSan 사용법

2) 시리얼큐와 sync

3) 디스패치 배리어(Dispatch Barrier)

4) 객체 설계 시에 주의할 점

1) TSan 사용법

(잠재적 경쟁 상황 찾는 법)

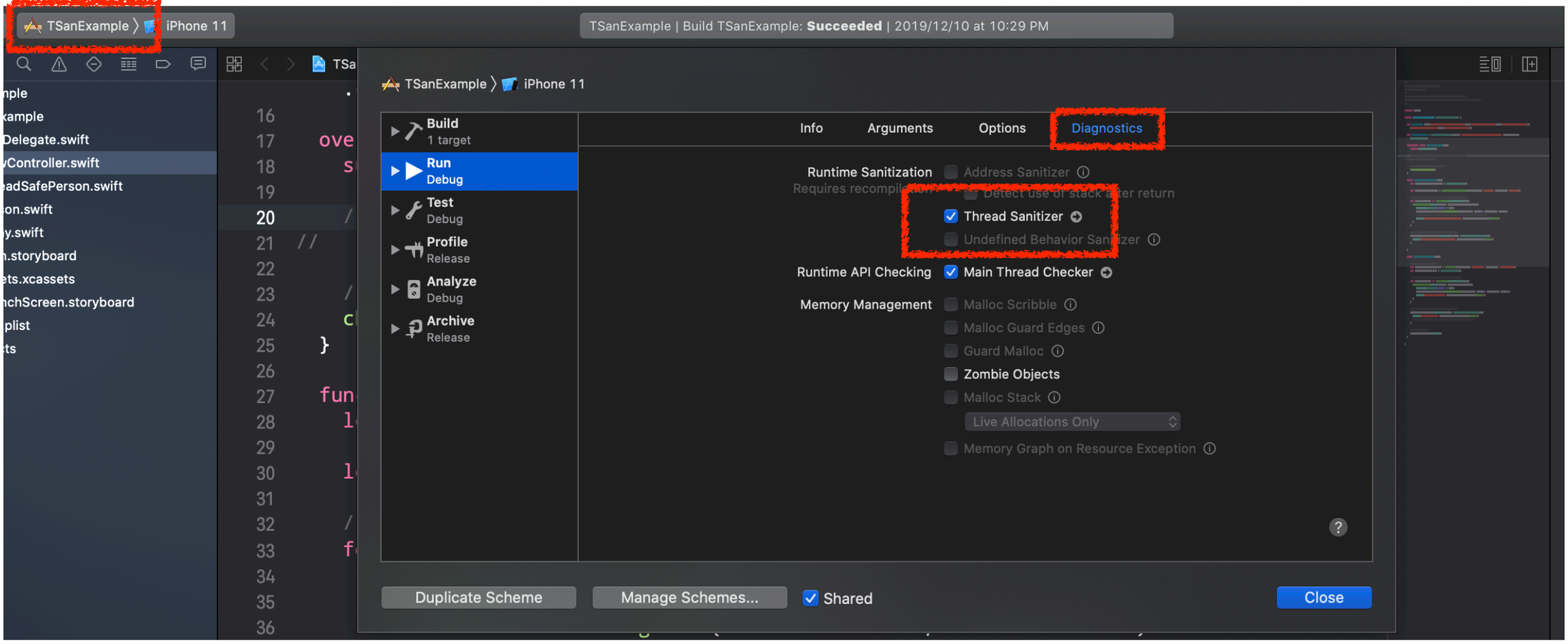
[thread sanitizer tool]

(소독제)

<앱스토어 출시전 반드시 해야할 일>

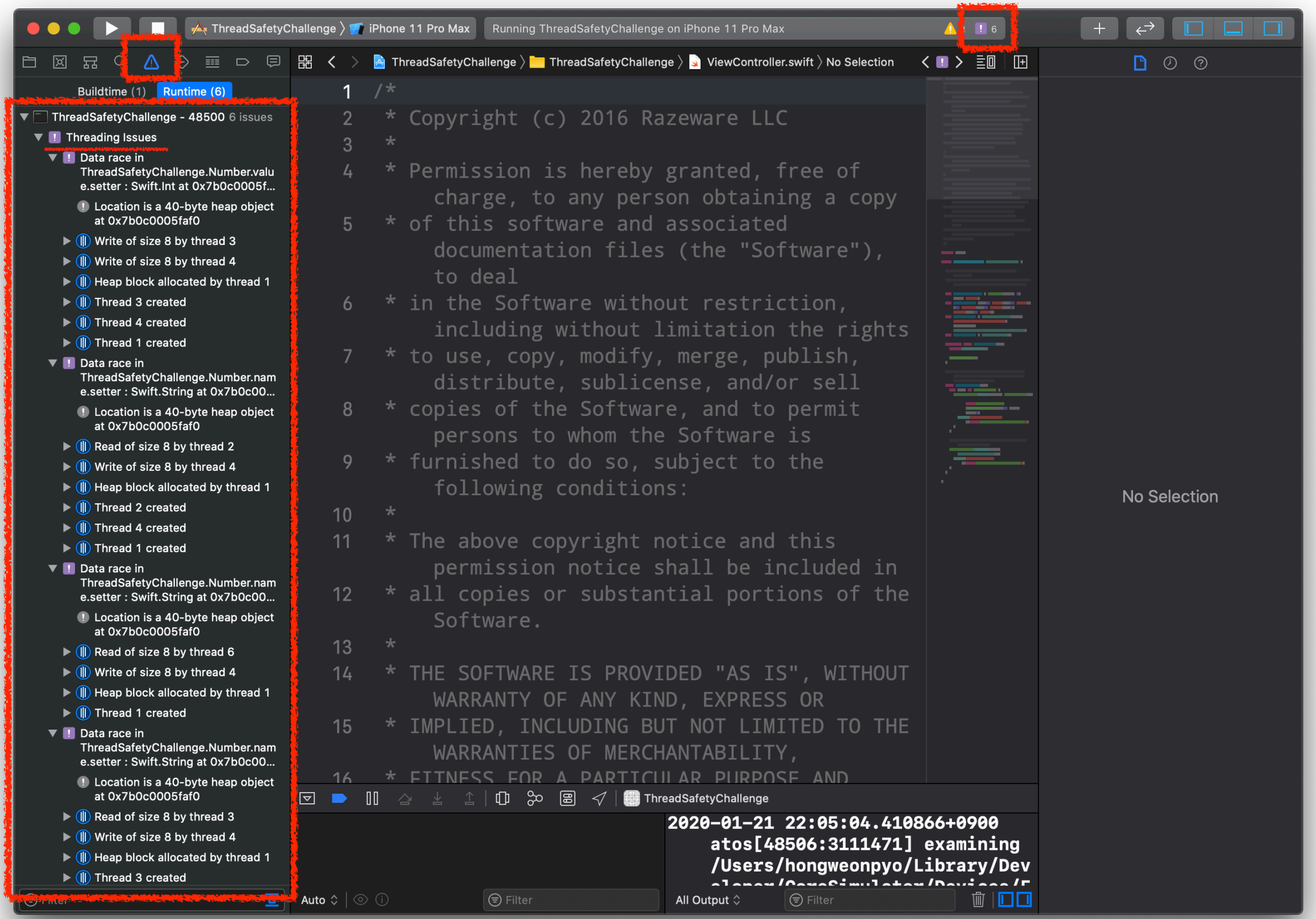
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

editScheme

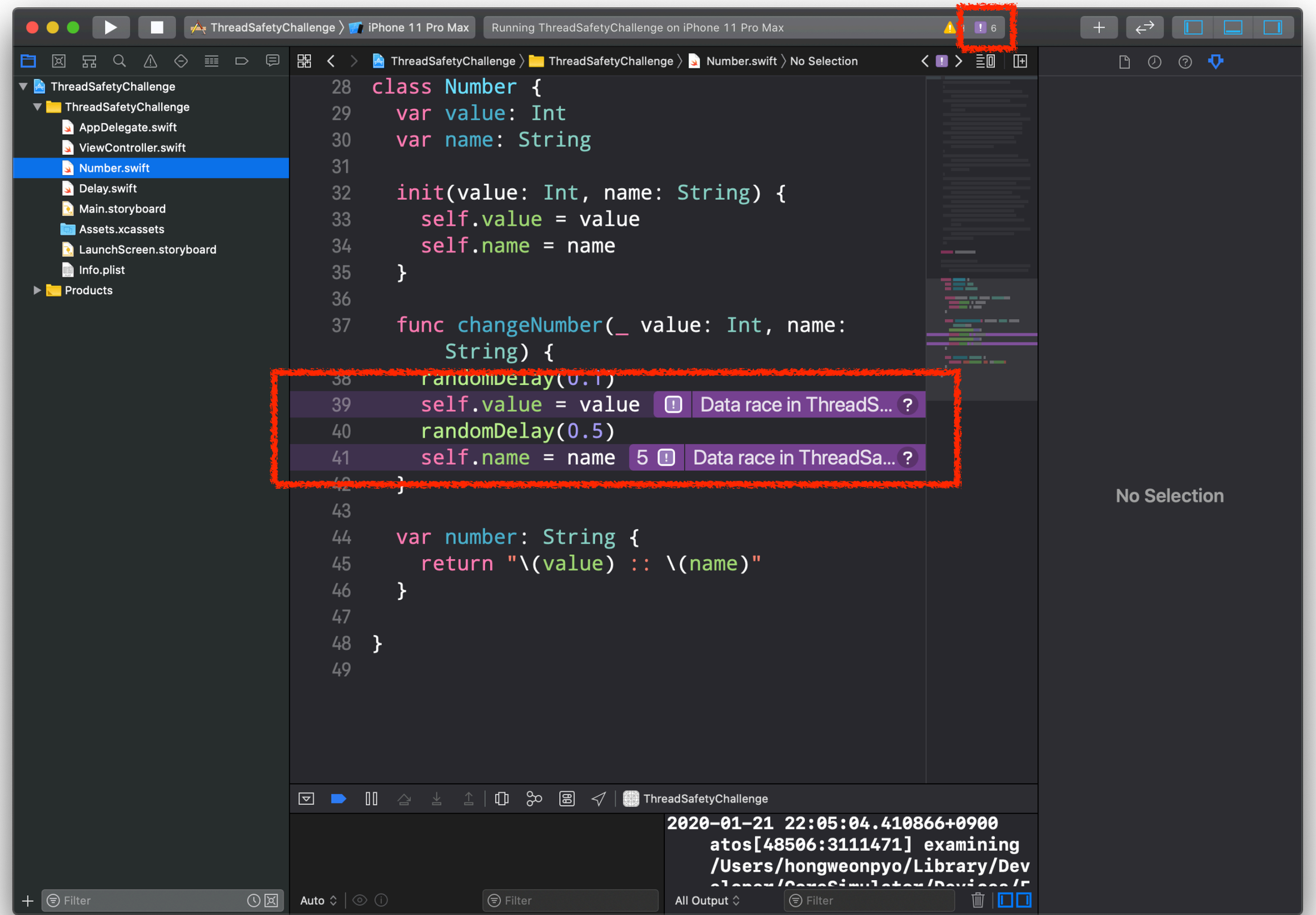


(다 사용하고 나면, 체크박스 해제하면 됨)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

TSan 사용법 확인 - 4.TSanExample

2) 시리얼큐와 sync

코드예시

동시큐 등에서 접근할 가능성이 있는 객체 설계시

```
private let serialQueue = DispatchQueue(label: "..." )
private var _count = 0
public var count: Int {
    get {
        return serialQueue.sync {
            _count
        }
    }
    set {
        serialQueue.sync {
            _count = newValue
        }
    }

}
```

시리얼큐에 넣으면 되었지, 왜 또 async가 아닌 sync일까?

시리얼큐 / sync

뷰 컨트롤러

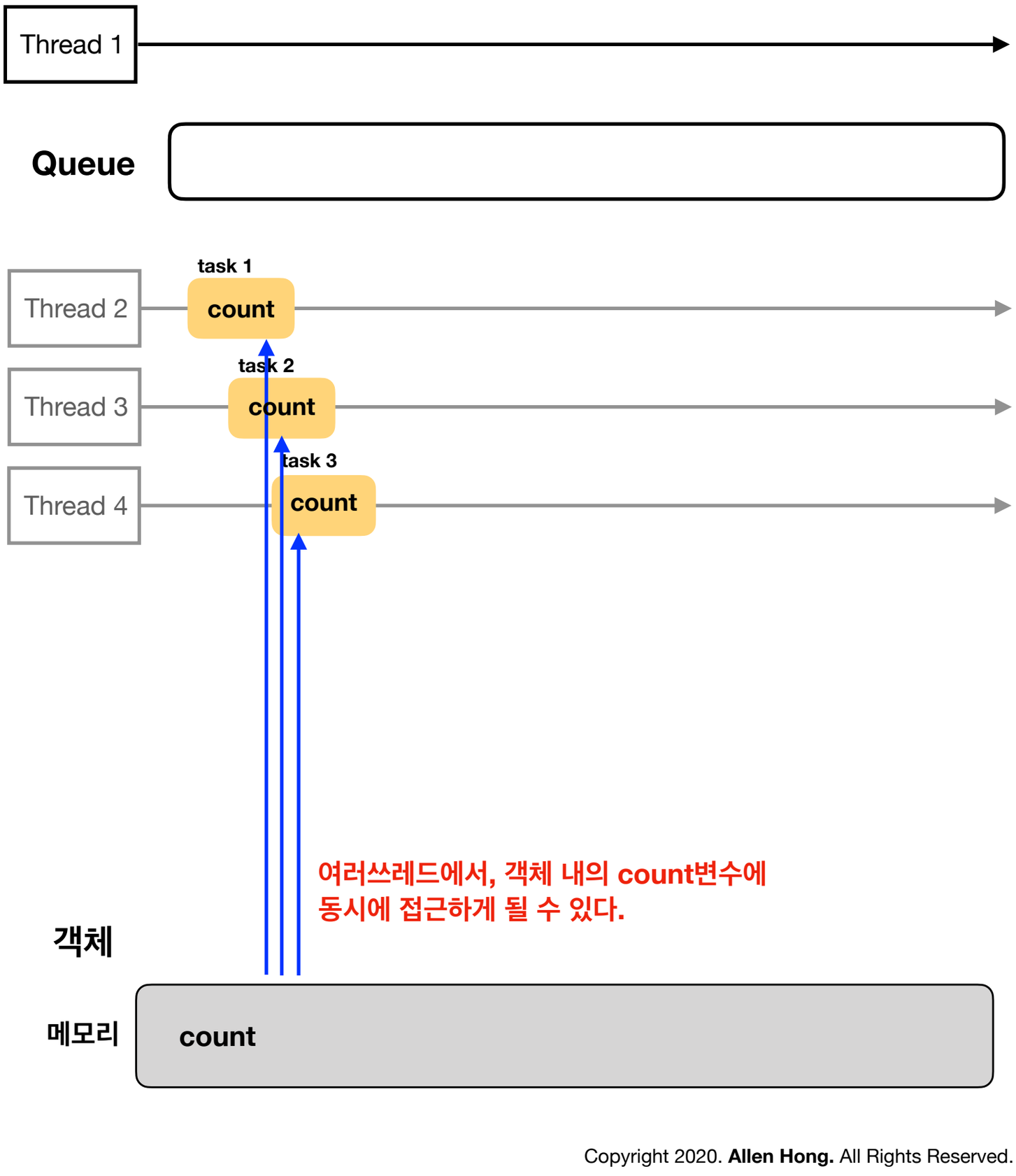
```
DispatchQueue.global( ).async {




}
```

객체

```
var count: Int {
    get {
        _count
    }
    set {
        _count = newValue
    }
}
```

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

시리얼큐 / sync



엄밀하게 말하면, 읽는 것은 동시에 해도
괜찮을 수 있으나, 읽기/쓰기가 동시에 이루어 진다던지
쓰기작업이 여러개가 동시에 이루어지는 것은
Thread-safe하지 않은 상황임

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

(어떤 객체에) 여러 쓰레드에서 동시다발적인 접속이 일어날 가능성이 있음

뷰 컨트롤러

```
DispatchQueue.global( ).async {

}
```

객체

```
var count: Int {
    get {
        _count
    }
    set {
        _count = newValue
    }
}
```

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

시리얼큐 / sync

뷰 컨트롤러

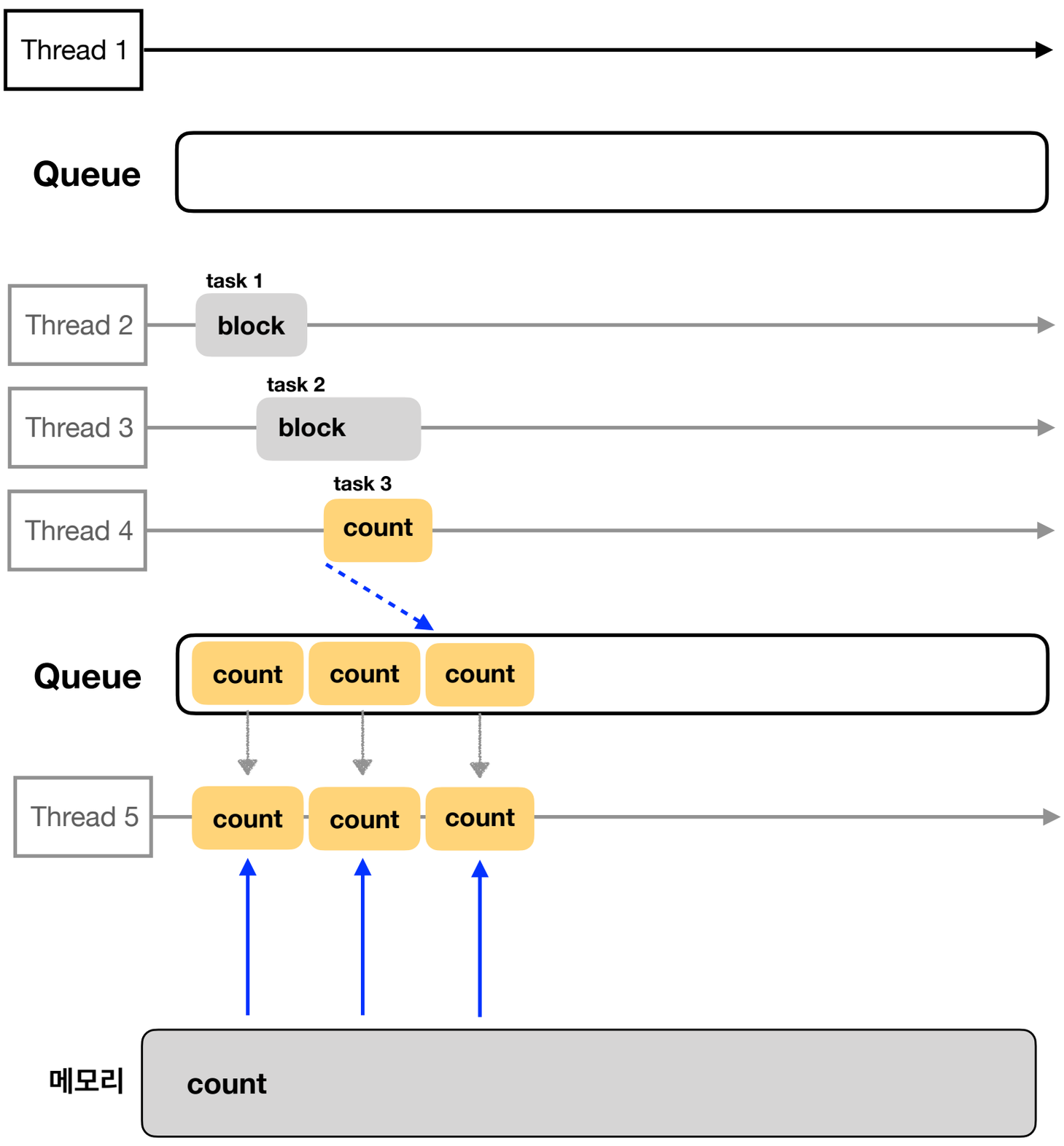
```
DispatchQueue.global( ).async {

}
```

객체

```
var count: Int {
    get {
        return serialQueue.sync {
            _count
        }
    }
    set {
        serialQueue.sync {
            _count = newValue
        }
    }
}
```

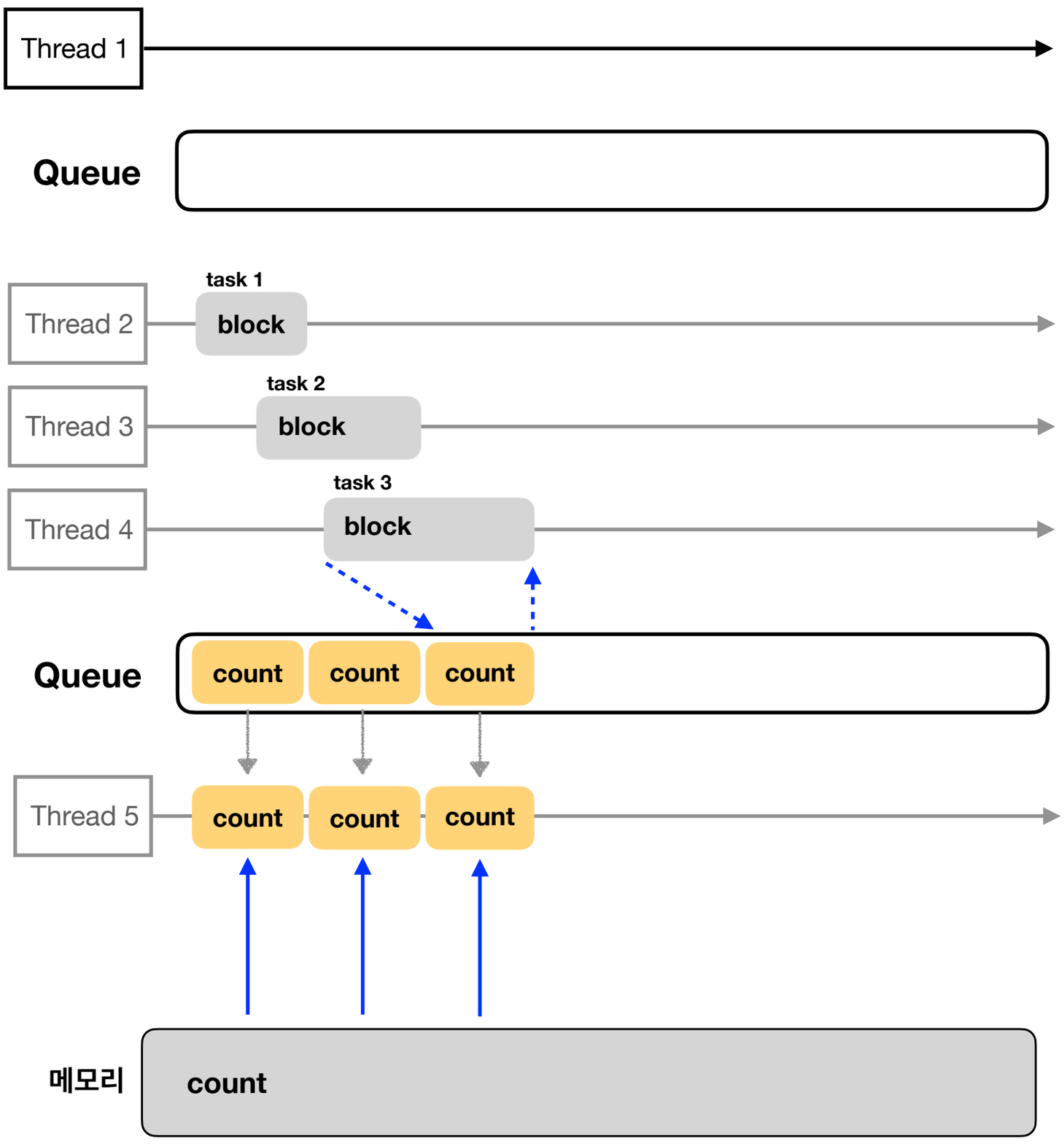
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

시리얼큐 / sync



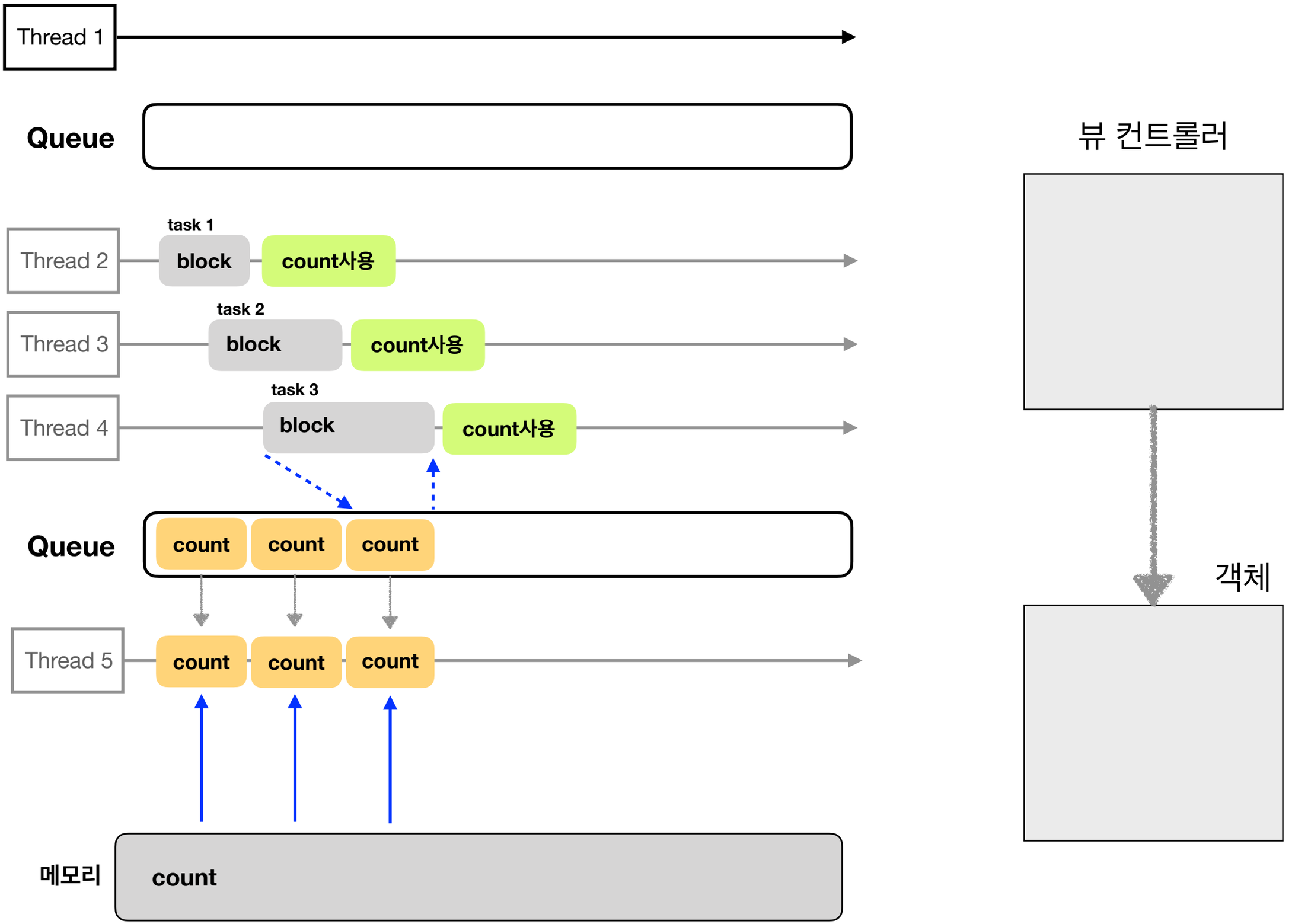
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

시리얼큐 / sync



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

시리얼큐 / sync



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

시리얼큐 / sync

뷰 컨트롤러

```
DispatchQueue.global( ).async {

    serialQueue.sync {
        _count
    }
}
```

객체

```
var count: Int {
    get {
        return serialQueue.sync {
            _count
        }
    }
    set {
        serialQueue.sync {
            _count = newValue
        }
    }
}
```

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
DispatchQueue.global( ).async {

    serialQueue.sync {
        _count
    }
    _count
}
```

async (X)

async 비동기 함수 내부에서
다시 비동기로 보내면...
제대로된 값을 못 얻은 다음
사용할 수 있다.

sync

즉, 여기서의 sync의 의미는
“글로벌 디폴트큐”에서 “시리얼큐”로 보낸
작업을 기다린다는 의미이다.
(디폴트 글로벌큐에서의 기다림
/ 시리얼큐에서의 기다림이 아님)

sync메서드는, 메인쓰레드가 아닌 곳에서 사용 가능하며 잠재적 경쟁상황을 피하는 데 유용하다.

메인큐에서는 절대 sync메서드를 사용하면 안된다. 그러나, sync메서드는 경쟁 상황을 피하는데는 매우 유용하다. 큐가 시리얼큐이고 객체에 접근하는 유일한 방법이라면, sync 메서드는 상호간의 배제 락(mutual exclusion lock)처럼 **“모든 쓰레드(메인 쓰레드가 아닌)가 일관된 값을 얻는 걸”** 보장하면서 실행한다.

➡ **메인 쓰레드에서 떨어져 비동기적으로 실행되는(예: DispatchQueue.global( ) 등) 작업들은 여러 쓰레드에서 동작하므로, 유일한 객체에 접근 할때는 시리얼큐 및 sync메서드를 사용하여 일관된 값을 얻게 하여야 한다.**

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

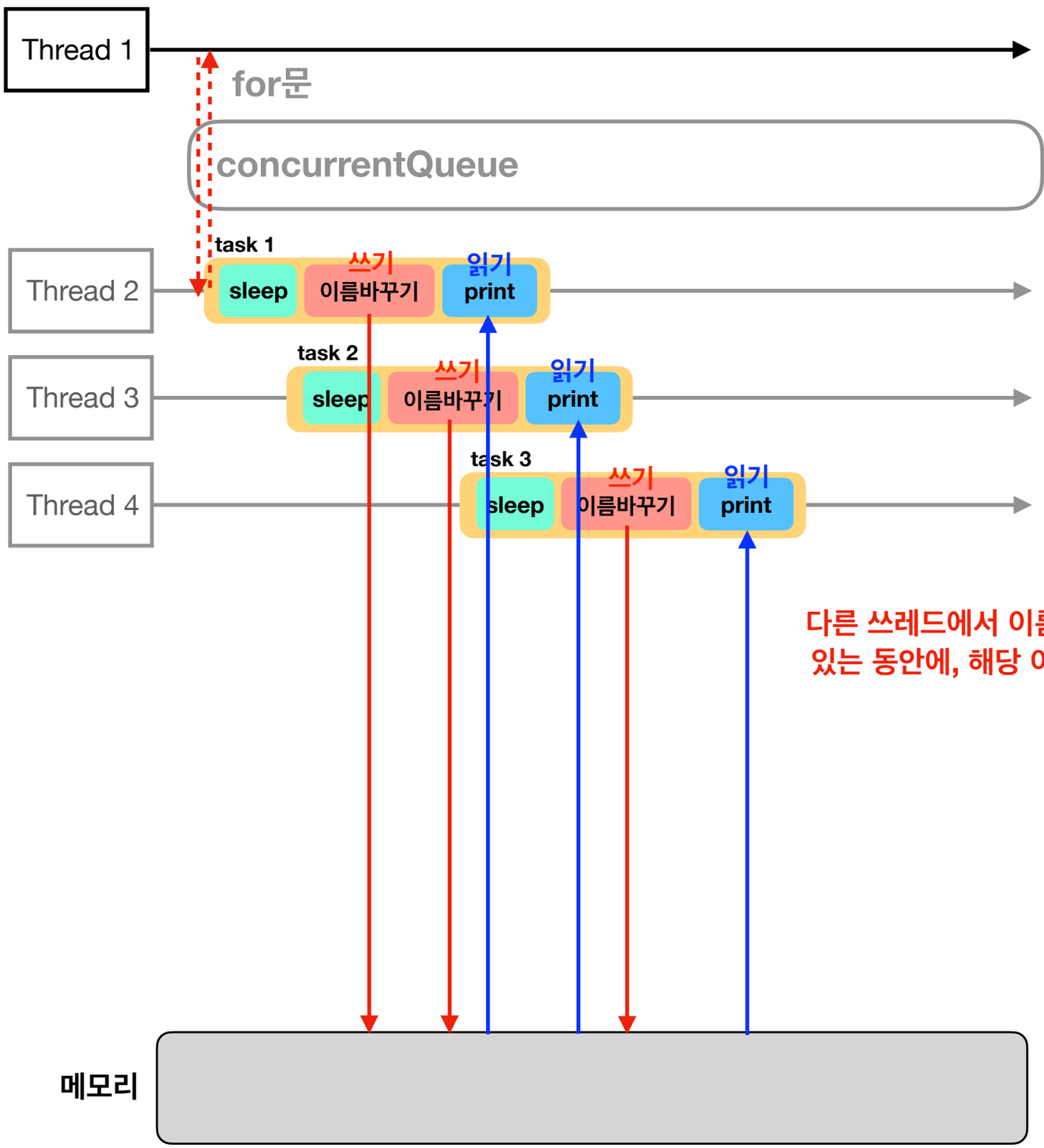
7-2.SerialQueueSync

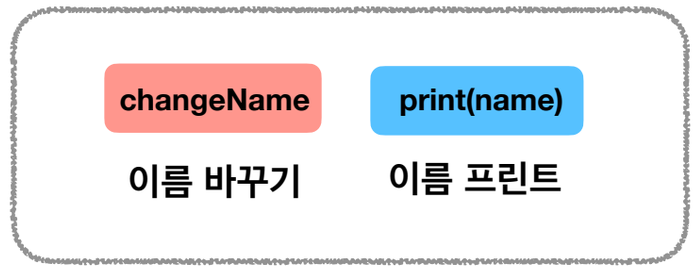
앱 - 폴더 5.SerialSyncProject

Thread-safe처리가 안된,
경쟁 상황의 경우의 그림 예시

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드를 그림으로 표현 6-1.AnotherRaceCondition에서 봤던 예제





다른 쓰레드에서 이름을 바꾸고
있는 동안에, 해당 이름에 접근

적정한 프린트가 이루어지지 않는 이유:
여러쓰레드에서 동시다발적으로
쓰기(changeName) 및 읽기(print)가 발생하고 있음

여러쓰레드가 경쟁적으로 데이터에 접근(Race Condition)

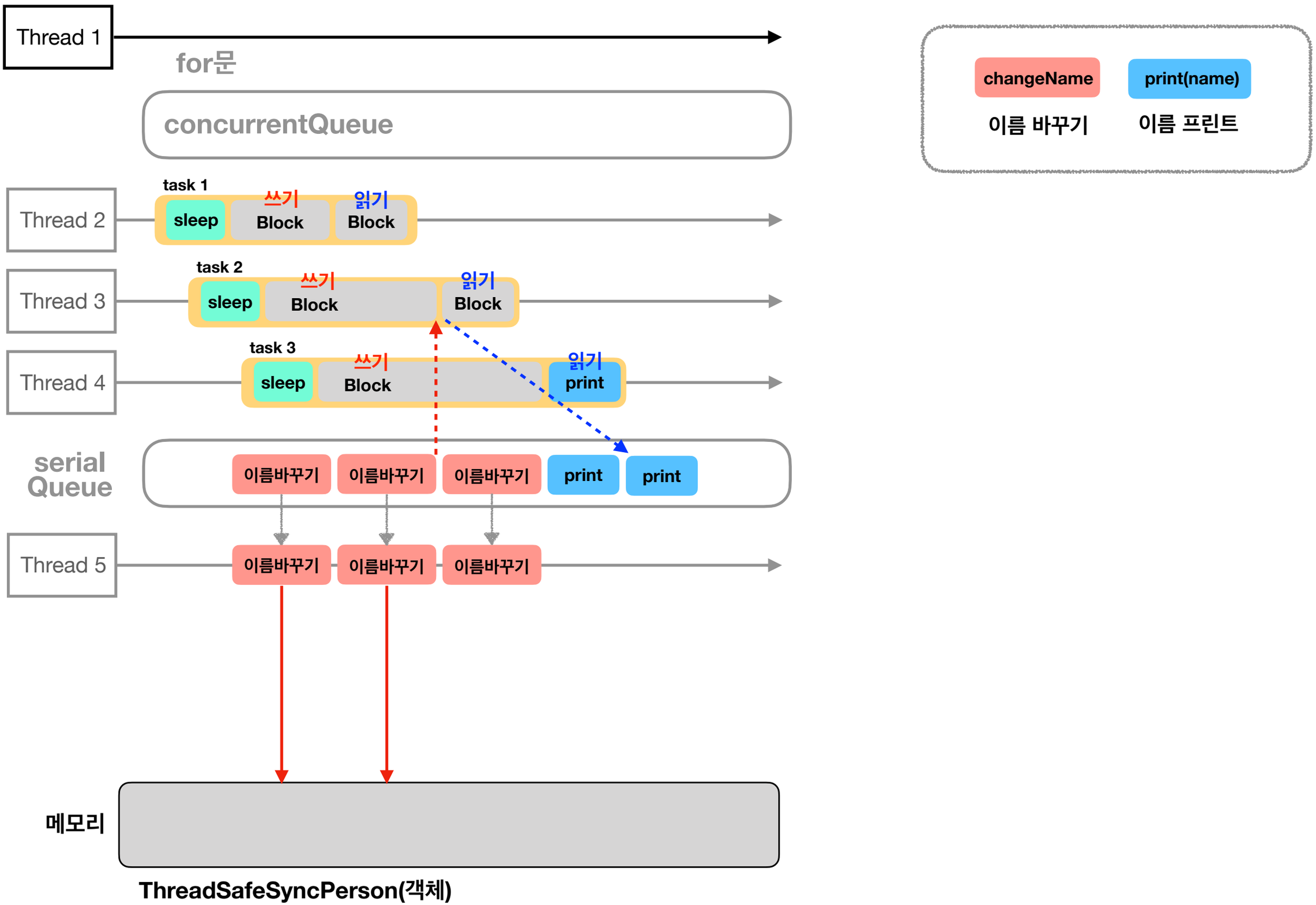
Person(객체) Person 클래스의 내부는 동기적으로 구현된 것처럼 보이지만, 실제 해당 객체에 접근하는 쓰레드가 더 중요

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

엄격한 Thread-safe처리가 제대로된 경우

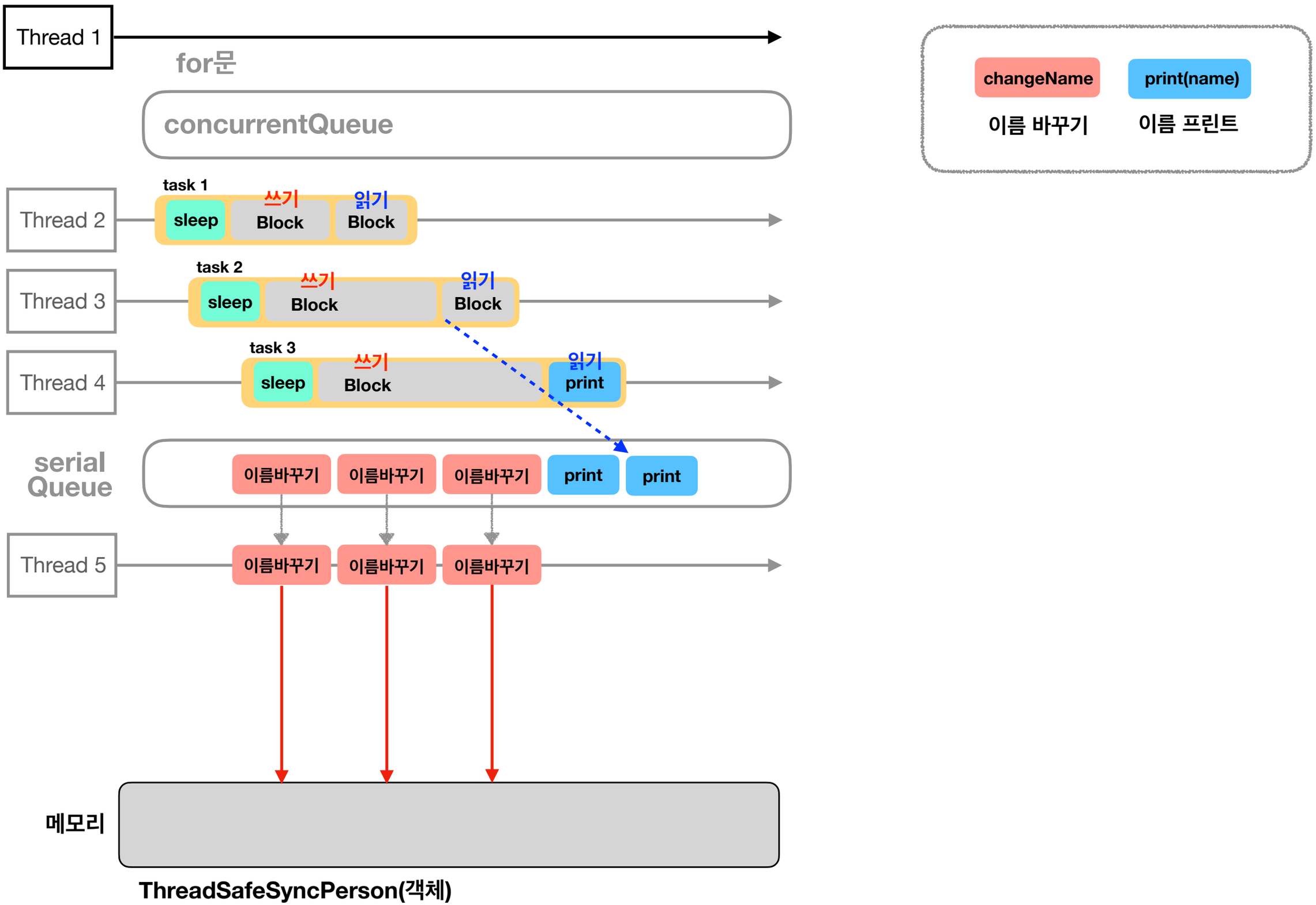
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드를 그림으로 표현 (7-2.SerialQueueSync) 쓰기 및 읽기를 모두 Serial + Sync처리 했을때



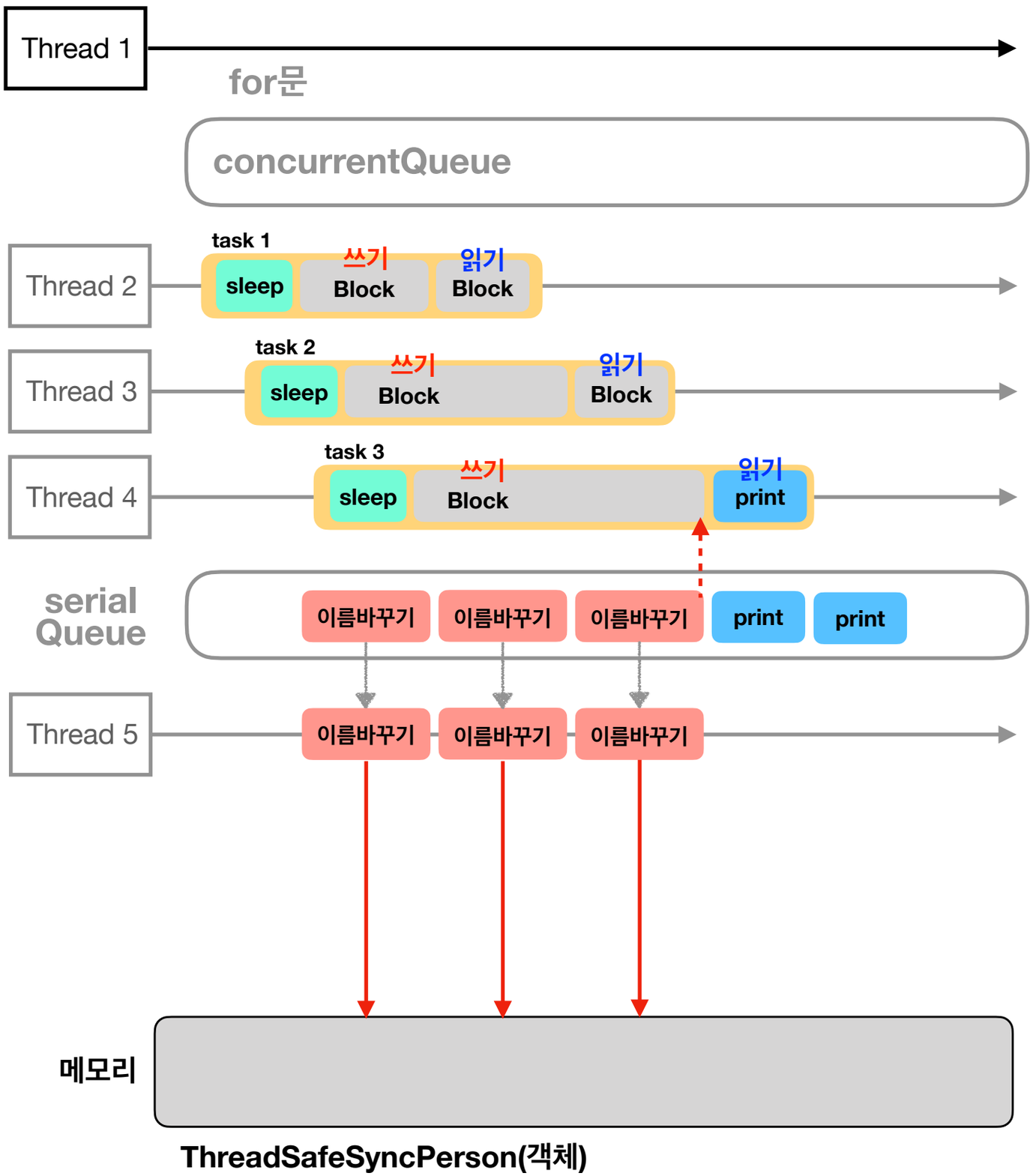
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

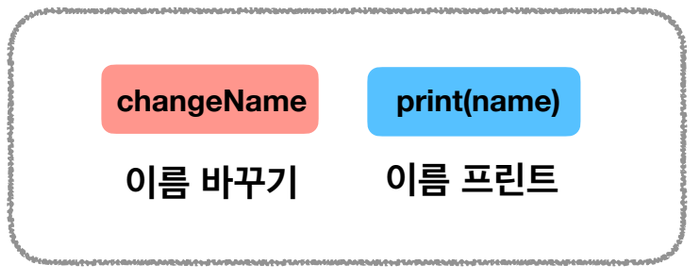
코드를 그림으로 표현 (7-2.SerialQueueSync) 쓰기 및 읽기를 모두 Serial + Sync처리 했을때



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

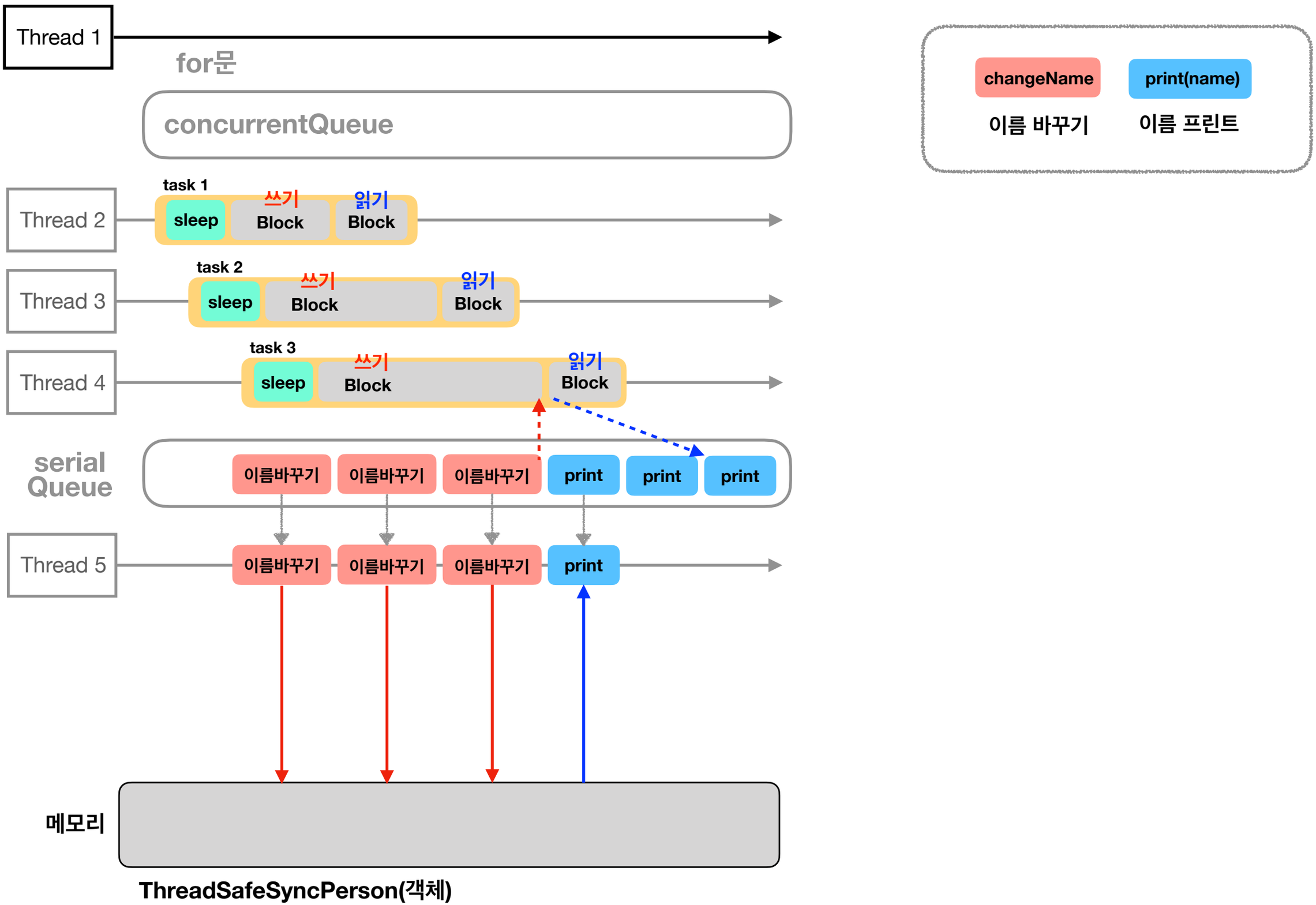
코드를 그림으로 표현 (7-2.SerialQueueSync) 쓰기 및 읽기를 모두 Serial + Sync처리 했을때





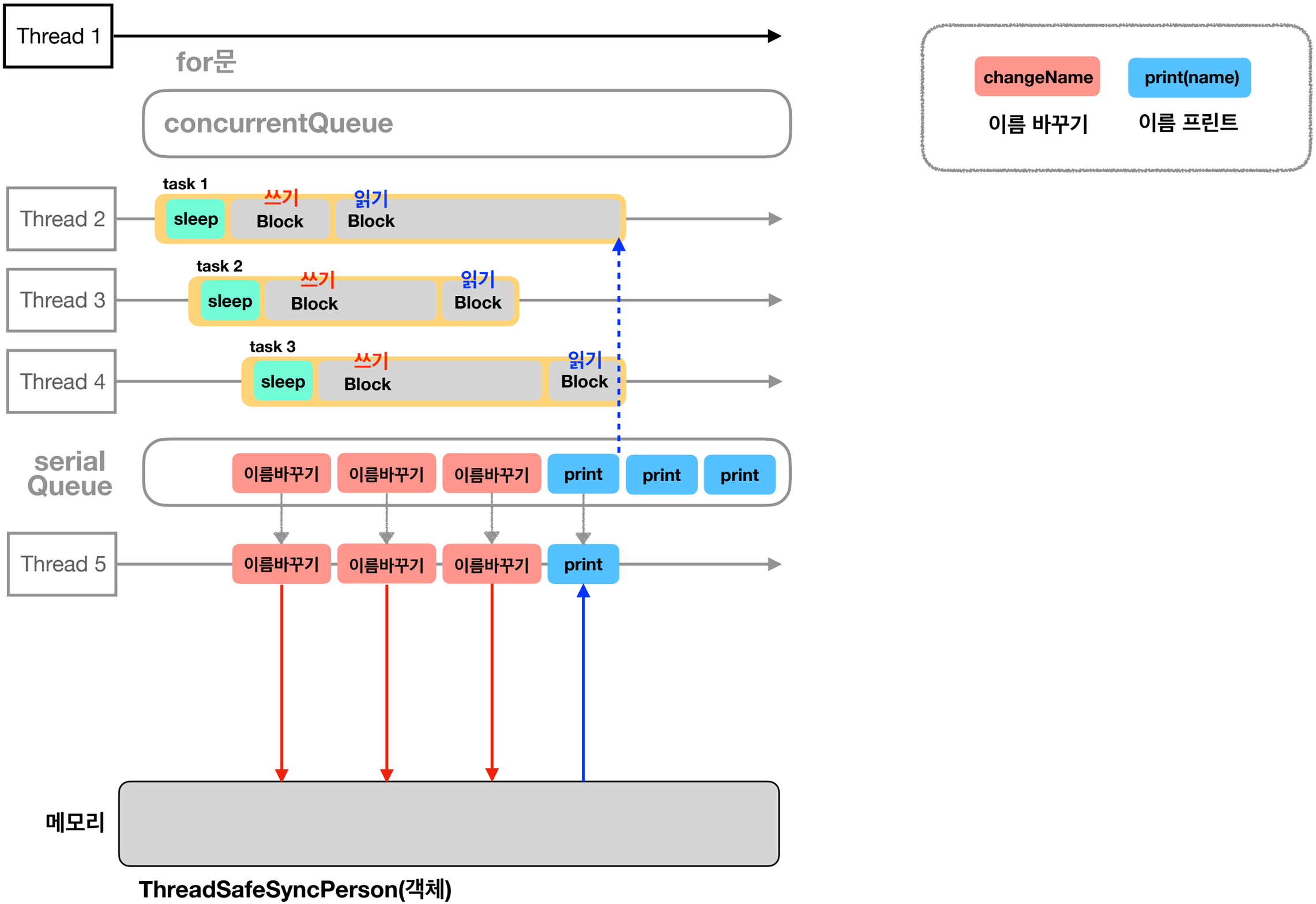
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드를 그림으로 표현 (7-2.SerialQueueSync) 쓰기 및 읽기를 모두 Serial + Sync처리 했을때



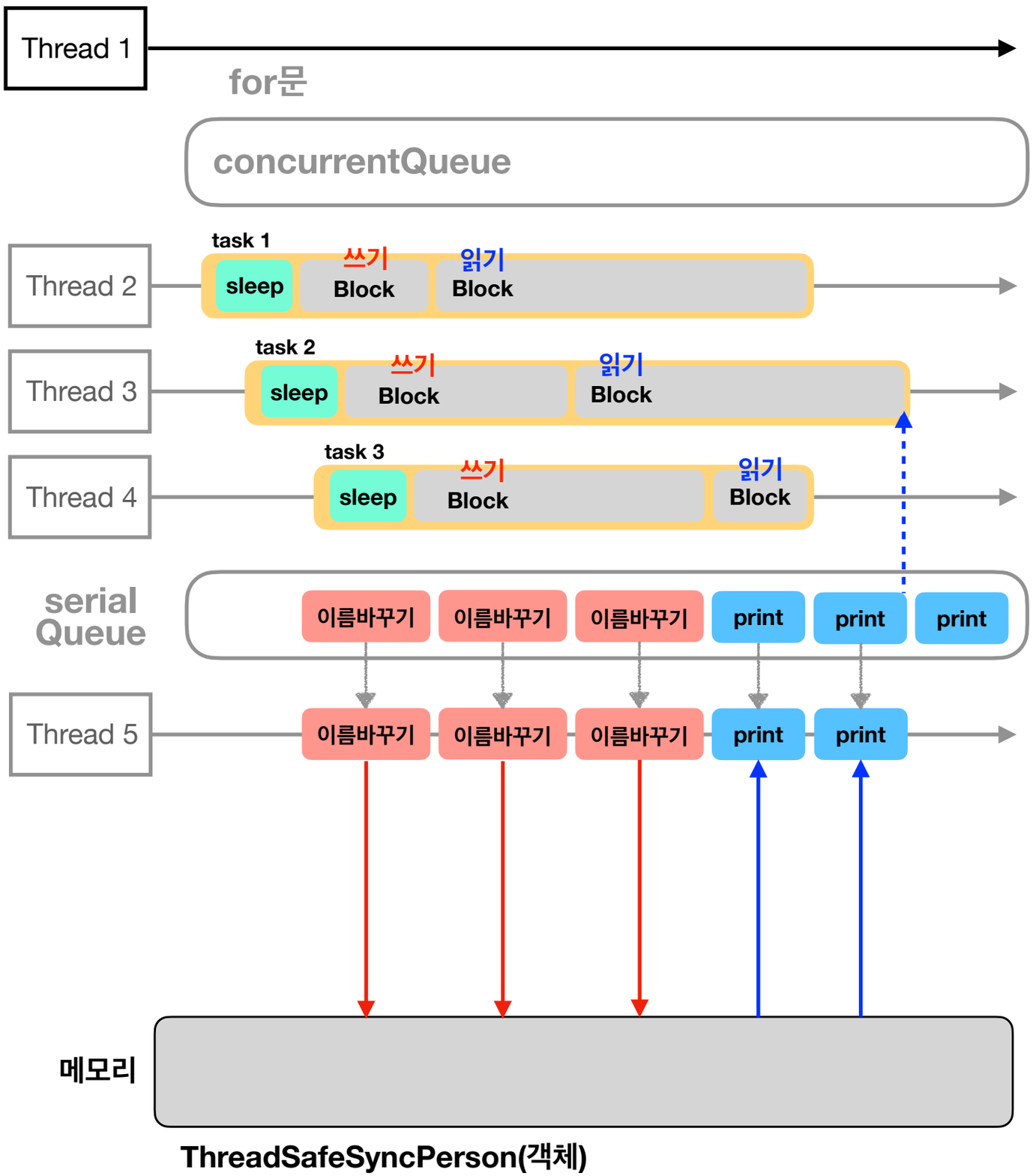
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

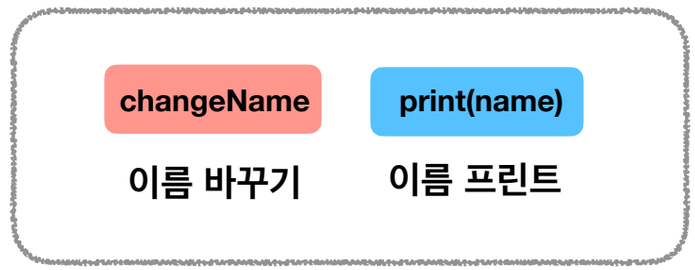
코드를 그림으로 표현 (7-2.SerialQueueSync) 쓰기 및 읽기를 모두 Serial + Sync처리 했을때



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드를 그림으로 표현 (7-2.SerialQueueSync) 쓰기 및 읽기를 모두 Serial + Sync처리 했을때

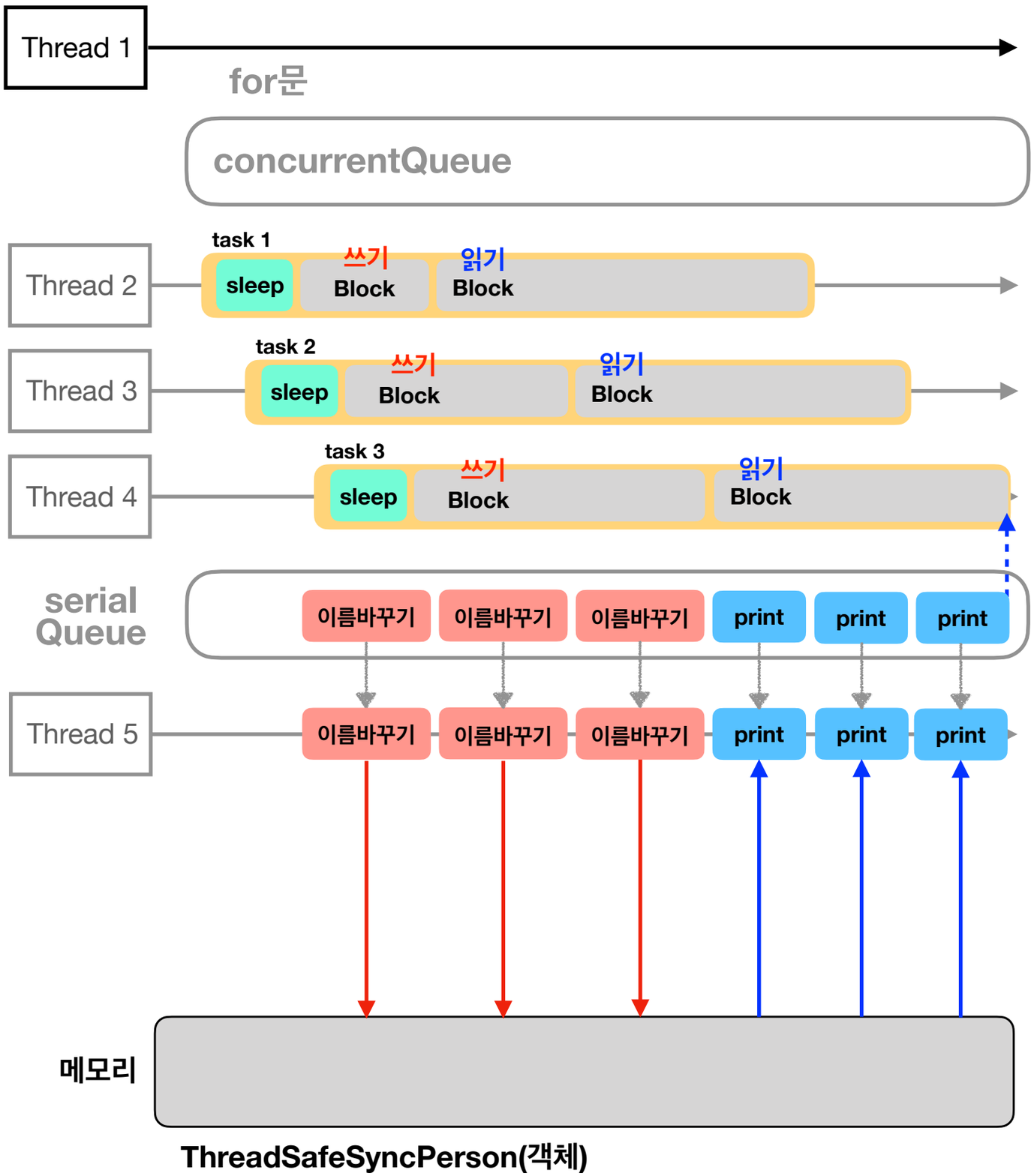


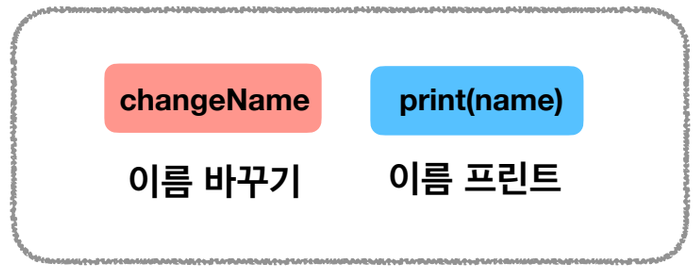


Thread2에서 print작업을 보내기 전에, 이미 다른 쓰레드(Thread3, Thread4)에서 이름을 바꾸는 작업을 시리얼큐에 보내기 때문에 이런 현상이 발생함

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드를 그림으로 표현 (7-2.SerialQueueSync) 쓰기 및 읽기를 모두 Serial + Sync처리 했을때





sync라는 것(“기다린다”)은 시작시점은 달라도 시킨일이 종료될때까지, 즉 종료시점까지 해당 쓰레드를 block한다는 것

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

엄격한 Thread-safe처리가 제대로된 경우

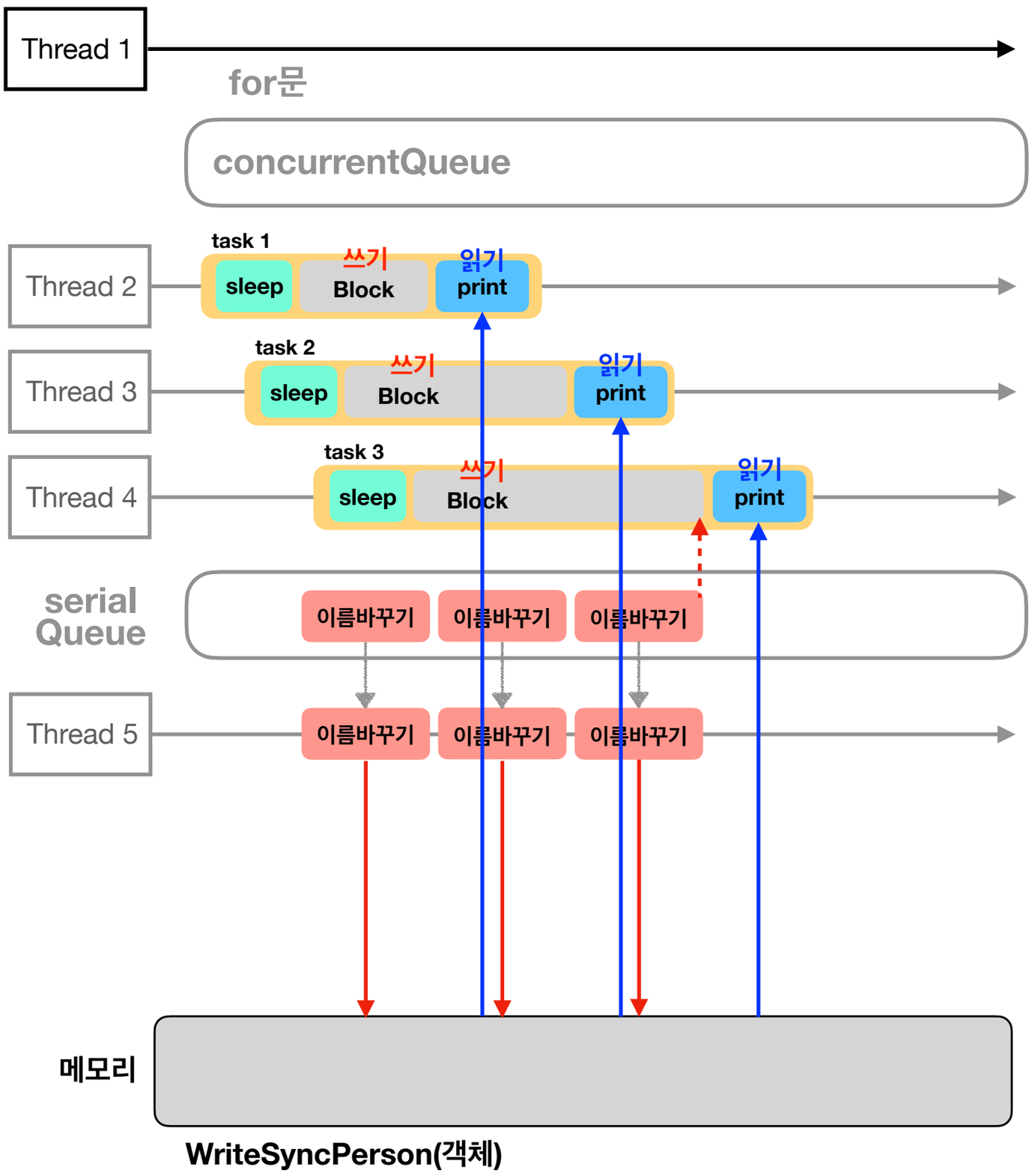
프린트하는 작업과는 별개로 객체에 한번에 한개의 쓰레드만 접근할 수 있으니 Thread-safe하게 처리된 것이 맞음

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

(참고) Thread-safe처리가 제대로되진 않았지만 프린트를 제대로 할 수있게 만들기

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드를 그림으로 표현 (7-2.SerialQueueSync) WriteSyncPerson



changeName — 이름 바꾸기

print(name) — 이름 프린트

<쓰기>

쓰기(write)작업은 **여러쓰레드에서 한꺼번에 접근하는 것을 막을 필요가 있어** 순서가 필요(Serial)하고,

값을 바꾸는 것을 완료한 후, 해당 값을 얻기 위해 기다림(Sync)이 필요함

<읽기> 프린트는 제대로 됨

읽기(print)작업은 **쓰기작업이 끝난 시점에만 바로 실행하면 되기 때문에** 굳이 시리얼(serial)이나 기다림(sync)이 필요하지 않음 **그러나 이경우, Thread-safe 하지 않음(앱에서 확인)**

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

폴더 5.SerialSyncProject

3) 디스패치 배리어(Dispatch Barrier)

```
public func async(group: DispatchGroup? = default,
  qos: DispatchQoS = default,
  flags: DispatchWorkItemFlags = default,
  execute work: @escaping @convention(block) () -> Swift.Void)
```

```
public static let barrier: DispatchWorkItemFlags
```

```
isolationQueue.async(flags: .barrier) {
  // barrier task
}
```

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드예시

```
private let concurrentQueue
                = DispatchQueue(label: “…”, attributes: .concurrent )
private let _count = 0
public var count: Int {
    get {
        return concurrentQueue.sync {
            _count
        }
    }
    set {
        concurrentQueue.async(flags: .barrier) {  [unowned self] in
            self._count = newValue
        }
    }

}
```

Dispatch Barrier

concurrent큐 내의 여러개 쓰레드 중에서 “배리어 작업”의 경우,
한개의 쓰레드만 사용해 serial(직렬)로 실행가능한 방법

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Dispatch Barrier

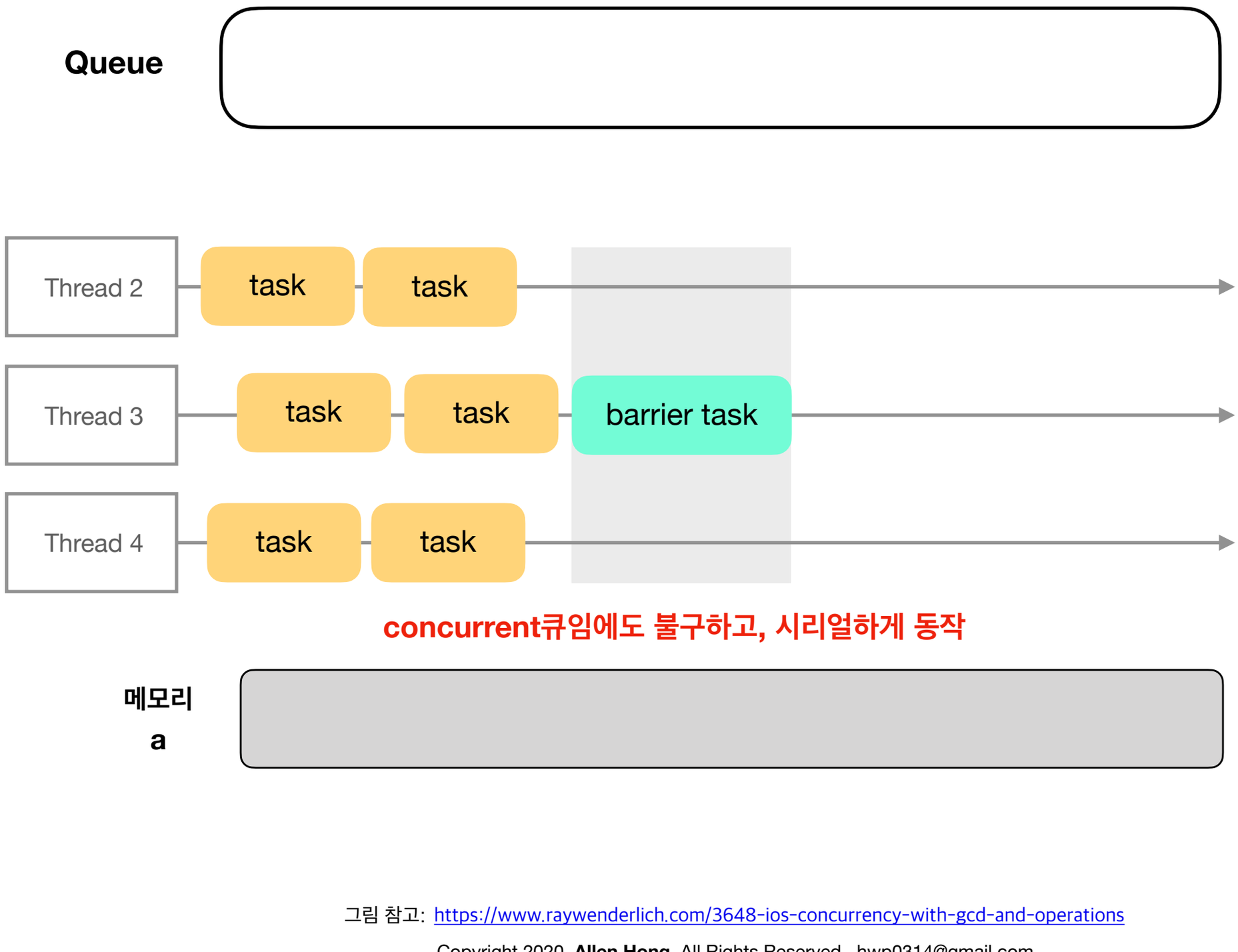


그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Dispatch Barrier

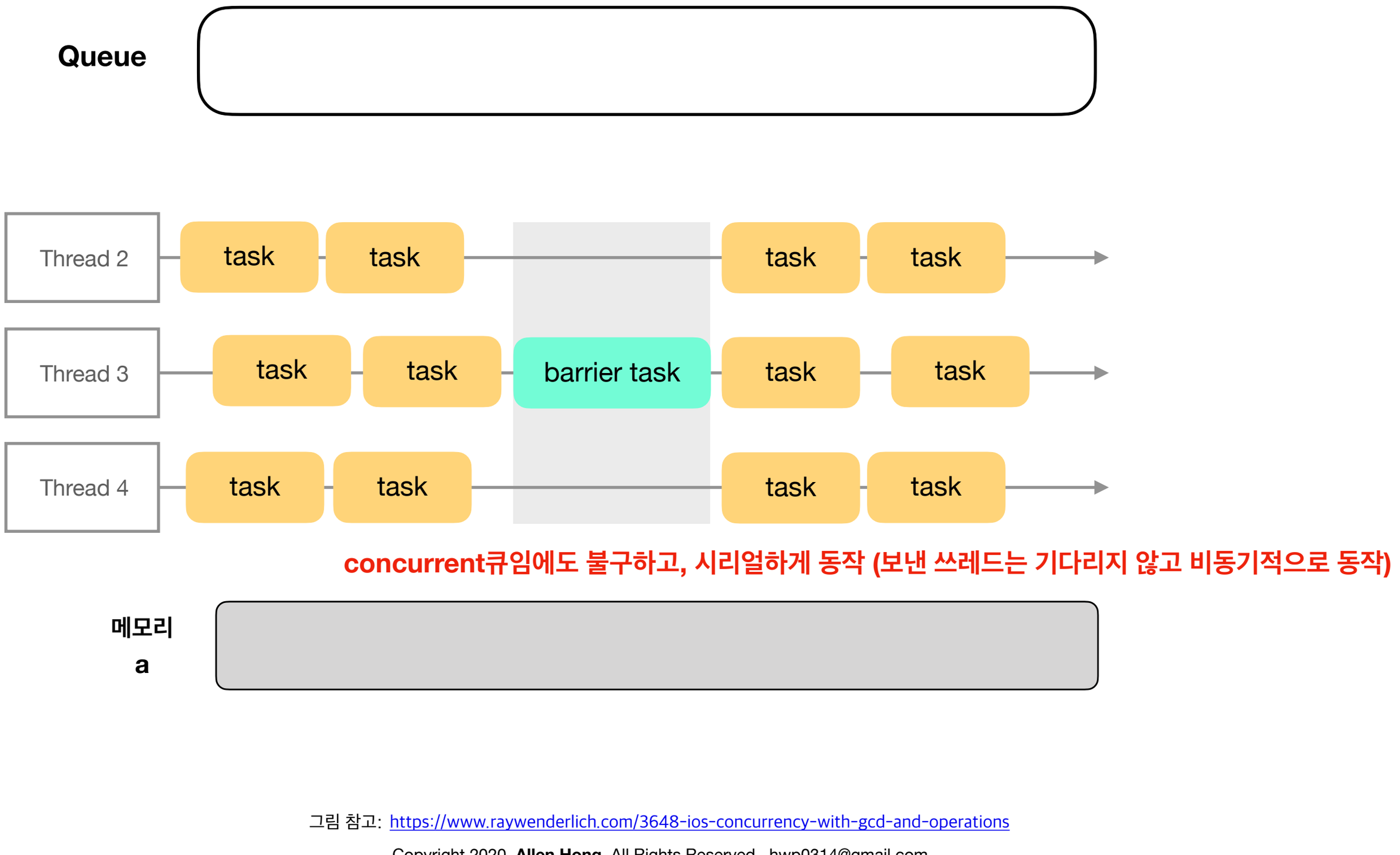


그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Dispatch Barrier

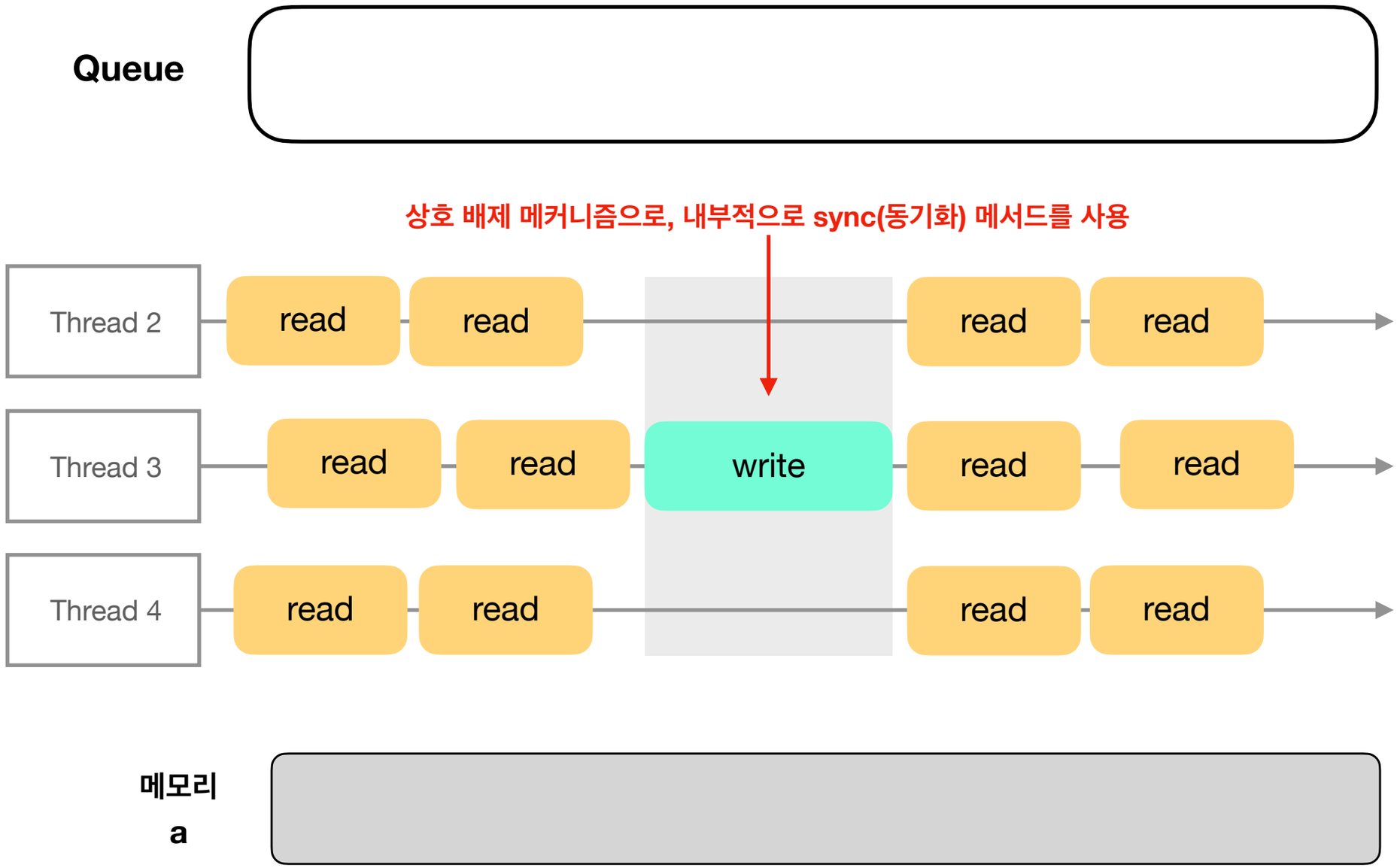
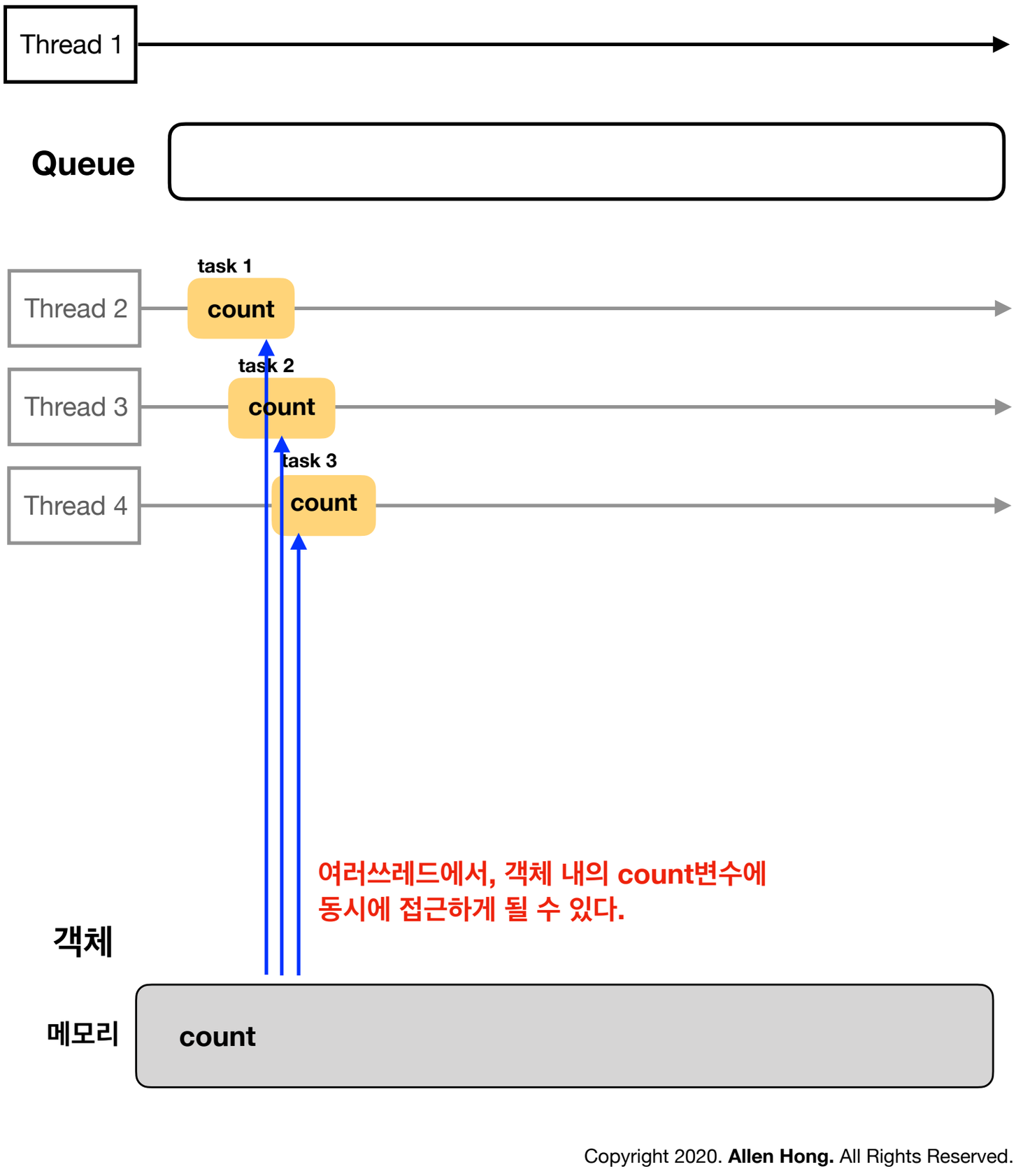


그림 참고: https://www.raywenderlich.com/3648-ios-concurrency-with-gcd-and-operations

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

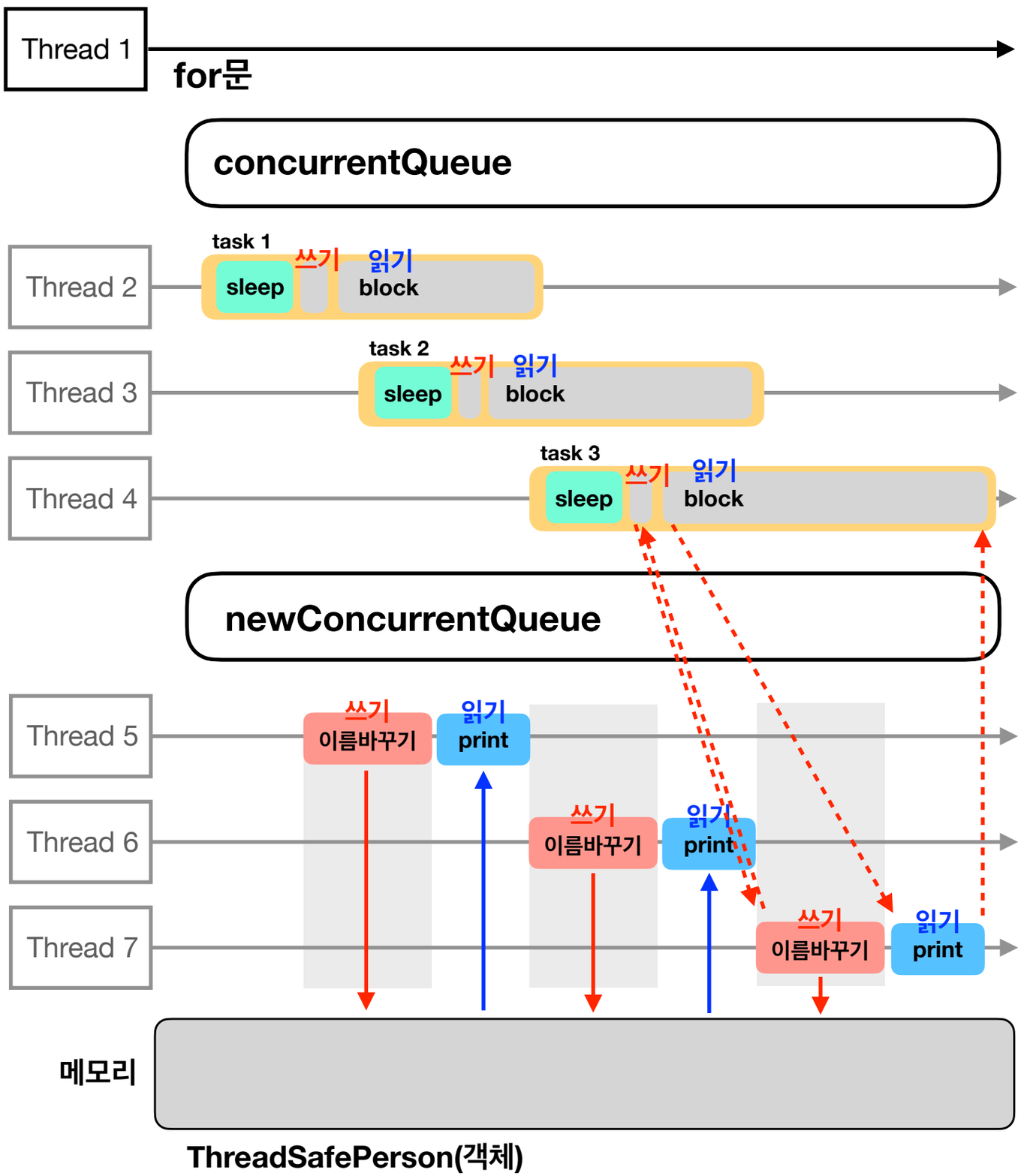
시리얼큐 / sync

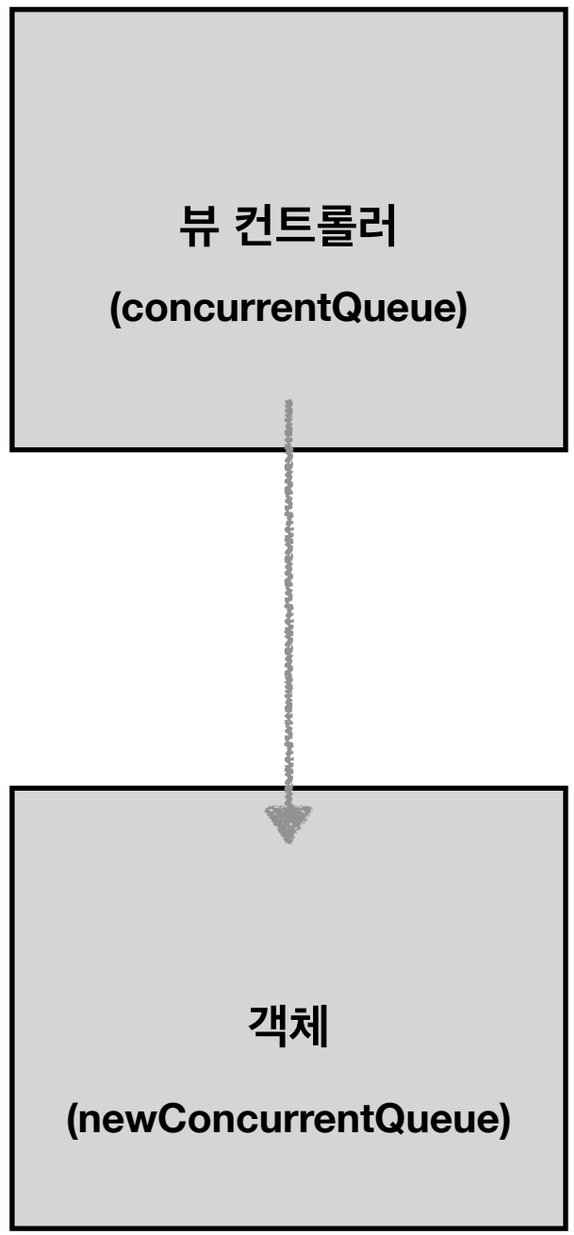


엄밀하게 말하면, 읽는 것은 동시에 해도
괜찮을 수 있으나, 읽기/쓰기가 동시에 이루어 진다던지
쓰기작업이 여러개가 동시에 이루어지는 것은
Thread-safe하지 않은 상황임

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Dispatch Barrier 주로는 메인쓰레드가 아닌 쓰레드에서 다시 접근할때 Barrier의미가 있음





Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

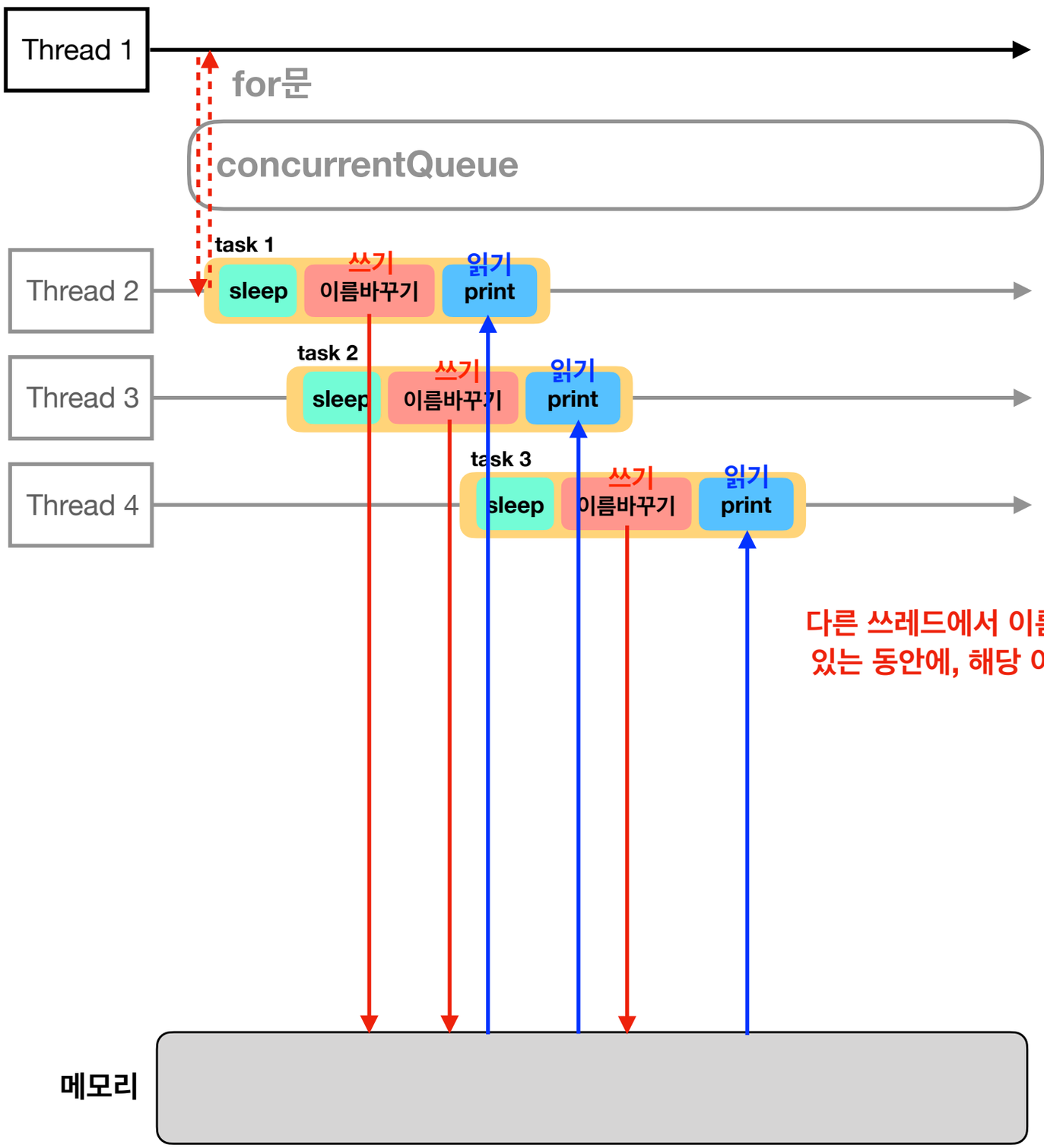
코드예시

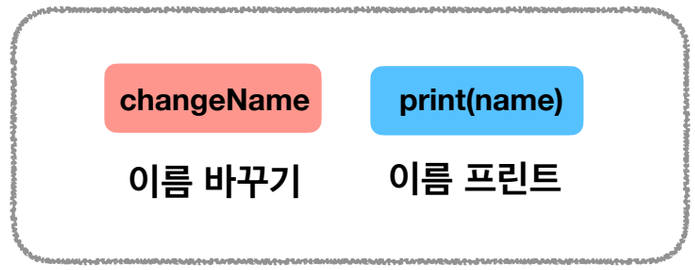
```
private let concurrentQueue
                = DispatchQueue(label: “…”, attributes: .concurrent )
private let _count = 0
public var count: Int {
    get {
        return concurrentQueue.sync {
            _count
        }
    }
    set {
        concurrentQueue.async(flags: .barrier) {  [unowned self] in
            self._count = newValue
        }
    }

}
```

실제 코드예시

7-3.DispatchBarrier

코드를 그림으로 표현 6-1.AnotherRaceCondition에서 봤던 예제





다른 쓰레드에서 이름을 바꾸고
있는 동안에, 해당 이름에 접근

적정한 프린트가 이루어지지 않는 이유:
여러쓰레드에서 동시다발적으로
쓰기(changeName) 및 읽기(print)가 발생하고 있음

여러쓰레드가 경쟁적으로 데이터에 접근(Race Condition)

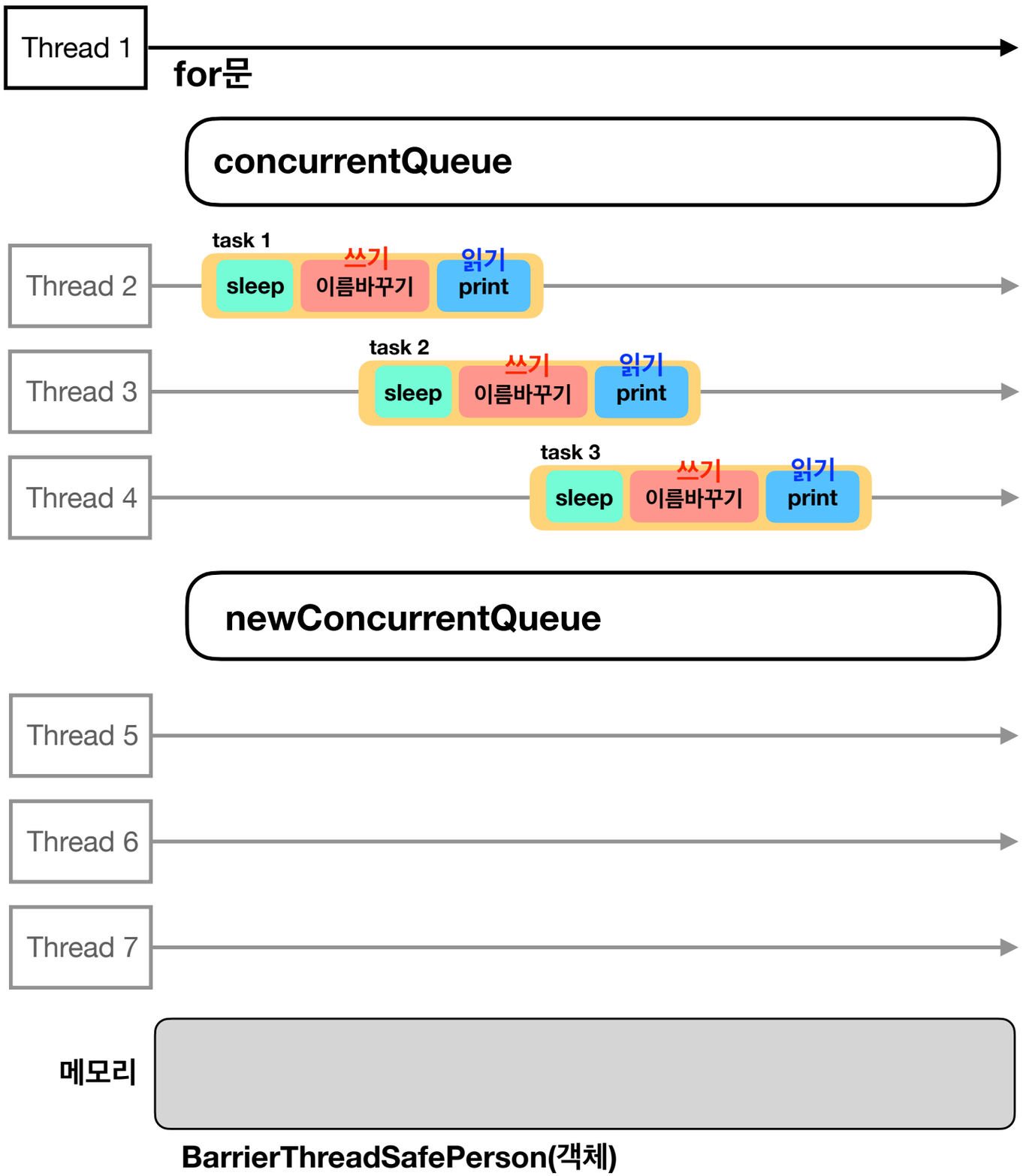
Person(객체) Person 클래스의 내부는 동기적으로 구현된 것처럼 보이지만, 실제 해당 객체에 접근하는 쓰레드가 더 중요

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

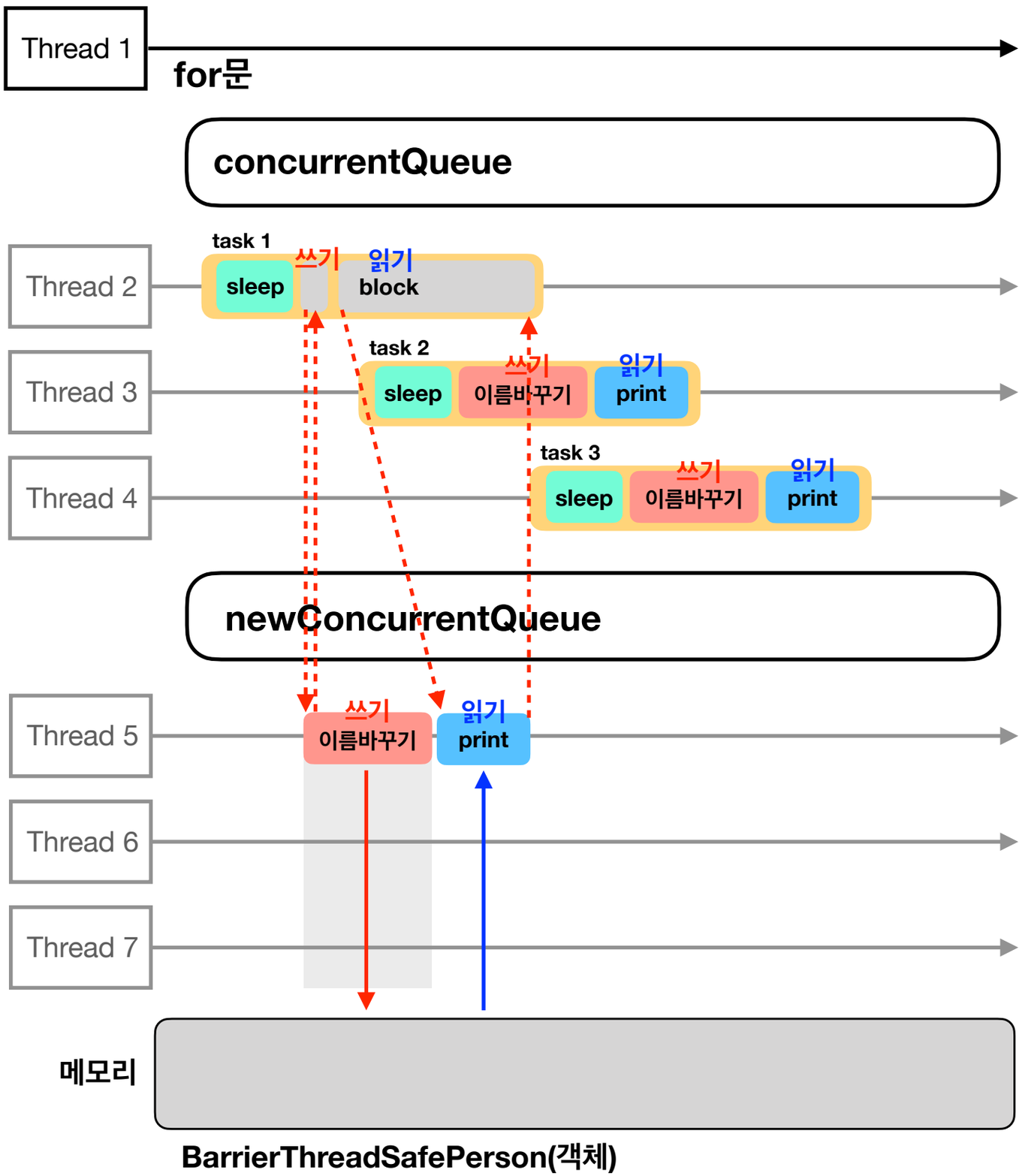
7-3.DispatchBarrier

코드를 그림으로 표현 (7-3.DispatchBarrier) BarrierThreadSafePerson객체



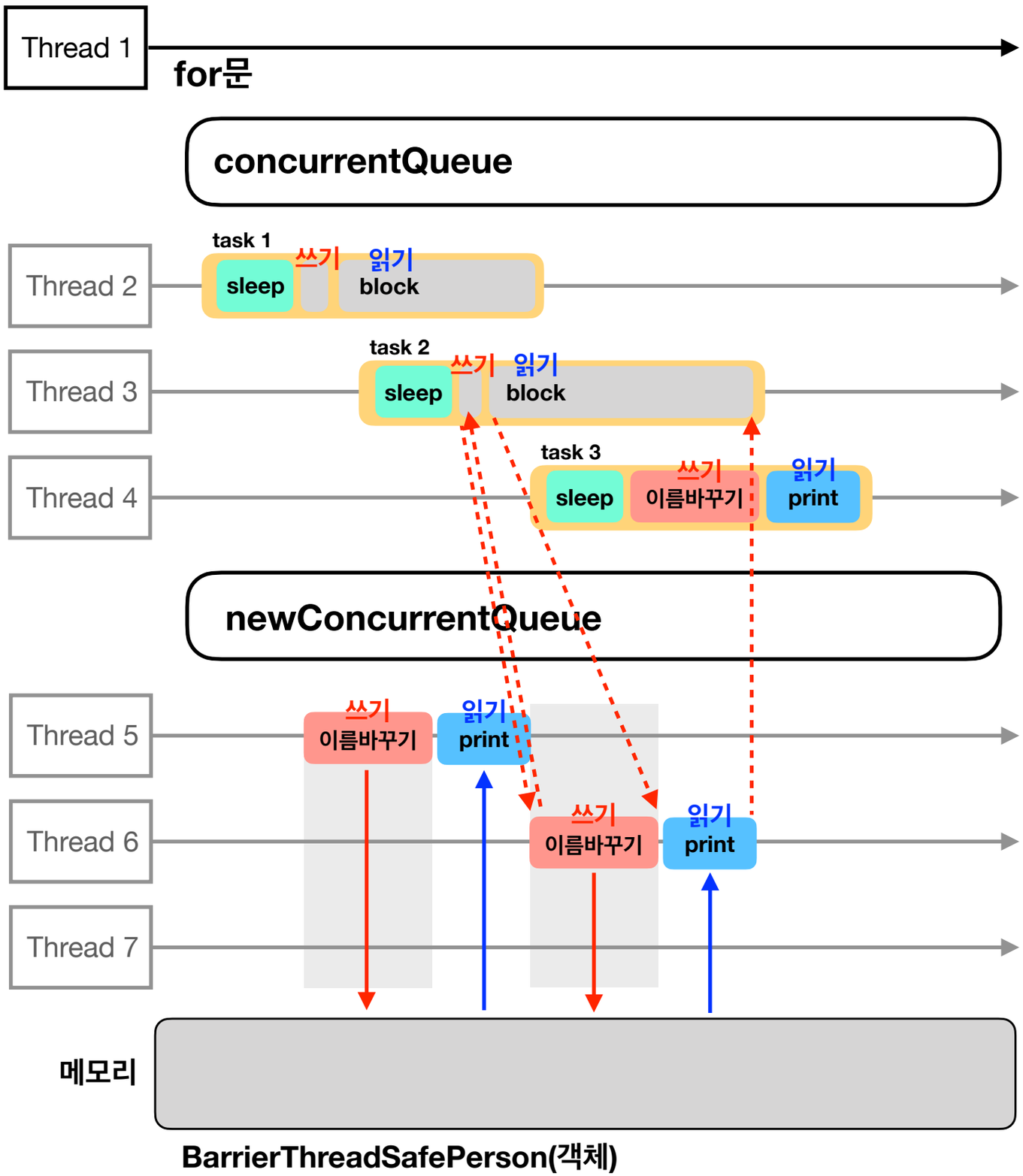
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드를 그림으로 표현 (7-3.DispatchBarrier) BarrierThreadSafePerson객체



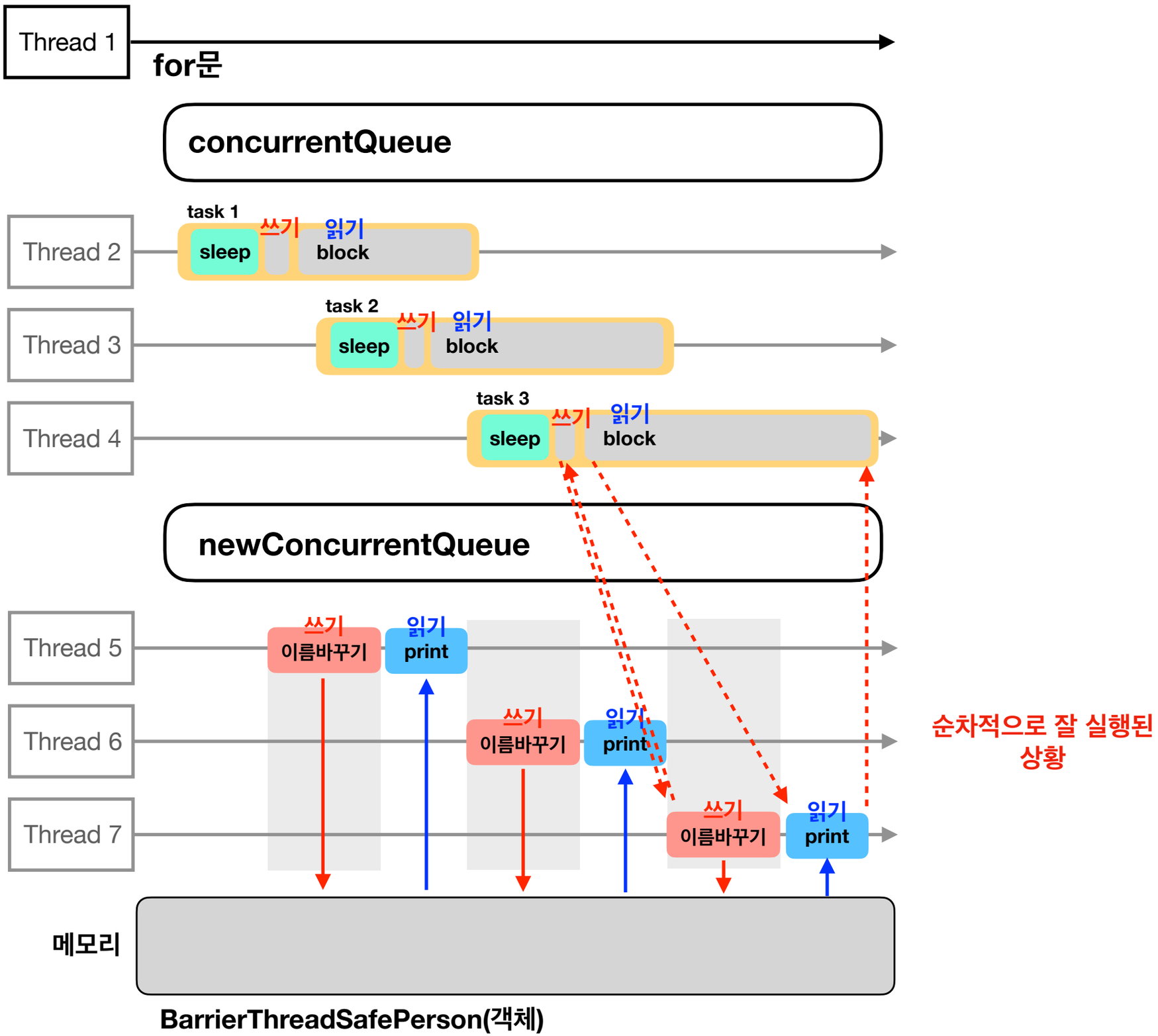
Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드를 그림으로 표현 (7-3.DispatchBarrier) BarrierThreadSafePerson객체



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

코드를 그림으로 표현 (7-3.DispatchBarrier) BarrierThreadSafePerson객체

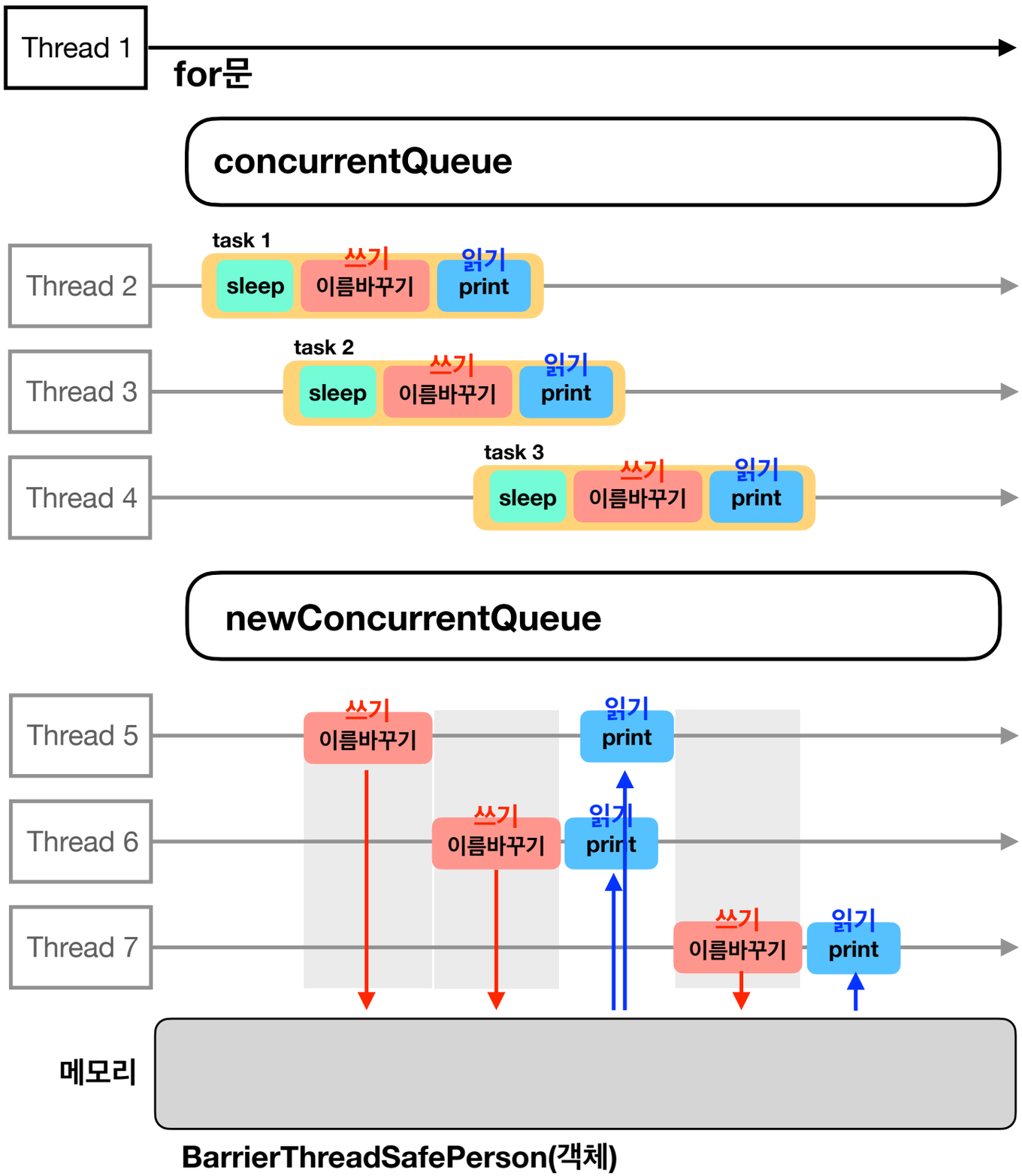


Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

7-3.DispatchBarrier

코드를 그림으로 표현 (7-3.DispatchBarrier) BarrierThreadSafePerson객체 - 그러나 프린트가 잘 동작하지는 않는 경우 예시



작업을 보낼때,
이름바꾸기(쓰기)작업과 print(읽기) 작업의
약간의 시간차로 인해 벌어질 수 있는 상황

(**print(읽기)작업**까지 엄격한 thread-safe처리가 되
어있는 것은 아님. 읽기 작업을 보내기 직전
"쓰기"작업을 약간 먼저 보내면 일어날 수 있는 상황)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

7-3.DispatchBarrier

실제 코드예시

폴더 6.ThreadSafety

Concurrency Problem

구 분	정 의	문제의 해결
(1) 경 쟁 상 황 (Race Condition)	2개이상의 쓰레드에서 공유된 데이터(변수나 객체)에 동시적으로 접근하는 경우	확인방법: TSan (editScheme -> Run -> Diagnostics -> Thread Sanitizer) **(1) 시리얼큐 + Sync (엄격한 Thread-safe)** **(2) 디스패치 배리어 작업 (조금더 효율적인 방법)** (추가적으로, Semaphore이용 동시실행자원도 제한가능) **다만, 객체 등 설계시에 주의해서 사용해야함**
(2) 교 착 상 태 (DeadLock)	한정된 자원을 여러 쓰레드에서 사용하려고 할때, 자원을 얻지 못해 다음 처리를 위한 진행을 못하고 있는 상태	**(1)** (단순한 처리방법으로) **시리얼큐로 해결 가능** **(2) 세마포어 등 제한된 리소스 순서 조심히 사용/ 객체 등 설계시 주의**
(3) 우선순위 뒤바뀜 (Priority Inversion)	다양한 경우에서, 작업의 Qos(서비스 품질)이 뒤바뀌어서 작업이 진행되는 경우	**(1) 1차적으로, GCD가 자원을 점유하고 있는 낮은 우선순위 작업의 우선순위를 높여서 처리.** **(2) 공유된 자원에 대한 접근시 동일한 Qos를 사용해야함.**

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

4) 객체 설계 시 주의할 점

객체에 접근할때 메인큐(쓰레드)가 아닌 다른 큐에서 접근할 가능성이 있는가?

뷰 컨트롤러

```
DispatchQueue.global( ).async {

}
```

★ 객체

```
var count: Int {
    get {
        _count
    }
    set {
        _count = newValue
    }
}
```

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

객체에 접근할때 메인큐(쓰레드)가 아닌 다른 큐에서 접근할 가능성이 있는가?

뷰 컨트롤러

```
DispatchQueue.global( ).async {

    return serialQueue.sync {
        _count
    }
}
```

★ 객체

```
var count: Int {
    get {
        return serialQueue.sync {
            _count
        }
    }
    set {
        serialQueue.sync {
            _count = newValue
        }
    }
}
```

객체 내부에 Thread-safe처리를 해야한다.

(1) 시리얼큐 + sync
(2) 배리어처리

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

Concurrency Problem

구 분	정 의	문제의 해결
(1) 경 쟁 상 황 (Race Condition)	2개이상의 쓰레드에서 공유된 데이터(변수나 객체)에 동시적으로 접근하는 경우	확인방법: TSan (editScheme -> Run -> Diagnostics -> Thread Sanitizer) **(1) 시리얼큐 + Sync (엄격한 Thread-safe)** **(2) 디스패치 배리어 작업 (조금더 효율적인 방법)** (추가적으로, Semaphore이용 동시실행자원도 제한가능) **다만, 객체 등 설계시에 주의해서 사용해야함**
(2) 교 착 상 태 (DeadLock)	한정된 자원을 여러 쓰레드에서 사용하려고 할때, 자원을 얻지 못해 다음 처리를 위한 진행을 못하고 있는 상태	**(1)** (단순한 처리방법으로) **시리얼큐로 해결 가능** **(2) 세마포어 등 제한된 리소스 순서 조심히 사용/ 객체 등 설계시 주의**
(3) 우선순위 뒤바뀜 (Priority Inversion)	다양한 경우에서, 작업의 Qos(서비스 품질)이 뒤바뀌어서 작업이 진행되는 경우	**(1) 1차적으로, GCD가 자원을 점유하고 있는 낮은 우선순위 작업의 우선순위를 높여서 처리.** **(2) 공유된 자원에 대한 접근시 동일한 Qos를 사용해야함.**

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

8. (심화) Lazy var와 관련된 이슈들 (또 다른 경쟁 상황의 예시)

lazy var

참고 글:

https://medium.com/what-i-talk-about-when-i-talk-about-ios-developmen/the-thread-safety-of-lazy-variables-in-swift-b20184ef5a38

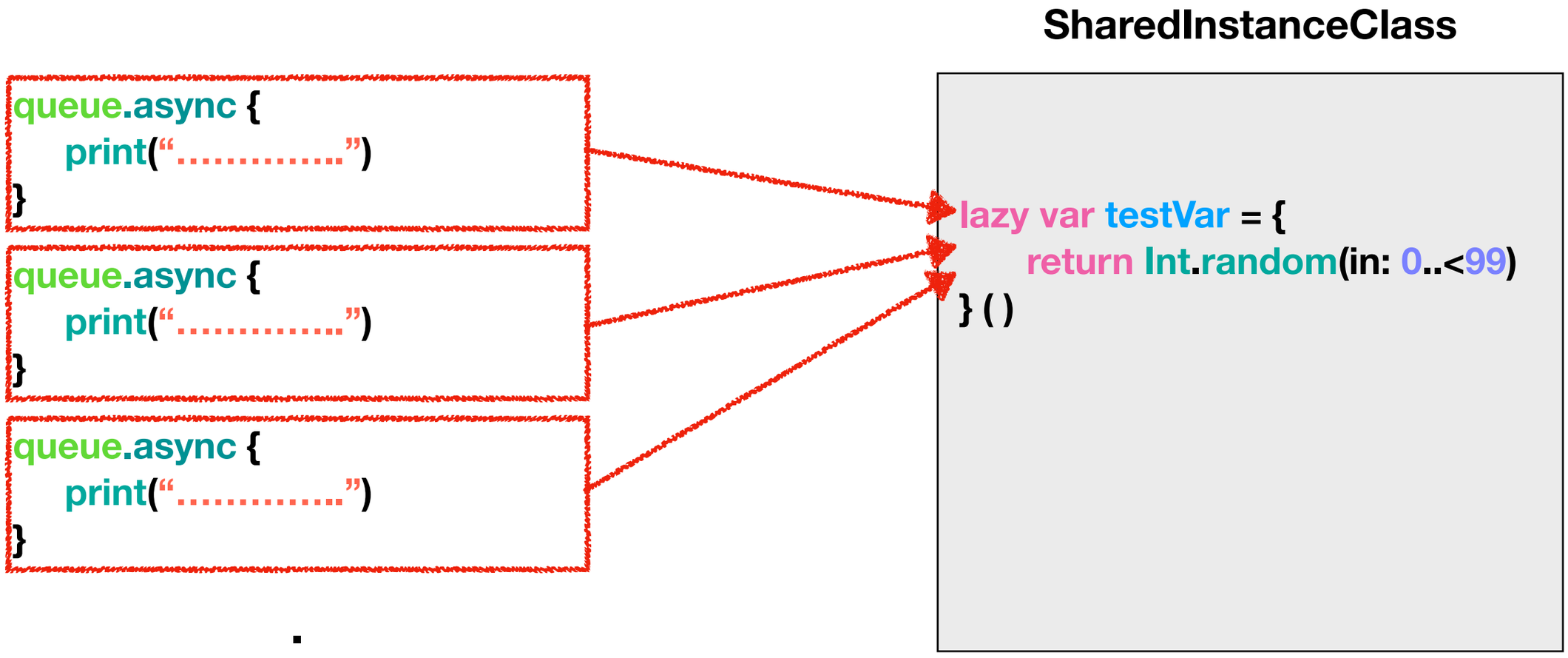
폴더 7.LazyVar이슈(번역본pdf)

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

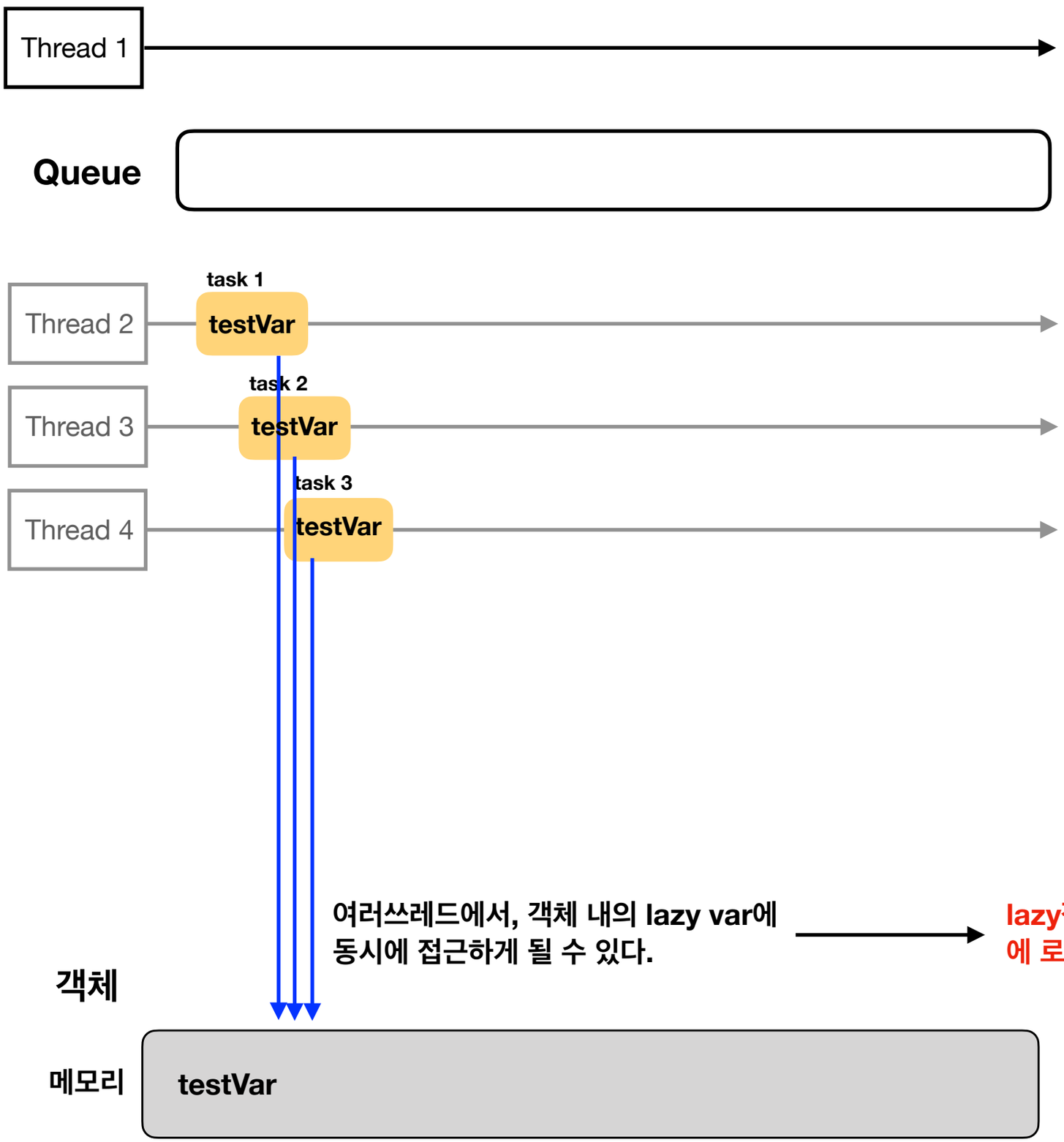
8.LazyVar

testVar변수가 여러개 생긴 상황



Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

lazy var 1번 케이스

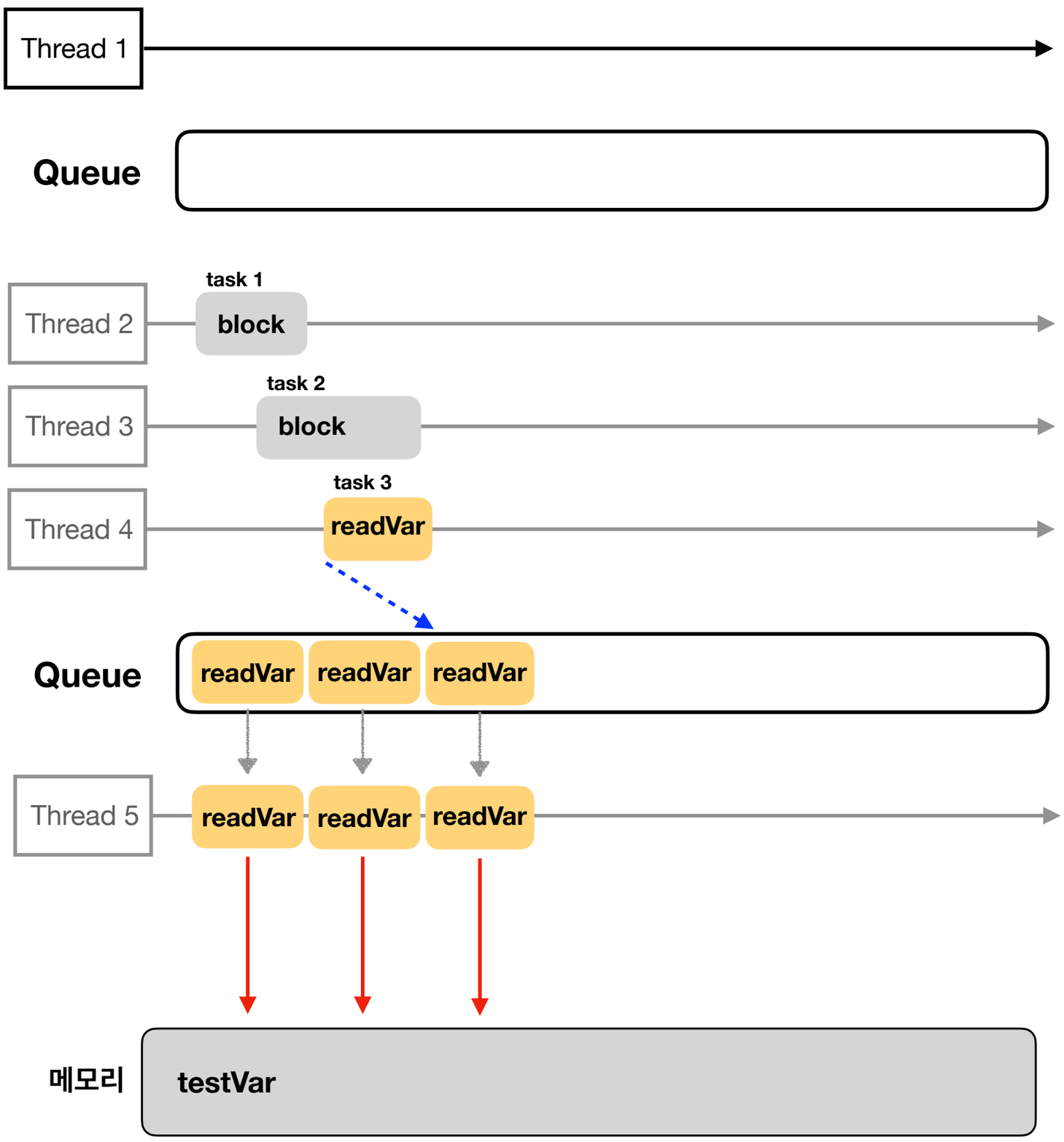


Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

8.LazyVar

lazy var 2번 케이스



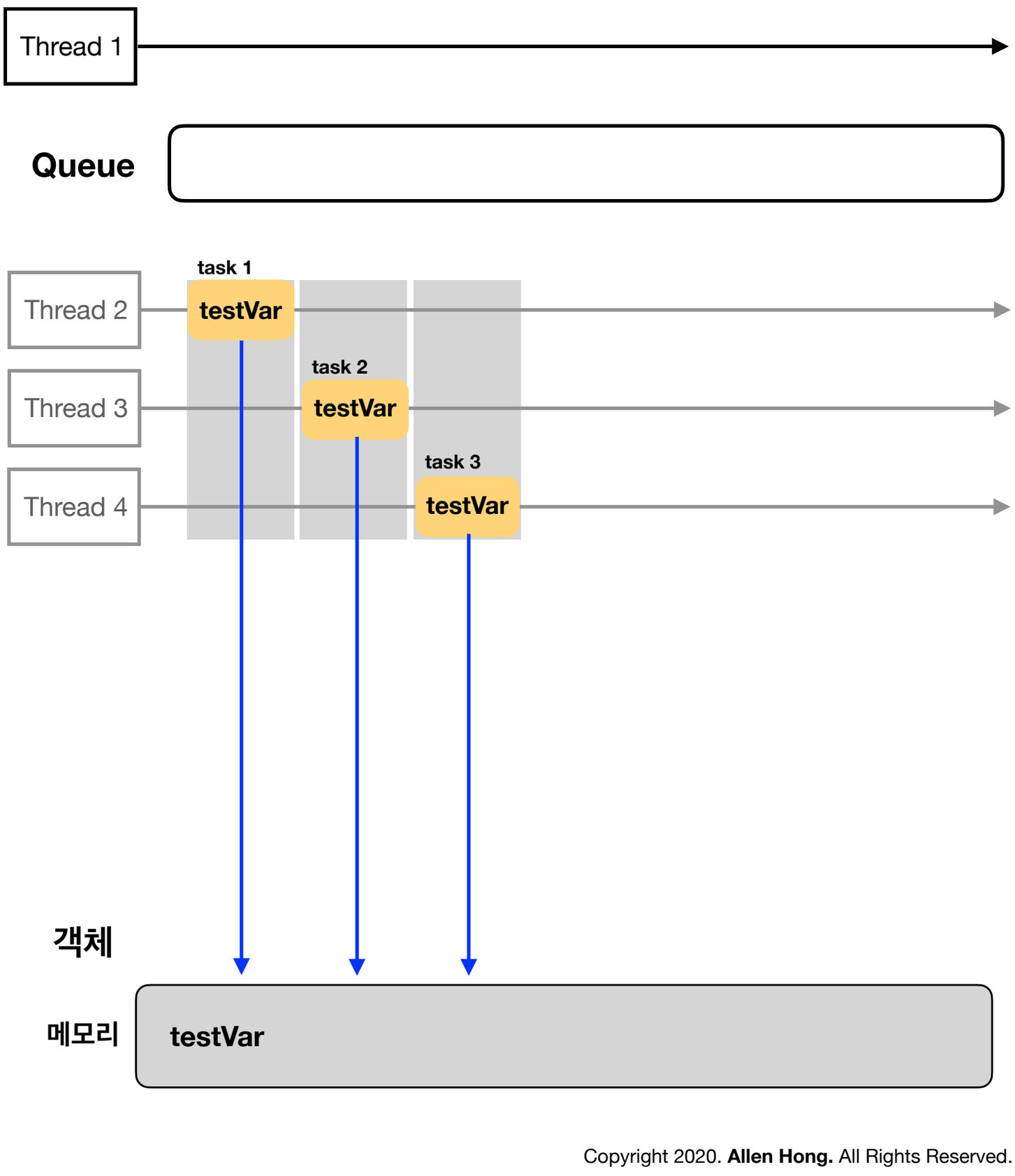
(1) serial큐로 해결

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

8.LazyVar

lazy var 3번 케이스



(2) 디스패치 배리어 작업으로 해결

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

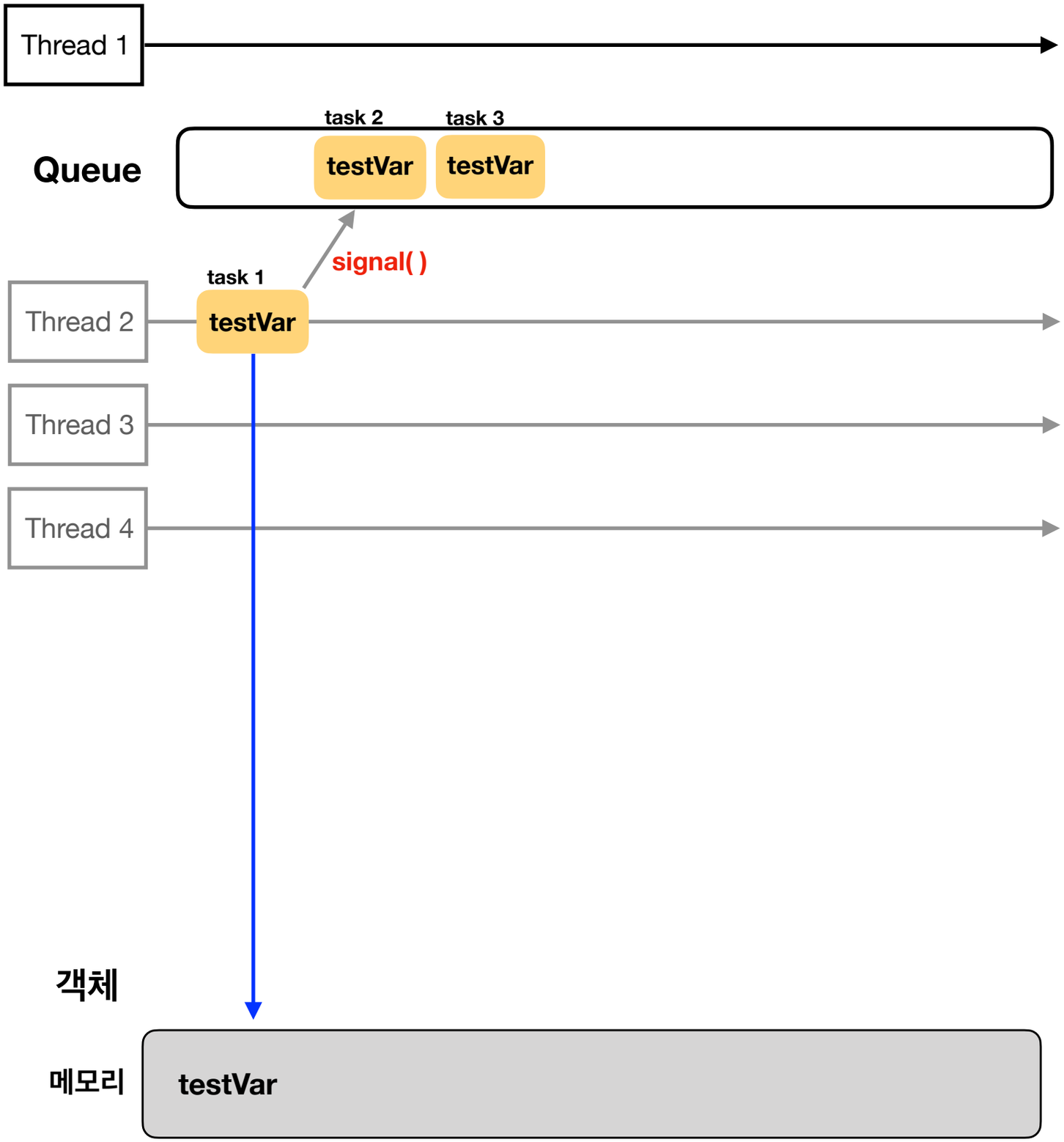
실제 코드예시

8.LazyVar

lazy var

4번 케이스

let semaphore = DispatchSemaphore(value: 1)



(3) 디스패치 세마포어로 한번에
한 작업만 일할 수 있도록해서 해결

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

실제 코드예시

8.LazyVar

<Lazy Var의 Thread-safe 처리 방법>

(1) 명확하게, lazy 변수 생성 후, 작업

(2) 시리얼큐 + Sync로 작업

(3) Dispatch Barrier로 작업

(4) 세마포어 이용, 작업의 동시 실행 갯수를 제한하기

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com

GCD관련 총정리

GCD

1. GCD/Operation에 앞서

1) 왜 동시성(Concurrent) 프로그래밍이 필요할까?

2) 간단한 GCD/Operation 소개

3) Synchronous(동기) VS Asynchronous(비동기)

4) Serial(직렬) VS Concurrent(동시)

2. 디스패치큐(GCD)의 종류와 특성

1) 메인 큐

2) 글로벌큐와 Qos의 이해

3) 프라이빗(커스텀) 큐

3. 디스패치큐(GCD) 사용시 주의해야할 사항

1) 반드시 메인큐에서 처리해야하는 작업

2) sync 메서드에 대한 주의사항

3) weak, strong캡처 주의

4) 컴플리션핸들러의 존재이유

5) 동기적 함수를 비동기함수 처럼 만드는 방법

4. 디스패치 그룹

1) 디스패치 그룹의 개념

2) 디스패치 그룹의 사용
- 디스패치 그룹과 동기함수
- 디스패치 그룹과 비동기함수

3) (참고) Dispatch WorkItem

4) (심화) Semaphore의 이해

5. (참고) GCD 프로젝트 살펴보기

6. (심화) 동시성과 관련된 문제

1) Race Condition (경쟁상황)

2) Deadlocks (교착상태)

3) Priority Inversion (우선 순위의 뒤바뀜)

7. (심화) Thread-safe한 코드의 구현과 방법

1) TSan 사용법

2) 시리얼큐와 sync

3) 디스패치 배리어(Dispatch Barrier)

4) 객체 설계 시 주의할 점

8. (심화) Lazy var와 관련된 이슈들

감사합니다

강의주소: https://www.inflearn.com/course/iOS-Concurrency-GCD-Operation#

Copyright 2020. **Allen Hong.** All Rights Reserved. hwp0314@gmail.com